TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

BMCS3183 ADVANCED DATABASE MANAGEMENT

Assignment

Semester 202505

Programme (Year, Sem & Group)	:	**RDS3S1G4**
Date Submitted	:	**01/10/2025**

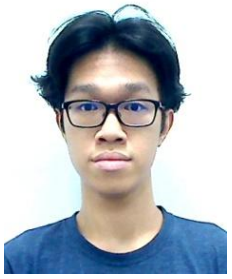
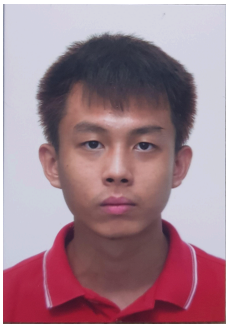
Team members:

No	Name (Block Letters - Alphabetical Order)	Student ID	Marks
1	CHIN YIN ERN	24WMR08855	
2	EDISON CHAN YOONG QI	24WMR08859	
3	KELLY TAN JIE LI	24WMR08867	
4	KENNY LOH KIT YI	24WMR08868	
5	KHIEW JIT CHEN	24WMR08869	

Declaration

We confirm that we have read and shall comply with all the terms and conditions of TAR UMT's plagiarism policy.

We declare that this assignment is free from all forms of plagiarism and for all intents and purposes is my own properly derived work.

				
Name: CHIN YIN ERN	Name: EDISON CHAN YOONG QI	Name: KELLY TAN JIE LI	Name: KENNY LOH KIT YI	Name: KHIEW JIT CHEN
Signature: *Chin*	Signature: *Chan*	Signature: *Kelly*	Signature: *Kenny*	Signature: *KHIEW*
Date:07/09/2025	Date: 8/1/2025	Date:07/09/2025	Date:07/09/2025	Date:07/09/2025

*Note: Using rea1 photo.

Assignment Assessment Form

No	**Member Name** (Alphabetical order)	**Programme** (Year-Semester-Group)
1.	CHIN YIN ERN	RDS3 S1 G4
2.	EDISON CHAN YOONG QI	RDS3 S1 G4
3.	KELLY TAN JIE LI	RDS3 S1 G4
4.	KENNY LOH KIT YI	RDS3 S1 G4
5.	KHIEW JIT CHEN	RDS3 S1 G4

Task No.	Task Descriptions	Weightage	Criteria	1	2	3	4	5	Comment
1 (CLO 1)	Entity Relationship Diagram	10%	● A complete ER data model in 3rd Normal Form. ● All primary keys, foreign keys, relationships and attributes must be clearly shown. ● Standalone system design will be given lower marks compare to online system design.						
2 (CLO 1)	Data Definition (DDL)	10%	● Relevant integrity constraints to ensure database integrity must be included. ● Necessary check constraints and default values to enforce business rules should also be included.						
3 (CLO 1)	Data records	5%	● Sufficient quality data records must be created for each table.						
4 (CLO 3)	Queries	5%	● Each team member is to design and produce two quality and useful queries for decision making at any two different management levels: strategic, tactical or operational. ● Single table queries are not allowed. ● Multiple table queries with aggregate functions (where appropriate) must be used. ● View(s) is/are ought to be incorporated, where necessary. ● These 2 queries cannot be used directly for report body.						
		5%							
5 (CLO 2)	Stored Procedures	5%	● Each team member is to design and create two stored procedures that cater for the use case scenarios for the system. ● Quality, usefulness and importance of the stored procedures must be considered.						
		5%							

6 (CLO 2)	Triggers	5% 5%	• Each team member is to design and create two triggers that enforces system-wide business rules and policies. • Quality, usefulness and functionality of the triggers must be considered.						
7 (CLO 3)	Reports	10% 10%	• Each team member is to create two procedures to generate two reports (summary, detail and on demand basis reports) for the company. • Parameter value(s) should be passed to the procedure, where necessary. • Cursor must be used in report generation (prefer nested cursors or multiple cursors). • Usefulness, complexity and presentation of the reports must be considered.						
8 (CLO 2)	Extra Effort	10%	• Sequences, indexes, views, output formatting, exceptions and/or functions, must be incorporated where necessary. • Usefulness and application of each of the above to enhance the efficiency and effectiveness of the information system must be considered. • Linking of all the tasks of every team member in creating a quality information system must be considered.						
9 (CLO 3)	Presentation & Participation	15%	• Run the single script file from Task 2 – 3 to create the new system database on the lab server. • Individual presentation on Task 4 – 8. • Q & A • Actively participate in class discussion.						
Assignment Marks / 100									

*CLO 1: Develop the relational database system with the appropriate integrity constraints and security control. (P3, PLO3)
*CLO 2: Design the solutions to issues pertaining to database efficiency and effectiveness using appropriate techniques. (C4, PLO2)
*CLO 3: Extract information from the database using efficient SQL query construct. (C4, PLO6)

Table of Content

Chapter 1: Background of the System **8**
1.1 Type of System 8
1.2 List of Modules 8
Chapter 2: Entity Relationship Modelling **10**
2.1 Business Rules and Assumptions 10
2.2 Entity Relationship Diagram 11
Chapter 3: Data Definition **12**
3.1 Create Table Statement 12
3.2 Sample Records (limit 10 rec per table) 18
Chapter 4: Queries, Procedures, Triggers and Reports **20**
4.1 CHIN YIN ERN 20
4.1.1 Query 1: Rank Member with Net Spending 20
4.1.2 Query 2: Membership Refund Status by Year 22
4.1.3 Procedure 1: Member Purchase Ticket 24
4.1.4 Procedure 2: Member Refund Ticket 26
4.1.5 Trigger 1: Prevent members refund tickets after scheduled departure time 28
4.1.6 Trigger 2: Audit Log for Ticket Fare 30
4.1.7 Report 1: Monthly Refunds Summary Report 33
4.1.8 Report 2: Member Booking Summary Report 36
4.2 EDISON CHAN YOONG QI 39
4.2.1 Query 1: Measure Revenue Generated by Each Bus Company Based on Their Destinations 39
4.2.2 Query 2: Ranking Profit Generated From Each Bus Company 41
4.2.3 Procedure 1: Member Registration 43
4.2.4 Procedure 2: Dynamically Insert Into Part Based on Part_Order 45
4.2.5 Trigger 1: Audit Log for Staff 47
4.2.6 Trigger 2: Update Ticket Based on Extension Table 49
4.2.7 Report 1: A Summary Report of The Number of Trips Each Drivers Has Driven For Each Company 52
4.2.8 Report 2: Summary Report of Shop Rental Income 55
4.3 KELLY TAN JIE LI 58
4.3.1 Query 1: Ranked Bus Reliability 58
4.3.2 Query 2: Yearly Maintenance Spending Trends Across Bus Companies 59
4.3.3 Procedure 1: Add New Maintenance Record for a Bus 60
4.3.4 Procedure 2: Search Bus Schedules with Passenger Details 63
4.3.5 Trigger 1: Update Bus Status and Cancel Conflicting Schedules During Maintenance 66
4.3.6 Trigger 2: Audit Maintenance Status Changes 69
4.3.7 Report 1: Bus Company Maintenance Report 71
4.3.8 Report 2: Bus Schedule Summary Report 75
4.4 KENNY LOH KIT YI 79

4.4.1 Query 1: Ranking Driver With Most Schedules Allocated for the Past 3 Months 79
4.4.2 Query 2: Ranking Staff With Most Rent Collected And Dominant Shop Type 80
4.4.3 Procedure 1: Adding Driver Allocation while Ensuring no Multiple Allocation on Same Date 82
4.4.4 Procedure 2: Adding Collected Rent with Validation and Auto Assign ID and Date 82
4.4.5 Trigger 1: Audit Log for Driver Allocation 84
4.4.6 Trigger 2: Update Driver Allocation Shift Based on Schedule 85
4.4.7 Report 1: Driver Schedule Report 86
4.4.8 Report 2: Staff Rent Collection Report 88
4.5 KHIEW JIT CHEN 91
4.5.1 Query 1: Route Occupancy Rate 91
4.5.2 Query 2: Monthly Part Orders and Cost Trend Analysis 94
4.5.3 Procedure 1: Full refund for bus schedule cancellation by any bus company 96
4.5.4 Procedure 2: Staff Shifting Allocation 98
4.5.5 Trigger 1: Audit log on Part table changes 100
4.5.6 Trigger 2: Real-time Part Stock and Maintenance Cost Update Trigger 102
4.5.7 Report 1: MONTHLY PARTS USAGE AND COST ANALYSIS REPORT 103
4.5.8 Report 2: SUPPLIER CONTRIBUTION AND VALUE ANALYSIS REPORT 106
Chapter 5: Extra Effort Highlight 110
5.1 CHIN YIN ERN 110
5.1.1 Views 110
5.1.2 Indexes 110
5.1.3 User Defined Functions 110
5.1.4 User Defined Exceptions 110
5.1.5 Sequence 112
5.1.6 Output Formatting 112
5.2 EDISON CHAN YOONG QI 113
5.2.1 Views For Query 1 and Query 2 113
5.2.2 Indexes 113
5.2.3 User Defined Functions 114
5.2.4 User Defined Exceptions 114
5.2.5 Sequence 115
5.2.6 Output Formatting 115
5.3 KELLY TAN JIE LI 116
5.3.1 Views 116
5.3.2 Indexes 116
5.3.3 User Defined Functions 116
5.3.4 User Defined Exceptions 116
5.3.5 Sequence 118
5.3.6 Output Formatting 118
5.4 KENNY LOH KIT YI 119

5.4.1 Views 119
5.4.2 Indexes 119
5.4.3 User Defined Functions 119
5.4.4 User Defined Exceptions 119
5.4.5 Sequence 120
5.4.6 Output Formatting 120
5.5 KHIEW JIT CHEN 122
5.5.1 Views 122
5.5.2 Indexes 122
5.5.3 User Defined Functions 122
5.5.4 User Defined Exceptions 127
5.5.5 Sequence creation 129
5.5.6 Output Formatting 129

Chapter 1: Background of the System

1.1 Type of System

The type of system is a Bus Station Management System. It allows staff to interact with the system to manage shops, view bus maintenance records and view total ticket sales. Members can register to the system to use the bus services. Staff can view rent collected, shop owners and their own information.

1.2 List of Modules

The list of modules which are available in the system are as follows:

1. Member Registration
    a. With a one time fee of RM 10
    b. Track membership tier (bronze, silver, gold)
2. Ticket Booking
    a. Member are able to purchase ticket online
    b. Member can cancel ticket 2 days in advance with 70% refund
3. Ticket Extension
    a. Extension of 2 days in advance with additional charge of RM 5
    b. Cannot extend refunded ticket or request an extension 2 days before the scheduled trip
4. Ticket Refund
    a. Full refund when trip is cancelled by the bus company
5. Company Management System
    a. Manage staff, shops, buses, bus platforms, bus companies, bus drivers and bus schedules.
    b. Maintenance, repair, tyre replace and washing services
    c. Parts and goods ordering.
6. Searching
    a. Advanced search options for bus schedules

Mandatory Module

Module	PIC	Done
Member registration	Edison Chan Yoong Qi	Y
Allow members to purchase/booking ticket(s) online.	Chin Yin Ern	Y
Allow cancellation of ticket before 2 days in advanced with 70% refund.	Chin Yin Ern	Y
Allow extension of ticket before 2 days in advanced at current ticket price with additional charge RM 5.00.	Edison Chan Yoong Qi	Y
Full refund for bus schedule cancellation by any bus company.	Khiew Jit Chen	Y
Manage staffs (counter staffs, cleaners and others), shops (food court stalls and shop lots), buses, bus platforms, bus companies, bus drivers (hired by each bus company) and bus schedules.	Kenny Loh Kit Yi	Y
Given maintenance, repair, tyre replace and washing services for the buses.	Kelly Tan Jie Li	Y
Enable advanced search options for bus schedules (e.g., search by bus, bus company, destination, price, time, date and others).	Kelly Tan Jie Li	Y

Chapter 2: Entity Relationship Modelling

2.1 Business Rules and Assumptions

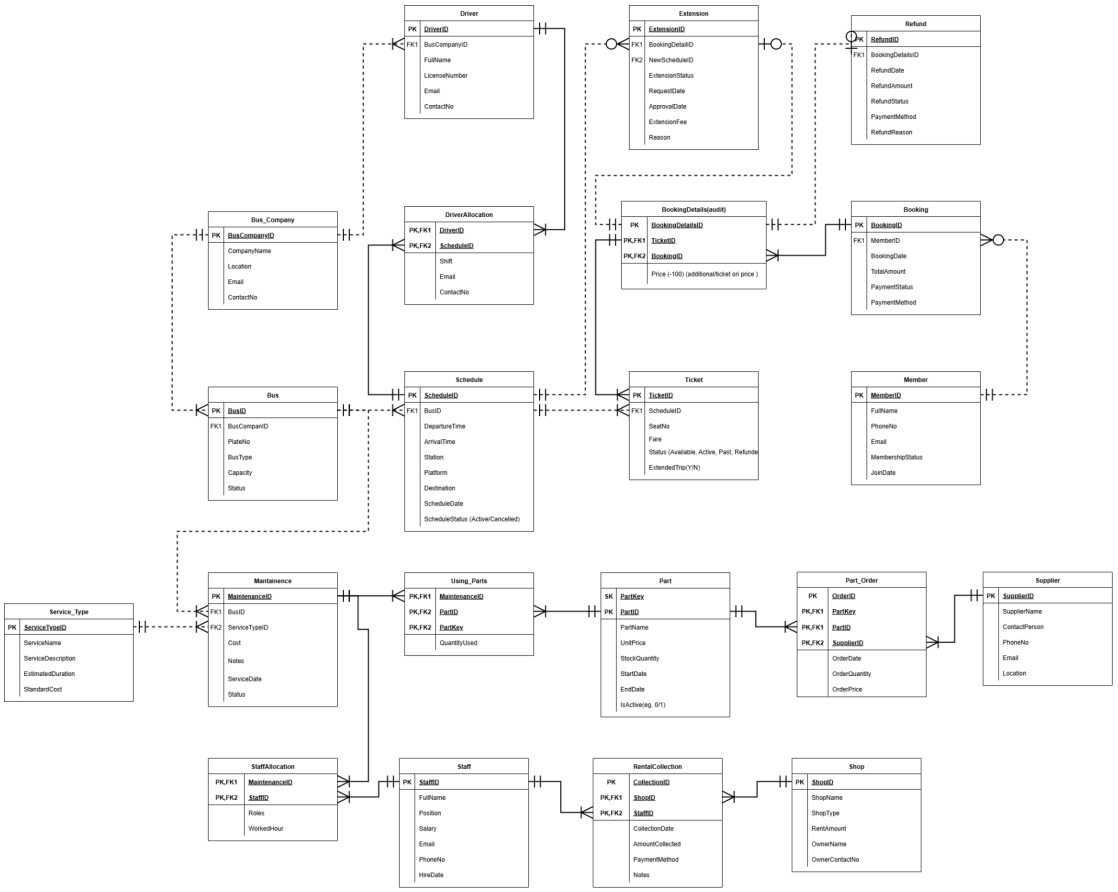
The Bus management system has the following business rules:

1. One member can make zero to many bookings
2. One member can only make 1 refund per ticket
3. One member can can only extend a ticket once
4. One booking can have many booking details
5. One ticket can only have zero or one extension
6. One ticket can only have zero or one refund
7. One schedule can have one to many tickets
8. One schedule can have one to many extensions
9. One bus can have one to many schedules
10. One schedule can have one to many drivers
11. One bus company can have one to many buses
12. One bus company can have one to many drivers
13. One bus can have one to many maintenance
14. One maintenance can have one to many staffs
15. One maintenance can use many parts (one to many)
16. One part can have one to many part orders
17. One part supplier can have one to many part orders
18. One staff can have one to many rental collections
19. One shop can have one to many rental collections

Assumptions:

1. If a member exists in the Member table, it means that they have paid RM 10 for the registration fees.
2. Tickets can only be extended once after purchase.
3. Only registered members can book tickets.
4. Tickets can only be booked if the status is “Available”.
5. Tickets that were refunded can be booked again only if the departure time is at least 2 days away from the current date.
6. Payment assumed to be complete at the time of booking.
7. Members can’t refund tickets that are not associated with them.
8. Members can’t refund twice for the same ticket.
9. Refund payment method used is the same as the booking payment method made
10. The refund amount is set to RM0 when the refund is rejected.

2.2 Entity Relationship Diagram



Link for easier viewing: [ERD Draw.io]

Chapter 3: Data Definition

3.1 Create Table Statement

```
DROP TABLE RentalCollection;
DROP TABLE Shop;
DROP TABLE StaffAllocation;
DROP TABLE Using_Parts;
DROP TABLE Part_Order;
DROP TABLE Maintenance;
DROP TABLE Extension;
DROP TABLE Refund CASCADE CONSTRAINTS;
DROP TABLE BookingDetails;
DROP TABLE Ticket;
DROP TABLE Booking;
DROP TABLE Schedule CASCADE CONSTRAINTS;
DROP TABLE Bus;
DROP TABLE DriverAllocation;
DROP TABLE Part;

DROP TABLE Staff;
DROP TABLE Supplier;
DROP TABLE Service_Type;
DROP TABLE Member;
DROP TABLE Driver;
DROP TABLE Bus_Company;

-- Data
-- FORMATTING
SET linesize 1000
SET pagesize 100
SET SERVEROUTPUT ON;

-- TO PREVENT UNINTENTIONAL LINEBREAKS
SET DEFINE OFF;

-- Bus Company
CREATE TABLE Bus_Company (
    BusCompanyID   VARCHAR2(8) PRIMARY KEY NOT NULL,
    CompanyName    VARCHAR2(50) NOT NULL,
    Location       VARCHAR2(30) NOT NULL,
    Email          VARCHAR2(40) NOT NULL,
    ContactNo      VARCHAR2(12) NOT NULL,
    CONSTRAINT BusCompanyID_chk CHECK
(REGEXP_LIKE(BusCompanyID, '^BC[0-9]{3}$')),
    CONSTRAINT CompanyName_chk CHECK
(REGEXP_LIKE(CompanyName, '^[A-Za-z ]+$')),
    CONSTRAINT Email_chk CHECK (REGEXP_LIKE(Email,
'.*@.*\..*')),
    CONSTRAINT ContactNo_chk CHECK (REGEXP_LIKE(ContactNo,
'^[0-9]{3}[- ]?[0-9]{7}$'))
);
```

```
-- Bus
CREATE TABLE Bus (
    BusID          VARCHAR2(8) PRIMARY KEY ,
    BusCompanyID   VARCHAR2(8) NOT NULL,
    PlateNo        VARCHAR2(8) NOT NULL,
    BusType        VARCHAR2(20) NOT NULL,
    Capacity       NUMBER NOT NULL,
    Status         VARCHAR2(20) NOT NULL,
    FOREIGN KEY (BusCompanyID) REFERENCES
Bus_Company(BusCompanyID),
    CONSTRAINT BusID_chk CHECK (REGEXP_LIKE(BusID,
'^B[0-9]{4}$')),
    CONSTRAINT PlateNo_chk CHECK (REGEXP_LIKE(PlateNo,
'^[a-zA-Z]{3}[0-9]{4}$'))
);

-- Schedule
CREATE TABLE Schedule (
    ScheduleID     VARCHAR2(10) PRIMARY KEY NOT NULL,
    BusID          VARCHAR2(10) NOT NULL,
    DepartureTime  DATE NOT NULL,
    ArrivalTime    DATE NOT NULL,
    Station        VARCHAR2(50) NOT NULL,
    Platform       VARCHAR2(20) NOT NULL,
    Destination    VARCHAR2(50) NOT NULL,
    ScheduleDate   DATE NOT NULL,
    ScheduleStatus VARCHAR2(20) NOT NULL, --
Active/Cancelled
    FOREIGN KEY (BusID) REFERENCES Bus(BusID)
);

-- Driver
CREATE TABLE Driver (
    DriverID       VARCHAR2(10) PRIMARY KEY NOT NULL,
    BusCompanyID   VARCHAR2(8) NOT NULL,
    FullName       VARCHAR2(50) NOT NULL,
    LicenseNumber  VARCHAR2(10) NOT NULL,
    Email          VARCHAR2(40) NOT NULL,
    ContactNo      VARCHAR2(12) NOT NULL,
    FOREIGN KEY (BusCompanyID) REFERENCES
Bus_Company(BusCompanyID),
    CONSTRAINT DriverID_chk CHECK (REGEXP_LIKE(DriverID,
'^D[0-9]{4}$')),
    CONSTRAINT FullName_chk CHECK (REGEXP_LIKE(FullName,
'^[A-Za-z ]+$')),
    CONSTRAINT LicenseNumber_chk CHECK
(REGEXP_LIKE(LicenseNumber, '^PSVE[0-9]{4}$'))
);

-- Driver Allocation
CREATE TABLE DriverAllocation (
    DriverID       VARCHAR2(10) NOT NULL,
    ScheduleID     VARCHAR2(10) NOT NULL,
    Shift          VARCHAR2(10) DEFAULT '-',
    PRIMARY KEY (DriverID, ScheduleID),
```

```
    FOREIGN KEY (DriverID) REFERENCES Driver(DriverID),
    FOREIGN KEY (ScheduleID) REFERENCES
Schedule(ScheduleID),
    CONSTRAINT chk_DriverID CHECK (REGEXP_LIKE(DriverID,
'^D[0-9]{4}$')),
    CONSTRAINT chk_ScheduleID CHECK
(REGEXP_LIKE(ScheduleID, '^SCH[0-9]{4}$'))
);

-- Member
CREATE TABLE Member (
    MemberID        VARCHAR2(8) PRIMARY KEY NOT NULL,
    FullName        VARCHAR2(50) NOT NULL,
    PhoneNo         VARCHAR2(12) NOT NULL,
    Email           VARCHAR2(40) NOT NULL,
    MembershipStatus VARCHAR2(20) NOT NULL,
    JoinDate        DATE,
    CONSTRAINT MemberID_chk CHECK (REGEXP_LIKE(MemberID,
'^M[0-9]{4}$'))
);

-- Booking
CREATE TABLE Booking (
    BookingID       VARCHAR2(8) PRIMARY KEY NOT NULL,
    MemberID        VARCHAR2(8) NOT NULL,
    BookingDate     DATE NOT NULL,
    TotalAmount     NUMBER(10, 2) NOT NULL,
    PaymentStatus   VARCHAR2(20) NOT NULL,
    PaymentMethod   VARCHAR2(30) NOT NULL,
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    CONSTRAINT BookingID_chk CHECK (REGEXP_LIKE(BookingID,
'^BK[0-9]{4}$'))
);

-- Ticket
CREATE TABLE Ticket (
    TicketID        VARCHAR2(10) PRIMARY KEY NOT NULL,
    ScheduleID      VARCHAR2(10) NOT NULL,
    SeatNo          VARCHAR2(5) NOT NULL,
    Fare            NUMBER(8, 2) NOT NULL,
    Status          VARCHAR2(20) NOT NULL, -- Refunded,
Active, Past
    ExtendedTrip    CHAR(1) NOT NULL,
    FOREIGN KEY (ScheduleID) REFERENCES
Schedule(ScheduleID),
    CONSTRAINT chk_TicketID CHECK (REGEXP_LIKE(TicketID,
'^TCK[0-9]{5}$'))
);

-- BookingDetails
CREATE TABLE BookingDetails (
    BookingDetailsID VARCHAR2(10) PRIMARY KEY NOT NULL,
    TicketID         VARCHAR2(10) NOT NULL,
    BookingID        VARCHAR2(10) NOT NULL,
    Price            NUMBER(10, 2) NOT NULL,
```

```
    FOREIGN KEY (TicketID) REFERENCES Ticket(TicketID),
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
    CONSTRAINT chk_BookingDetailsID CHECK
(REGEXP_LIKE(BookingDetailsID, '^BD[0-9]{6}$'))
);

-- Extension
CREATE TABLE Extension (
    ExtensionID      VARCHAR2(10) PRIMARY KEY NOT NULL,
    BookingDetailsID VARCHAR2(10),
    NewScheduleID    VARCHAR2(10),
    NewTicketID      VARCHAR(10),
    ExtensionStatus  VARCHAR2(20),
    RequestDate      DATE,
    ApproveDate      DATE,
    ExtensionFee     NUMBER(8, 2) DEFAULT 0,
    Reason           VARCHAR2(100),
    FOREIGN KEY (BookingDetailsID) REFERENCES
BookingDetails(BookingDetailsID),
    FOREIGN KEY (NewScheduleID) REFERENCES
Schedule(ScheduleID)
);

-- Refund
CREATE TABLE Refund (
    RefundID        VARCHAR2(10) PRIMARY KEY NOT NULL,
    BookingDetailsID VARCHAR2(10) NOT NULL,
    RefundDate      DATE NOT NULL,
    RefundAmount    NUMBER(10, 2) NOT NULL,
    RefundStatus    VARCHAR2(20) NOT NULL,
    PaymentMethod   VARCHAR2(30) NOT NULL,
    RefundReason    VARCHAR2(100) NOT NULL,
    FOREIGN KEY (BookingDetailsID) REFERENCES
BookingDetails(BookingDetailsID)
);

-- Service_Type
CREATE TABLE Service_Type (
    ServiceTypeID       VARCHAR2(8) PRIMARY KEY NOT NULL,
    ServiceName         VARCHAR2(30) NOT NULL,
    ServiceDescription  VARCHAR2(50) NOT NULL,
    EstimatedDuration   NUMBER NOT NULL,
    StandardCost        NUMBER(10, 2) NOT NULL,
    CONSTRAINT ServiceTypeID_chk CHECK
(REGEXP_LIKE(ServiceTypeID, '^SV[0-9]{3}$')),
    CONSTRAINT ServiceName_chk CHECK
(REGEXP_LIKE(ServiceName, '^[A-Za-z ]+$'))
);

-- Maintenance
CREATE TABLE Maintenance (
    MaintenanceID   VARCHAR2(10) PRIMARY KEY NOT NULL,
    BusID           VARCHAR2(10) NOT NULL,
    ServiceTypeID   VARCHAR2(10) NOT NULL,
    Cost            NUMBER(10, 2) NOT NULL,
```

```
    Notes           VARCHAR2(50) DEFAULT '-',
    ServiceDate     DATE NOT NULL,
    Status          VARCHAR2(20) NOT NULL,
    FOREIGN KEY (BusID) REFERENCES Bus(BusID),
    FOREIGN KEY (ServiceTypeID) REFERENCES
Service_Type(ServiceTypeID),
    CONSTRAINT chk_MaintenanceID CHECK
(REGEXP_LIKE(MaintenanceID, '^MT[0-9]{5}$'))
);

-- Supplier
CREATE TABLE Supplier (
    SupplierID      VARCHAR2(8) PRIMARY KEY NOT NULL,
    SupplierName    VARCHAR2(30) NOT NULL,
    ContactPerson   VARCHAR2(30) NOT NULL,
    PhoneNo         VARCHAR2(12) NOT NULL,
    Email           VARCHAR2(40) NOT NULL,
    Location        VARCHAR2(40) NOT NULL,
    CONSTRAINT SupplierID_chk CHECK
(REGEXP_LIKE(SupplierID, '^SUP[0-9]{3}$')),
    CONSTRAINT SupplierName_chk CHECK
(REGEXP_LIKE(SupplierName, '^[A-Za-z &]+$')),
    CONSTRAINT PhoneNo_chk CHECK (REGEXP_LIKE(PhoneNo,
'^[0-9]{3}[- ]?[0-9]{7}$'))
);

-- Part
CREATE TABLE Part (
    PartKey         NUMBER NOT NULL,
    PartID          VARCHAR2(8) NOT NULL,
    PartName        VARCHAR2(50) NOT NULL,
    StockQuantity   NUMBER NOT NULL,
    UnitPrice       NUMBER(10,2) NOT NULL,
    StartDate       DATE NOT NULL,
    EndDate         DATE DEFAULT '31-DEC-9999',
    IsActive        Number(1) DEFAULT 1,
    PRIMARY KEY (PartKey, PartID),
    CONSTRAINT PartID_chk CHECK (REGEXP_LIKE(PartID,
'^PRT[0-9]{3}$')),
    CONSTRAINT PartName_chk CHECK (REGEXP_LIKE(PartName,
'^[A-Za-z ]+$')),
    CONSTRAINT IsActive_chk CHECK (IsActive IN (0, 1))
);
COLUMN UnitPrice FORMAT A15;

-- Using_Parts
CREATE TABLE Using_Parts (
    MaintenanceID   VARCHAR2(10) NOT NULL,
    PartKey         NUMBER NOT NULL,
    PartID          VARCHAR2(8) NOT NULL,
    QuantityUsed    NUMBER NOT NULL,
    PRIMARY KEY (MaintenanceID, PartID),
    FOREIGN KEY (MaintenanceID) REFERENCES
Maintenance(MaintenanceID),
    FOREIGN KEY (PartKey, PartID) REFERENCES Part(PartKey,
```

```
PartID),
    CONSTRAINT chk_MaintenanceID_UsingParts CHECK
(REGEXP_LIKE(MaintenanceID, '^MT[0-9]{5}$')),
    CONSTRAINT chk_PartID CHECK (REGEXP_LIKE(PartID,
'^PRT[0-9]{3}$'))
);

-- Part_Order
CREATE TABLE Part_Order (
    OrderID         VARCHAR2(8) PRIMARY KEY NOT NULL,
    OrderDate       DATE NOT NULL,
    PartKey         NUMBER NOT NULL,
    PartID          VARCHAR2(8) NOT NULL,
    SupplierID      VARCHAR2(8) NOT NULL,
    OrderQuantity   NUMBER NOT NULL,
    Price           Number(10,2),
    FOREIGN KEY (PartKey, PartID) REFERENCES Part(PartKey,
PartID),
    FOREIGN KEY (SupplierID) REFERENCES
Supplier(SupplierID),
    CONSTRAINT chk_OrderID CHECK (REGEXP_LIKE(OrderID,
'^PO[0-9]{4}$'))
);

-- Staff
CREATE TABLE Staff (
    StaffID         VARCHAR2(8) PRIMARY KEY NOT NULL,
    FullName        VARCHAR2(35) NOT NULL,
    Position        VARCHAR2(20) NOT NULL,
    Salary          NUMBER(10, 2) NOT NULL,
    Email           VARCHAR2(40) NOT NULL,
    PhoneNo         VARCHAR2(12) NOT NULL,
    HireDate        DATE NOT NULL,
    CONSTRAINT StaffID_chk CHECK (REGEXP_LIKE(StaffID,
'^STF[0-9]{3}$'))
);

-- StaffAllocation
CREATE TABLE StaffAllocation (
    MaintenanceID   VARCHAR2(10) NOT NULL,
    StaffID         VARCHAR2(10) NOT NULL,
    Role            VARCHAR2(50) NOT NULL,
    WorkedHour      NUMBER NOT NULL,
    PRIMARY KEY (MaintenanceID, StaffID),
    FOREIGN KEY (MaintenanceID) REFERENCES
Maintenance(MaintenanceID),
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID),
    CONSTRAINT chk_MaintenanceID_StaffAlloc CHECK
(REGEXP_LIKE(MaintenanceID, '^MT[0-9]{5}$')),
    CONSTRAINT chk_StaffID CHECK (REGEXP_LIKE(StaffID,
'^STF[0-9]{3}$'))
);

-- Shop
CREATE TABLE Shop (
```

```
    ShopID          VARCHAR2(8) PRIMARY KEY NOT NULL,
    ShopName        VARCHAR2(50) NOT NULL,
    ShopType        VARCHAR2(20) NOT NULL,
    RentAmount      NUMBER(10, 2) NOT NULL,
    OwnerName       VARCHAR2(50) NOT NULL,
    OwnerContactNo  VARCHAR2(12) NOT NULL,
    CONSTRAINT ShopID_chk CHECK (REGEXP_LIKE(ShopID,
'^SHP[0-9]{3}$'))
);

-- Rental Collection
CREATE TABLE RentalCollection (
    CollectionID    VARCHAR2(8) NOT NULL,
    ShopID          VARCHAR2(8) NOT NULL,
    StaffID         VARCHAR2(8) NOT NULL,
    CollectionDate  DATE NOT NULL,
    AmountCollected NUMBER(10, 2) NOT NULL,
    PaymentMethod   VARCHAR2(30) NOT NULL,
    Notes           VARCHAR2(40) DEFAULT '-',
    PRIMARY KEY (CollectionID, ShopID, StaffID),
    FOREIGN KEY (ShopID) REFERENCES Shop(ShopID),
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID),
    CONSTRAINT chk_CollectionID CHECK
(REGEXP_LIKE(CollectionID, '^COL[0-9]{3}$')),
    CONSTRAINT chk_ShopID CHECK (REGEXP_LIKE(ShopID,
'^SHP[0-9]{3}$'))
);
```

3.2 Sample Records (limit 10 rec per table)

```
-- SERVICE TYPE
DROP SEQUENCE service_type_seq;
CREATE SEQUENCE service_type_seq
MINVALUE 1
MAXVALUE 999
START WITH 1
INCREMENT BY 1
NOCACHE;

INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Wheel
Alignment', 'Ensure vehicle performance and safety', 215,
325.75);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Tire
Rotation', 'Ensure vehicle performance and safety', 86,
438.53);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Tire
Rotation', 'Check and refill essential fluids', 168,
384.84);
```

```
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Car Wash',
'System performance optimization', 40, 320.79);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Oil Change',
'System performance optimization', 97, 264.05);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Transmission
Service', 'Standard vehicle maintenance service', 42,
449.46);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Engine
Diagnostics', 'Preventive maintenance service', 125,
393.15);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Coolant
Flush', 'Replacement of worn out components', 113, 392.93);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Brake
Inspection', 'Replacement of worn out components', 34,
236.22);
INSERT INTO Service_Type VALUES('SV' ||
TO_CHAR(service_type_seq.nextval, 'FM000'), 'Battery
Replacement', 'Check and refill essential fluids', 43,
465.18);
```

Chapter 4: Queries, Procedures, Triggers and Reports

4.1 CHIN YIN ERN

4.1.1 Query 1: Rank Member with Net Spending

Purpose: The purpose of this query is to analyse each member's net financial contribution by calculating their total booking and booking amount after refunds. It identify high-value members who generate most net revenue by spending behaviour. With this information it helps to segment customers for targeted marketing campaigns, adjust pricing strategies to increase retention among members, and flag potential refund abuse cases for stricter approval processes.

SQL Statement:

```
DROP INDEX member_name_idx;
CREATE INDEX member_name_idx ON Member(UPPER(FullName));

SET LINESIZE 120
SET PAGESIZE 50
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

TTITLE CENTER 'Ranking of Member Net Spending on ' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2

COLUMN MemberName FORMAT A30 HEADING 'Member Name'
COLUMN TotalBookings FORMAT 9999 HEADING 'Total Bookings'
COLUMN TotalBookingAmount FORMAT $999,999,999.99 HEADING 'Total Booking
Amount'
COLUMN TotalRefundAmount FORMAT $999,999,999.99 HEADING 'Total Refunds'
COLUMN NetSpend FORMAT $999,999,999.99 HEADING 'Net Spend'

CREATE OR REPLACE VIEW Member_Booking_Summary AS
SELECT
    m.MemberID,
    m.FullName AS MemberName,
    COUNT(DISTINCT b.BookingID) AS TotalBookings,
    SUM(b.TotalAmount) AS TotalBookingAmount,
    NVL(SUM(refund_summary.TotalRefund), 0) AS TotalRefundAmount,
    SUM(b.TotalAmount) - NVL(SUM(refund_summary.TotalRefund), 0) AS
NetSpend
FROM Member m
JOIN Booking b
    ON m.MemberID = b.MemberID
LEFT JOIN (
    SELECT bd.BookingID, SUM(r.RefundAmount) AS TotalRefund
    FROM BookingDetails bd
    LEFT JOIN Refund r
        ON bd.BookingDetailsID = r.BookingDetailsID
    GROUP BY bd.BookingID
) refund_summary
    ON b.BookingID = refund_summary.BookingID
GROUP BY m.MemberID, m.FullName;

SELECT *
FROM Member_Booking_Summary
ORDER BY NetSpend DESC;

CLEAR COLUMNS
CLEAR BREAKS
TTITLE OFF;
```

Sample Output:

Page: 1 Ranking of Member Net Spending on 07-09-2025

MEMBERID	Member Name	Total Bookings	Total Booking Amount	Total Refunds	Net Spend
M0006	Daniel Tan Hong Yee	10	$800.00	$70.00	$730.00
M0004	Lim Chee How	10	$800.00	$126.00	$674.00
M0001	Alicia Tan Mei Ling	10	$720.00	$84.00	$636.00
M0008	Muhammad Hafiz Bin Saad	10	$660.00	$84.00	$576.00
M0010	Goh Jia Xin	15	$580.00	$98.00	$482.00
M0009	Rachel Ong Li Wen	10	$540.00	$98.00	$442.00
M0005	Siti Aminah Bt Zulkifli	10	$480.00	$42.00	$438.00
M0002	John Lee Wei Sheng	10	$480.00	$84.00	$396.00
M0003	Nurul Izzah Binti Ahmad	10	$600.00	$238.00	$362.00
M0007	Tan Wei Xin	10	$440.00	$98.00	$342.00

10 rows selected.

4.1.2 Query 2: Membership Refund Status by Year

Purpose: The purpose of this query is to provide a yearly breakdown of ticket sales and refunds by membership status. It helps track refund trends across different membership tiers. This supports decisions on membership policy adjustments, or refine refund policies for different membership by implementing different rates of penalty for refund.

SQL Statement:

```
DROP INDEX membership_status_idx;
CREATE INDEX membership_status_idx ON Member(UPPER(MembershipStatus));

SET LINESIZE 80
SET PAGESIZE 50
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

TTITLE CENTER 'Membership Refund Status by Year on ' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2

COLUMN BookingYear       FORMAT 9999 HEADING 'Year'
COLUMN MembershipStatus  FORMAT A20 HEADING 'Membership Status'
COLUMN TotalTickets      FORMAT 999 HEADING 'Tickets Sold'
COLUMN TotalRefunded     FORMAT 999 HEADING 'Tickets Refunded'
COLUMN TotalRefundAmount FORMAT $999,999,999.99 HEADING 'Refund Amount'

CREATE OR REPLACE VIEW Membership_Refund_Yearly AS
SELECT
    EXTRACT(YEAR FROM b.BookingDate) AS BookingYear,
    m.MembershipStatus,
    COUNT(DISTINCT bd.TicketID) AS TotalTickets,
    COUNT(DISTINCT r.BookingDetailsID) AS TotalRefunded,
    NVL(SUM(r.RefundAmount),0) AS TotalRefundAmount
FROM Member m
JOIN Booking b
    ON m.MemberID = b.MemberID
JOIN BookingDetails bd
    ON b.BookingID = bd.BookingID
LEFT JOIN Refund r
    ON bd.BookingDetailsID = r.BookingDetailsID
GROUP BY EXTRACT(YEAR FROM b.BookingDate), m.MembershipStatus;

SELECT *
FROM Membership_Refund_Yearly
ORDER BY BookingYear;

CLEAR COLUMNS
CLEAR BREAKS
TTITLE OFF;
```

Sample Output:

```
SQL> @"C:\Users\user\Dropbox\My PC (LAPTOP-E4O7INQT)\Downloads\query2_neww.txt"

Index dropped.


Index created.


Session altered.


View created.


Page:   1        Membership Refund Status by Year on 07-09-2025

 Year Membership Status    Tickets Sold Tickets Refunded    Refund Amount
----- -------------------- ------------ ---------------- ----------------
 2024 Bronze                       75               23          $322.00
 2024 Gold                         77               17          $196.00
 2024 Silver                       64               18          $238.00
 2025 Bronze                       28                9           $70.00
 2025 Gold                         34               11          $126.00
 2025 Silver                       27                8           $70.00

6 rows selected.
```

4.1.3 Procedure 1: Member Purchase Ticket

Purpose: The purpose of this query is to let members purchase tickets after they register.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_book_ticket (
    v_BookingID       IN VARCHAR2,
    v_MemberID        IN VARCHAR2,
    v_TicketID        IN VARCHAR2,
    v_PaymentMethod   IN VARCHAR2
) IS
    v_price                Ticket.Fare%TYPE;
    v_count                NUMBER;
    v_new_bookingdetailsid BookingDetails.BookingDetailsID%TYPE;
    v_next_num             NUMBER;
    v_prefix               VARCHAR2(2);
    v_last_id              BookingDetails.BookingDetailsID%TYPE;
    v_departuretime        Schedule.DepartureTime%TYPE;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM Member
    WHERE MemberID = v_MemberID;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20030, 'Member ID ' || v_MemberID || '
does not exist.');
    END IF;

    BEGIN
       SELECT t.Fare, s.DepartureTime
       INTO v_price, v_departuretime
       FROM Ticket t
       JOIN Schedule s ON t.ScheduleID = s.ScheduleID
       WHERE t.TicketID = v_TicketID
         AND (
              t.Status = 'Available'
              OR (t.Status = 'Refunded' AND s.DepartureTime - SYSDATE >=
2)
             );

    EXCEPTION
       WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20020, 'Ticket not available for
booking. It may be Past, Active, or already booked.');
    END;

    SELECT MAX(BookingDetailsID) INTO v_last_id
    FROM BookingDetails;

    IF v_last_id IS NOT NULL THEN
       v_next_num := TO_NUMBER(REGEXP_SUBSTR(v_last_id, '\d+$')) + 1;
       v_prefix := REGEXP_SUBSTR(v_last_id, '^[A-Za-z]+');
    ELSE
       v_next_num := 1;
       v_prefix := 'BD';
    END IF;

    v_new_bookingdetailsid := v_prefix || LPAD(v_next_num, 6, '0');

    INSERT INTO Booking (BookingID, MemberID, BookingDate, TotalAmount,
PaymentStatus, PaymentMethod)
    VALUES (v_BookingID, v_MemberID, SYSDATE, v_price, 'Complete',
v_PaymentMethod);

    INSERT INTO BookingDetails (BookingDetailsID, TicketID, BookingID,
Price)
```

```
    VALUES (v_new_bookingdetailsid, v_TicketID, v_BookingID, v_price);

    UPDATE Ticket
    SET Status = 'Unavailable'
    WHERE TicketID = v_TicketID;

    DBMS_OUTPUT.PUT_LINE('Booking successful for Member ' || v_MemberID ||
                         ', Ticket ' || v_TicketID ||
                         ', Amount RM ' || v_price);

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(-20021, 'Booking ID already exists.');
END;
/

--Error using 'Active' ticket
EXEC prc_book_ticket('BK0101', 'M0005', 'TCK00001', 'Online Banking');

--Error using same BookingID
EXEC prc_book_ticket('BK0100', 'M0005', 'TCK00121', 'Online Banking');

--Error when member does not exists
EXEC prc_book_ticket('BK0107', 'M0016', 'TCK00291', 'Online Banking');

EXEC prc_book_ticket('BK0106', 'M0011', 'TCK00291', 'Online Banking');
```

Sample Output:

```
Procedure created.

BEGIN prc_book_ticket('BK0101', 'M0005', 'TCK00001', 'Online Banking'); END;

*
ERROR at line 1:
ORA-20020: Ticket not available for booking. It may be Past, Active, or already booked.
ORA-06512: at "TRY_ADM.PRC_BOOK_TICKET", line 36
ORA-06512: at line 1


BEGIN prc_book_ticket('BK0100', 'M0005', 'TCK00121', 'Online Banking'); END;

*
ERROR at line 1:
ORA-20021: Booking ID already exists.
ORA-06512: at "TRY_ADM.PRC_BOOK_TICKET", line 68
ORA-06512: at line 1


BEGIN prc_book_ticket('BK0107', 'M0016', 'TCK00291', 'Online Banking'); END;

*
ERROR at line 1:
ORA-20030: Member ID M0016 does not exist.
ORA-06512: at "TRY_ADM.PRC_BOOK_TICKET", line 20
ORA-06512: at line 1


Booking successful for Member M0011, Ticket TCK00291, Amount RM 20

PL/SQL procedure successfully completed.
```

4.1.4 Procedure 2: Member Refund Ticket

Purpose: The purpose of this query is to let members refund tickets after they purchase the ticket.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_request_refund (
    v_member_id     IN VARCHAR2,
    v_ticket_id     IN VARCHAR2,
    v_refund_reason IN VARCHAR2
) IS
    v_count NUMBER;
    v_bookingdetails_id BookingDetails.BookingDetailsID%TYPE;
    v_payment_method    Booking.PaymentMethod%TYPE;
    v_refund_id         Refund.RefundID%TYPE;

    MEMBER_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(MEMBER_NOT_FOUND, -20000);

    TICKET_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(TICKET_NOT_FOUND, -20001);

BEGIN
    SELECT COUNT(*) INTO v_count
    FROM Member
    WHERE MemberID = v_member_id;

    IF v_count = 0 THEN
        RAISE MEMBER_NOT_FOUND;
    END IF;

    SELECT COUNT(*) INTO v_count
    FROM BookingDetails bd
    JOIN Booking b ON bd.BookingID = b.BookingID
    WHERE bd.TicketID = v_ticket_id
      AND b.MemberID = v_member_id;

    IF v_count = 0 THEN
        RAISE TICKET_NOT_FOUND;
    END IF;

    SELECT bd.BookingDetailsID, b.PaymentMethod
    INTO v_bookingdetails_id, v_payment_method
    FROM BookingDetails bd
    JOIN Booking b ON bd.BookingID = b.BookingID
    WHERE bd.TicketID = v_ticket_id
      AND b.MemberID = v_member_id;

    SELECT 'R' || LPAD(NVL(MAX(TO_NUMBER(SUBSTR(RefundID, 2))), 0) + 1, 4,
'0')
    INTO v_refund_id
    FROM Refund;

    INSERT INTO Refund (RefundID, BookingDetailsID, RefundDate,
RefundAmount, PaymentMethod, RefundStatus, RefundReason)
    VALUES (v_refund_id, v_bookingdetails_id, SYSDATE, 0,
v_payment_method, 'Pending', v_refund_reason);

    DBMS_OUTPUT.PUT_LINE('Refund request submitted for Ticket ' ||
v_ticket_id);

EXCEPTION
    WHEN MEMBER_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Member ID ' || v_member_id || ' not
found.');
    WHEN TICKET_NOT_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Ticket ID ' || v_ticket_id || ' not found
for Member ID ' || v_member_id);
END;
/


--Error: ticket refund not belongs to member
EXEC prc_request_refund('M0010','TCK00279', 'Wrong Destination');

EXEC prc_request_refund('M0009','TCK00012', 'Wrong Destination');
```

Sample Output:

```
Procedure created.

Ticket ID TCK00279 not found for Member ID M0010

PL/SQL procedure successfully completed.

Refund request submitted for Ticket TCK00012

PL/SQL procedure successfully completed.
```

4.1.5 Trigger 1: Prevent members refund tickets after scheduled departure time

Purpose: The purpose of this trigger is to prevent member refund tickets twice and also validate the refund request by comparing the current system date and time against the departure time of the associated schedule before allowing the refund record to be inserted into the refund table.

SQL Statement:

```
CREATE OR REPLACE TRIGGER trg_refund_processing
BEFORE INSERT ON Refund
FOR EACH ROW
DECLARE
    v_price          BookingDetails.Price%TYPE;
    v_departure_time Schedule.DepartureTime%TYPE;
    v_ticket_status  Ticket.Status%TYPE;
    v_ticket_id      Ticket.TicketID%TYPE;
BEGIN
    SELECT bd.Price, s.DepartureTime, t.Status, bd.TicketID
    INTO v_price, v_departure_time, v_ticket_status, v_ticket_id
    FROM BookingDetails bd
    JOIN Ticket t ON bd.TicketID = t.TicketID
    JOIN Schedule s ON t.ScheduleID = s.ScheduleID
    WHERE bd.BookingDetailsID = :NEW.BookingDetailsID;

    IF v_ticket_status = 'Refunded' THEN
        RAISE_APPLICATION_ERROR(-20002, 'This ticket has already been
refunded.');
    END IF;

    IF v_ticket_status = 'Available' THEN
        RAISE_APPLICATION_ERROR(-20002, 'This ticket not been purchase.');
    END IF;

    IF v_departure_time < SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20003, 'Cannot refund a ticket with past
departure time.');
    END IF;

    IF v_departure_time - SYSDATE >= 2 THEN
        :NEW.RefundAmount := v_price * 0.7;
        :NEW.RefundStatus := 'Approved';
    ELSE
        :NEW.RefundAmount := 0;
        :NEW.RefundStatus := 'Rejected';
    END IF;

    IF :NEW.RefundAmount > 0 THEN
        UPDATE Ticket
        SET Status = 'Refunded'
        WHERE TicketID = v_ticket_id;
    END IF;
END;
/

--Error: duplicate refund
EXEC prc_request_refund('M0010','TCK00276', 'Wrong Destination');

--Error: refund ticket after departure date
EXEC prc_request_refund('M0001','TCK00295', 'Wrong Destination');

--Error: refund ticket that not been purchase
EXEC prc_request_refund('M0005','TCK00291', 'Wrong Destination');

EXEC prc_request_refund('M0004','TCK00252', 'Wrong Destination');
```

Sample Output:

```
Trigger created.

Ticket ID TCK00276 not found for Member ID M0010

PL/SQL procedure successfully completed.

An unexpected error occurred: ORA-20003: Cannot refund a ticket with past departure time.
ORA-06512: at "TRY_ADM.TRG_REFUND_PROCESSING", line 23
ORA-04088: error during execution of trigger 'TRY_ADM.TRG_REFUND_PROCESSING'

PL/SQL procedure successfully completed.

An unexpected error occurred: ORA-20002: This ticket not been purchase.
ORA-06512: at "TRY_ADM.TRG_REFUND_PROCESSING", line 19
ORA-04088: error during execution of trigger 'TRY_ADM.TRG_REFUND_PROCESSING'

PL/SQL procedure successfully completed.
```

Before trigger:

REFUNDID	BOOKINGDET	REFUNDDATE	REFUNDAMOUNT	REFUNDSTATUS	PAYMENTMETHOD	REFUNDREASON
R0106	BD000252	07-09-2025	0	Pending	Credit Card	Wrong Destination

After trigger:

REFUNDID	BOOKINGDET	REFUNDDATE	REFUNDAMOUNT	REFUNDSTATUS	PAYMENTMETHOD	REFUNDREASON
R0106	BD000252	07-09-2025	14	Approved	Credit Card	Wrong Destination

4.1.6 Trigger 2: Audit Log for Ticket Fare

Purpose: The purpose of this trigger is to record every changes in ticket fare to allow admin track easily

SQL Statement:

```
Drop table Ticket_Fare_Audit;
Drop sequence ticket_audit_seq;

CREATE TABLE Ticket_Fare_Audit (
    AuditID      NUMBER PRIMARY KEY,
    TicketID     VARCHAR2(10),
    OldFare      NUMBER,
    NewFare      NUMBER,
    ActionBy     VARCHAR2(50),
    ActionDate   DATE,
    ActionTime   VARCHAR2(8),
    ActionType   VARCHAR2(10),    -- INSERT / UPDATE / DELETE
    Status       VARCHAR2(20)     -- SUCCESS / FAILED
);

CREATE SEQUENCE ticket_audit_seq
MINVALUE 1
MAXVALUE 999999
START WITH 1
INCREMENT BY 1
NOCACHE;

CREATE OR REPLACE TRIGGER trg_ticket_audit
BEFORE INSERT OR UPDATE OF Fare OR DELETE ON Ticket
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    IF INSERTING THEN
        INSERT INTO Ticket_Fare_Audit (AuditID, TicketID, OldFare,
NewFare,
                                       ActionBy, ActionDate, ActionTime,
ActionType, Status)
        VALUES (ticket_audit_seq.NEXTVAL, :NEW.TicketID, NULL, :NEW.Fare,
                USER, SYSDATE, TO_CHAR(SYSDATE,'HH24:MI:SS'), 'INSERT',
'SUCCESS');

    ELSIF UPDATING('Fare') THEN
        SELECT COUNT(*)
        INTO v_count
        FROM BookingDetails bd
        JOIN Booking b ON bd.BookingID = b.BookingID
        WHERE bd.TicketID = :OLD.TicketID
          AND b.PaymentStatus = 'Incomplete';

        IF v_count > 0 THEN
            -- Log failed attempt
            INSERT INTO Ticket_Fare_Audit (AuditID, TicketID, OldFare,
NewFare,
                                           ActionBy, ActionDate,
ActionTime, ActionType, Status)
            VALUES (ticket_audit_seq.NEXTVAL, :OLD.TicketID, :OLD.Fare,
:NEW.Fare,
                    USER, SYSDATE, TO_CHAR(SYSDATE,'HH24:MI:SS'),
'UPDATE', 'FAILED');

            RAISE_APPLICATION_ERROR(-20001,
                'Fare cannot be changed: there are incomplete bookings for
this ticket.');
        ELSE
```

```
            INSERT INTO Ticket_Fare_Audit (AuditID, TicketID, OldFare,
NewFare,
                                           ActionBy, ActionDate,
ActionTime, ActionType, Status)
            VALUES (ticket_audit_seq.NEXTVAL, :OLD.TicketID, :OLD.Fare,
:NEW.Fare,
                    USER, SYSDATE, TO_CHAR(SYSDATE,'HH24:MI:SS'),
'UPDATE', 'SUCCESS');
        END IF;

    ELSIF DELETING THEN
        SELECT COUNT(*)
        INTO v_count
        FROM BookingDetails
        WHERE TicketID = :OLD.TicketID;

        IF v_count > 0 THEN
            -- Log failed delete attempt
            INSERT INTO Ticket_Fare_Audit (AuditID, TicketID, OldFare,
NewFare,
                                           ActionBy, ActionDate,
ActionTime, ActionType, Status)
            VALUES (ticket_audit_seq.NEXTVAL, :OLD.TicketID, :OLD.Fare,
NULL,
                    USER, SYSDATE, TO_CHAR(SYSDATE,'HH24:MI:SS'),
'DELETE', 'FAILED');

            RAISE_APPLICATION_ERROR(-20002,
                'Ticket cannot be deleted: there are existing bookings for
this ticket.');
        ELSE
            INSERT INTO Ticket_Fare_Audit (AuditID, TicketID, OldFare,
NewFare,
                                           ActionBy, ActionDate,
ActionTime, ActionType, Status)
            VALUES (ticket_audit_seq.NEXTVAL, :OLD.TicketID, :OLD.Fare,
NULL,
                    USER, SYSDATE, TO_CHAR(SYSDATE,'HH24:MI:SS'),
'DELETE', 'SUCCESS');
        END IF;
    END IF;
END;
/

--test case
INSERT INTO Ticket
VALUES ('TCK00306', 'SCH0305', 'Z9', 50, 'Active', 'N');

UPDATE Ticket
SET Fare = 10
WHERE TicketID = 'TCK00306';

DELETE FROM Ticket WHERE TicketID = 'TCK00306';

Select * from ticket_fare_audit;
```

Sample Output:

```
Table dropped.


Sequence dropped.


Table created.


Sequence created.


Trigger created.


1 row created.


1 row updated.


1 row deleted.


   AUDITID TICKETID      OLDFARE    NEWFARE ACTIONBY                                            ACTIONDATE ACTIONTI ACTIONTYPE STATUS
---------- ---------- ---------- ---------- ------------------------------------------------- ---------- -------- ---------- --------------
         1 TCK00306                      50 TRY_ADM                                           07-09-2025 02:12:15 INSERT     SUCCESS
         2 TCK00306           50         10 TRY_ADM                                           07-09-2025 02:12:15 UPDATE     SUCCESS
         3 TCK00306           10            TRY_ADM                                           07-09-2025 02:12:15 DELETE     SUCCESS
```

4.1.7 Report 1: Monthly Refunds Summary Report

Purpose: The purpose of this report is to identify how many refunds happened in each month. This is to help management understand refund patterns and take actions to reduce refunds in the future.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_refunds_by_date IS
    v_refundMonth   NUMBER;
    v_refundYear    NUMBER;
    v_monthTotal    NUMBER(15,2);
    v_grandTotal    NUMBER(15,2) := 0;
    v_refundCount   NUMBER := 0;

    CURSOR monthCursor IS
        SELECT DISTINCT EXTRACT(MONTH FROM RefundDate) AS refundMonth,
                        EXTRACT(YEAR  FROM RefundDate) AS refundYear
        FROM Refund
        WHERE RefundStatus = 'Approved'
        ORDER BY refundYear, refundMonth;

    CURSOR refundCursor IS
        SELECT r.RefundID,
               m.MemberID,
               m.FullName,
               bd.BookingDetailsID,
               bd.TicketID,
               r.RefundAmount,
               r.RefundDate
        FROM Refund r
        JOIN BookingDetails bd ON r.BookingDetailsID = bd.BookingDetailsID
        JOIN Booking b ON bd.BookingID = b.BookingID
        JOIN Member m ON b.MemberID = m.MemberID
        WHERE EXTRACT(MONTH FROM r.RefundDate) = v_refundMonth
          AND EXTRACT(YEAR  FROM r.RefundDate) = v_refundYear
          AND r.RefundStatus = 'Approved';

    refundRec refundCursor%ROWTYPE;
BEGIN
    OPEN monthCursor;

    LOOP
        FETCH monthCursor INTO v_refundMonth, v_refundYear;
        EXIT WHEN monthCursor%NOTFOUND;

        --Header
        DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
        DBMS_OUTPUT.PUT_LINE(CHR(10));
        DBMS_OUTPUT.PUT_LINE('Refunds for Month: ' ||
                             TO_CHAR(TO_DATE(v_refundMonth,'MM'),'Mon') ||
'-' || v_refundYear);
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));

        --Heading
        DBMS_OUTPUT.PUT_LINE(RPAD('RefundID',12) || ' ' ||
                             RPAD('MemberID',12) || ' ' ||
                             RPAD('FullName',30) || ' ' ||
                             RPAD('BookingDetailsID',18) || ' ' ||
                             RPAD('TicketID',12) || ' ' ||
                             RPAD('RefundAmount',15) || ' ' ||
                             RPAD('RefundDate',12));
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));

        OPEN refundCursor;
        v_monthTotal := 0;
```

```
        v_refundCount := 0;

        LOOP
            FETCH refundCursor INTO refundRec;
            EXIT WHEN refundCursor%NOTFOUND;

            v_refundCount := v_refundCount + 1;

            DBMS_OUTPUT.PUT_LINE(RPAD(refundRec.RefundID,12) || ' ' ||
                                 RPAD(refundRec.MemberID,12) || ' ' ||
                                 RPAD(refundRec.FullName,30) || ' ' ||
                                 RPAD(refundRec.BookingDetailsID,18) || '
' ||
                                 RPAD(refundRec.TicketID,12) || ' ' ||

RPAD(TO_CHAR(NVL(refundRec.RefundAmount,0),'FM$999,999,999.00'),15) || ' '
||

TO_CHAR(refundRec.RefundDate,'DD-MON-YYYY'));

            v_monthTotal := v_monthTotal + NVL(refundRec.RefundAmount,0);
        END LOOP;

        DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
        DBMS_OUTPUT.PUT_LINE('Total Approved Refunds in ' ||
                             TO_CHAR(TO_DATE(v_refundMonth,'MM'),'Month')
|| v_refundYear ||
                             ': ' ||
TO_CHAR(v_monthTotal,'FM$999,999,999.00'));
        DBMS_OUTPUT.PUT_LINE('Number of Approved Refunds: ' ||
v_refundCount);

        CLOSE refundCursor;

        v_grandTotal := v_grandTotal + NVL(v_monthTotal,0);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
    DBMS_OUTPUT.PUT_LINE('Grand Total Approved Refunds: ' ||
TO_CHAR(v_grandTotal,'FM$999,999,999.00'));
    DBMS_OUTPUT.PUT_LINE('Total Months with Refunds: ' ||
monthCursor%ROWCOUNT);
    CLOSE monthCursor;

    DBMS_OUTPUT.PUT_LINE(LPAD('=',50,'=') || RPAD(' END OF REPORT ',
70,'='));
END;
/

EXEC prc_refunds_by_date;
```

Sample Output:

```
Refunds for Month: Jun-2025
=============================================================================================================================
RefundID     MemberID     FullName                  BookingDetailsID   TicketID     RefundAmount    RefundDate
=============================================================================================================================
R0057        M0001        Alicia Tan Mei Ling       BD000017           TCK00017     $14.00          07-JUN-2025
R0040        M0003        Nurul Izzah Binti Ahmad   BD000086           TCK00086     $14.00          03-JUN-2025
R0032        M0003        Nurul Izzah Binti Ahmad   BD000231           TCK00231     $14.00          29-JUN-2025
R0088        M0004        Lim Chee How              BD000125           TCK00125     $14.00          27-JUN-2025
R0071        M0006        Daniel Tan Hong Yee       BD000032           TCK00032     $14.00          14-JUN-2025
R0096        M0008        Muhammad Hafiz Bin Saad   BD000068           TCK00068     $14.00          24-JUN-2025
=============================================================================================================================
Total Approved Refunds in June     2025: $84.00
Number of Approved Refunds: 6
-----------------------------------------------------------------------------------------------------------------------------


Refunds for Month: Jul-2025
=============================================================================================================================
RefundID     MemberID     FullName                  BookingDetailsID   TicketID     RefundAmount    RefundDate
=============================================================================================================================
R0047        M0001        Alicia Tan Mei Ling       BD000239           TCK00239     $14.00          05-JUL-2025
R0020        M0002        John Lee Wei Sheng        BD000083           TCK00083     $14.00          19-JUL-2025
R0034        M0003        Nurul Izzah Binti Ahmad   BD000139           TCK00139     $14.00          28-JUL-2025
R0063        M0009        Rachel Ong Li Wen         BD000254           TCK00254     $14.00          23-JUL-2025
R0084        M0009        Rachel Ong Li Wen         BD000292           TCK00292     $14.00          08-JUL-2025
R0031        M0010        Goh Jia Xin               BD000123           TCK00123     $14.00          04-JUL-2025
R0061        M0010        Goh Jia Xin               BD000118           TCK00118     $14.00          27-JUL-2025
=============================================================================================================================
Total Approved Refunds in July     2025: $98.00
Number of Approved Refunds: 7
-----------------------------------------------------------------------------------------------------------------------------


Refunds for Month: Aug-2025
=============================================================================================================================
RefundID     MemberID     FullName                  BookingDetailsID   TicketID     RefundAmount    RefundDate
=============================================================================================================================
R0064        M0002        John Lee Wei Sheng        BD000234           TCK00234     $14.00          23-AUG-2025
R0048        M0005        Siti Aminah Bt Zulkifli   BD000237           TCK00237     $14.00          04-AUG-2025
R0087        M0009        Rachel Ong Li Wen         BD000022           TCK00022     $14.00          21-AUG-2025
=============================================================================================================================
Total Approved Refunds in August   2025: $42.00
Number of Approved Refunds: 3
=============================================================================================================================
Grand Total Approved Refunds: $1,022.00
Total Months with Refunds: 13
============================================== END OF REPORT ================================================

PL/SQL procedure successfully completed.
```

4.1.8 Report 2: Member Booking Summary Report

Purpose: The purpose of this report is to show the total booking amount,, monthly booking peak and preferred destinations for each member. Based on this insight, the company can implement promotion on the peak month and destination and enforce retention strategy like limited-time discount for low active members during off peak season.

SQL Statement:

```
SET SERVEROUTPUT ON;
SET LINESIZE 150;
SET PAGESIZE 150;

CREATE OR REPLACE PROCEDURE prc_member_booking_report IS
    v_memberID        Member.MemberID%TYPE;
    v_memberName      Member.FullName%TYPE;
    v_booking_count    NUMBER;
    v_ticket_count     NUMBER;
    v_total_spent      NUMBER(12,2);
    v_percentage       NUMBER(5,2);

    CURSOR member_summary_cur IS
        SELECT m.MemberID,
               m.FullName,
               COUNT(b.BookingID) AS booking_count,
               NVL(SUM(b.TotalAmount),0) AS total_spent,
               ROUND(NVL(SUM(b.TotalAmount),0) * 100.0 /
                     (SELECT SUM(TotalAmount) FROM Booking), 2) AS
percentage_contribution
        FROM Member m
        JOIN Booking b ON m.MemberID = b.MemberID
        GROUP BY m.MemberID, m.FullName
        ORDER BY total_spent DESC;

    CURSOR top_destinations_cur(v_memberID VARCHAR2) IS
        SELECT *
        FROM (
            SELECT s.Destination,
                   COUNT(bd.BookingDetailsID) AS trip_count
            FROM Booking b
            JOIN BookingDetails bd ON b.BookingID = bd.BookingID
            JOIN Ticket t ON bd.TicketID = t.TicketID
            JOIN Schedule s ON t.ScheduleID = s.ScheduleID
            WHERE b.MemberID = v_memberID
            GROUP BY s.Destination
            ORDER BY trip_count DESC
        ) WHERE ROWNUM <= 3;

    CURSOR top_months_cur(v_memberID VARCHAR2) IS
        SELECT *
        FROM (
            SELECT TO_CHAR(b.BookingDate, 'MON-YYYY') AS booking_month,
                   SUM(b.TotalAmount) AS month_spent
            FROM Booking b
            WHERE b.MemberID = v_memberID
            GROUP BY TO_CHAR(b.BookingDate, 'MON-YYYY')
            ORDER BY month_spent DESC
        ) WHERE ROWNUM <= 3;

BEGIN
    -- Report Header
    DBMS_OUTPUT.PUT_LINE(CHR(10));
    DBMS_OUTPUT.PUT_LINE(RPAD('MEMBER BOOKING ANALYSIS REPORT', 120, '
'));
    DBMS_OUTPUT.PUT_LINE(RPAD('Generated on: ' || TO_CHAR(SYSDATE,
'DD-MON-YYYY HH24:MI:SS'), 120, ' '));
```

```
    OPEN member_summary_cur;
    LOOP
        FETCH member_summary_cur INTO v_memberID, v_memberName,
v_booking_count, v_total_spent, v_percentage;
        EXIT WHEN member_summary_cur%NOTFOUND;

        -- Count total tickets purchased by member
        SELECT COUNT(BookingDetailsID)
        INTO v_ticket_count
        FROM BookingDetails bd
        JOIN Booking b ON bd.BookingID = b.BookingID
        WHERE b.MemberID = v_memberID;

        DBMS_OUTPUT.PUT_LINE(CHR(10));
        DBMS_OUTPUT.PUT_LINE('Booking Summary for Member: ' ||
v_memberName || ' (' || v_memberID || ')');
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 80, '='));

        DBMS_OUTPUT.PUT_LINE(
            RPAD('Total Bookings', 20) ||
            RPAD('Total Tickets', 20) ||
            RPAD('Total Spent', 20) ||
            RPAD('% Contribution', 20)
        );
        DBMS_OUTPUT.PUT_LINE(RPAD('-', 80, '-'));

        DBMS_OUTPUT.PUT_LINE(
            RPAD(v_booking_count, 20) ||
            RPAD(v_ticket_count, 20) ||
            RPAD(TO_CHAR(v_total_spent,'FM$999,999.00'), 20) ||
            RPAD(v_percentage || '%', 20)
        );

        -- Top 3 Destinations
        DBMS_OUTPUT.PUT_LINE(RPAD('Top 3 Destinations:', 80, ' '));
        FOR dest_rec IN top_destinations_cur(v_memberID) LOOP
            DBMS_OUTPUT.PUT_LINE(
                '   - ' || RPAD(dest_rec.Destination, 30) ||
                'Tickets: ' || RPAD(dest_rec.trip_count, 10)
            );
        END LOOP;

        -- Top 3 Months by Spending
        DBMS_OUTPUT.PUT_LINE(RPAD('Top 3 Months by Spending:', 80, ' '));
        FOR month_rec IN top_months_cur(v_memberID) LOOP
            DBMS_OUTPUT.PUT_LINE(
                '   - ' || RPAD(month_rec.booking_month, 30) ||
                'Amount: ' ||
RPAD(TO_CHAR(month_rec.month_spent,'FM$999,999.00'), 12)
            );
        END LOOP;

        DBMS_OUTPUT.PUT_LINE(LPAD('-',80,'-'));
    END LOOP;
    CLOSE member_summary_cur;

    -- Footer
    DBMS_OUTPUT.PUT_LINE(CHR(10));
    DBMS_OUTPUT.PUT_LINE(LPAD('=',30,'=') || RPAD(' END OF REPORT ',
50,'='));
END;
/
EXEC prc_member_booking_report;
```

Sample Output:

```
Booking Summary for Member: John Lee Wei Sheng (M0002)
================================================================================
Total Bookings      Total Tickets       Total Spent         % Contribution
--------------------------------------------------------------------------------
10                  24                  $480.00             7.87%
Top 3 Destinations:
- Gemas                         Tickets: 4
- Butterworth                   Tickets: 4
- KLIA Transit                  Tickets: 2
Top 3 Months by Spending:
- AUG-2024                      Amount: $140.00
- DEC-2024                      Amount: $120.00
- MAR-2025                      Amount: $80.00
--------------------------------------------------------------------------------


Booking Summary for Member: Tan Wei Xin (M0007)
================================================================================
Total Bookings      Total Tickets       Total Spent         % Contribution
--------------------------------------------------------------------------------
10                  22                  $440.00             7.21%
Top 3 Destinations:
- Gemas                         Tickets: 3
- Rawang                        Tickets: 3
- Sungai Petani                 Tickets: 2
Top 3 Months by Spending:
- JUN-2025                      Amount: $120.00
- JUN-2024                      Amount: $80.00
- NOV-2024                      Amount: $40.00
--------------------------------------------------------------------------------


============================== END OF REPORT ===================================

PL/SQL procedure successfully completed.
```

4.2 EDISON CHAN YOONG QI

4.2.1 Query 1: Measure Revenue Generated by Each Bus Company Based on Their Destinations

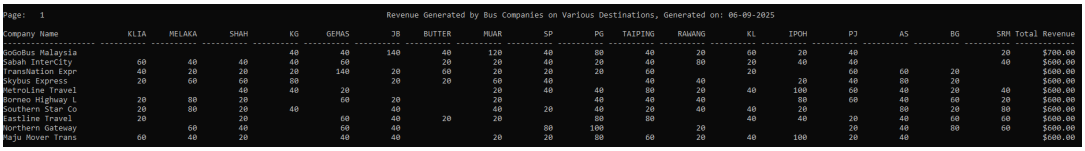
Purpose: The purpose of this query is to compare revenue earned from various bus companies to other destinations throughout the entire operational lifetime of the company, it serves as a strategic planning method as executives are able to see and compare which bus companies bring in the most revenue from each destination schedule. They may choose to allocate more or less resources to each bus company by improving the routes or stopping them.

SQL Statement:

```
CL SCR
SET LINESIZE 250
SET PAGESIZE 100
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

DROP INDEX destination_idx;
CREATE INDEX destination_idx ON Schedule(UPPER(Destination));

COLUMN CompanyName FORMAT A20 HEADING 'Company Name'
COLUMN Total_Revenue FORMAT $999999.99 HEADING 'Total Revenue'
COLUMN Destination FORMAT 99,999.99

COMPUTE SUM LABEL 'Grand Total Revenue: ' OF Total_Revenue ON REPORT

TTITLE CENTER 'Revenue Generated by Bus Companies on Various Destinations,
Generated on: ' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2
CREATE OR REPLACE VIEW EDISON_PIVOT_1 AS
SELECT
    CompanyName,
    KLIA, MELAKA, SHAH, KG, GEMAS, JB, BUTTER, MUAR, SP, PG,
    TAIPING, RAWANG, KL, IPOH, PJ, "AS", BG, SRM,
    (NVL(KLIA,0) + NVL(MELAKA,0) + NVL(SHAH,0) + NVL(KG,0) + NVL(GEMAS,0)
+
     NVL(JB,0) + NVL(BUTTER,0) + NVL(MUAR,0) + NVL(SP,0) + NVL(PG,0) +
     NVL(TAIPING,0) + NVL(RAWANG,0) + NVL(KL,0) + NVL(IPOH,0) + NVL(PJ,0)
+
     NVL("AS",0) + NVL(BG,0) + NVL(SRM,0)) AS Total_Revenue
FROM (
    SELECT
        SUBSTR(BC.CompanyName, 1, 16) AS CompanyName,
        UPPER(TRIM(S.Destination)) AS Destination,
        SUM(BD.Price) AS Revenue
    FROM Bus_Company BC
    JOIN Bus B ON BC.BusCompanyID = B.BusCompanyID
    JOIN Schedule S ON B.BusID = S.BusID
    JOIN Ticket T ON S.ScheduleID = T.ScheduleID
    JOIN BookingDetails BD ON T.TicketID = BD.TicketID
    JOIN Booking BK ON BD.BookingID = BK.BookingID
    GROUP BY BC.CompanyName, UPPER(TRIM(S.Destination))
)
PIVOT (
    SUM(Revenue)
    FOR Destination IN (
        'KLIA TRANSIT'          AS KLIA,
        'MELAKA SENTRAL'        AS MELAKA,
        'SHAH ALAM'             AS SHAH,
        'KAJANG'                AS KG,
        'GEMAS'                 AS GEMAS,
        'JOHOR BAHRU SENTRAL'   AS JB,
        'BUTTERWORTH'           AS BUTTER,
```

```
        'MUAR'                    AS MUAR,
        'SUNGAI PETANI'           AS SP,
        'PASIR GUDANG'            AS PG,
        'TAIPING'                 AS TAIPING,
        'RAWANG'                  AS RAWANG,
        'KUALA LUMPUR CENTRAL'    AS KL,
        'IPOH'                    AS IPOH,
        'PUTRAJAYA'               AS PJ,
        'ALOR SETAR'              AS "AS",
        'BATU GAJAH'              AS BG,
        'SEREMBAN'                AS SRM
    )
)
ORDER BY Total_Revenue DESC;

SELECT * FROM EDISON_PIVOT_1;
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF
```

Sample Output:



Page: 1 Revenue Generated by Bus Companies on Various Destinations, Generated on: 06-09-2025

Company Name	KLIA	MELAKA	SHAH	KG	GEMAS	JB	BUTTER	MUAR	SP	PG	TAIPING	RAWANG	KL	IPOH	PJ	AS	BG	SRM	Total Revenue
GoGoBus Malaysia				40	40	140	40	120	40	80	40	20	60	20	40			20	$700.00
Sabah InterCity	60	40	40	40	60		20	20	40	20	40	80	20	40	40			40	$600.00
TransNation Expr	40	20	20	20	140	20	60	20	20	20	60		20		60	60	20		$600.00
Skybus Express	20	60	60	80		20	20	60	40		40	40		20	40	80	20		$600.00
MetroLine Travel			40	40	20			20	40	40	80	20	40	100	60	40	20	40	$600.00
Borneo Highway L	20	80	20		60	20		20		40	40	40		80	60	40	60	20	$600.00
Southern Star Co	20	80	20	40		40		40	20	40	20	40	40	20		80	20	80	$600.00
Eastline Travel	20		20		60	40	20	20		80	80		40	40	20	40	60	60	$600.00
Northern Gateway		60	40		60	40			80	100		20			20	40	80	60	$600.00
Maju Mover Trans	60	40	20		40	40		20	20	80	60	20	40	100	20	40			$600.00

4.2.2 Query 2: Ranking Profit Generated From Each Bus Company

Purpose: The purpose of this query is to see which bus company provides the most revenue to the company, this serves to see which company is doing well and which company is not doing so well, this allows for better resource allocation in the hopes of improving their performance or financial standing. Serves as a tactical planning as they can plan for the future.

SQL Statement:

```
CL SCR
SET LINESIZE 100
SET PAGESIZE 100
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

COLUMN "Company Name" FORMAT A30 HEADING 'Company Name'
COLUMN "Total Revenue" FORMAT $999999.99 HEADING 'Total Revenue'
COLUMN "Revenue Difference" FORMAT $999999.99 HEADING 'Revenue Difference'

TTITLE CENTER 'Total Revenue Generated by each Bus Company; Generated on:
' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2

CREATE OR REPLACE VIEW CTE_EDISON_QUERY_1 AS
WITH CompanyRevenue AS (
    SELECT
        BC.CompanyName AS "Company Name",
        SUM(BK.TotalAmount) AS "Total Revenue"
    FROM Bus_Company BC
    JOIN Bus B ON BC.BusCompanyID = B.BusCompanyID
    JOIN Schedule S ON B.BusID = S.BusID
    JOIN Ticket T ON S.ScheduleID = T.ScheduleID
    JOIN BookingDetails BD ON T.TicketID = BD.TicketID
    JOIN Booking BK ON BD.BookingID = BK.BookingID
    GROUP BY BC.CompanyName
),
MaxRevenue AS (
    SELECT MAX("Total Revenue") AS "Max Rev" FROM CompanyRevenue
)
SELECT
    CR."Company Name",
    CR."Total Revenue",
    MR."Max Rev" - CR."Total Revenue" AS "Revenue Difference"
FROM CompanyRevenue CR
CROSS JOIN MaxRevenue MR
ORDER BY "Total Revenue" DESC;


BREAK ON REPORT
COMPUTE SUM LABEL 'Total Revenue: ' OF "Total Revenue" ON REPORT
COMPUTE AVG LABEL 'Average Difference: ' OF "Revenue Difference" ON REPORT

SELECT * FROM CTE_EDISON_QUERY_1;
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF
```

Sample Output:

Page: 1 Total Revenue Generated by each Bus Company Generated on: 06-09-2025

Company Name	Total Revenue	Revenue Difference
GoGoBus Malaysia	$3440.00	$.00
Southern Star Coaches	$2940.00	$500.00
Sabah InterCity Transport	$2600.00	$840.00
Skybus Express	$2460.00	$980.00
TransNation Express	$2420.00	$1020.00
Maju Mover Transport	$2420.00	$1020.00
Eastline Travel	$2420.00	$1020.00
Northern Gateway Buses	$2320.00	$1120.00
MetroLine Travel Services	$2220.00	$1220.00
Borneo Highway Link	$2220.00	$1220.00
Average Difference:		$894.00
Total Revenue:	$25460.00	

4.2.3 Procedure 1: Member Registration

Purpose: The purpose of this procedure is for customers to register as a member for the first time. If a customer exists in the Member table, it means that they have already paid for the registration fees. This procedure does not allow for customer duplicates, which means customers of the same name cannot register twice.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_Edison_1 (
    v_FullName IN VARCHAR2,
    v_PhoneNo IN VARCHAR2,
    v_Email IN VARCHAR2,
    v_MembershipStatus IN VARCHAR2
)
IS
    v_duplicate_count NUMBER := 0;
    v_generated_memberid VARCHAR2(8);

    -- User defined exception
    dup_member_ex EXCEPTION;

    CURSOR member_cursor IS
        SELECT COUNT(*)
        FROM Member
        WHERE UPPER(FullName) = UPPER(v_FullName);

BEGIN
    -- Use cursor to check for duplicate member names
    OPEN member_cursor;
    FETCH member_cursor INTO v_duplicate_count;
    CLOSE member_cursor;

    IF v_duplicate_count > 0 THEN
        RAISE dup_member_ex;   -- raise user-defined exception
    ELSE
        -- Ensure sequence exists
        BEGIN
            EXECUTE IMMEDIATE 'CREATE SEQUENCE member_num_seq START WITH
1001 INCREMENT BY 1';
        EXCEPTION
            WHEN OTHERS THEN
                NULL; -- ignore if sequence already exists
        END;

        -- Generate the MemberID
        SELECT 'M' || TO_CHAR(member_num_seq.NEXTVAL, 'FM0000')
        INTO v_generated_memberid
        FROM DUAL;

        -- Insert the new member if no duplicates found
        INSERT INTO Member (
            MemberID,
            FullName,
            PhoneNo,
            Email,
            MembershipStatus,
            JoinDate
        ) VALUES (
            v_generated_memberid,
            v_FullName,
            v_PhoneNo,
            v_Email,
            v_MembershipStatus,
            SYSDATE
        );
```

```
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Member ' || v_FullName || ' added
successfully with ID: ' || v_generated_memberid);
    END IF;

EXCEPTION
    WHEN dup_member_ex THEN
        DBMS_OUTPUT.PUT_LINE('Error: Member with name "' || v_FullName ||
'" already exists.');
        DBMS_OUTPUT.PUT_LINE('No data has been added.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error adding member: ' || SQLERRM);
        RAISE;
END prc_Edison_1;
/
```

Sample Output:

```
SQL> EXEC prc_Edison_1 ('Qu', '0271952172', 'nameless@gmail.com', 'Gold')
Error: Member with name "Qu" already exists.
No data has been added.

PL/SQL procedure successfully completed.

SQL> EXEC prc_Edison_1 ('Edi Edi', '0271952172', 'nameless@gmail.com', 'Gold')
Member Edi Edi added successfully with ID: M0016

PL/SQL procedure successfully completed.
```

4.2.4 Procedure 2: Dynamically Insert Into Part Based on Part_Order

Purpose: The purpose of this procedure is when purchasing parts, users can automatically set the new price, thus inserting a new row stating the new price, if the user enters the same price, a new row will not be added, instead the current quantity will be increased accordingly.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_Edison_2(
    v_part_id        IN VARCHAR2,
    v_supplier_id    IN VARCHAR2,
    v_order_quantity IN NUMBER,
    v_new_unit_price IN Part.UnitPrice%TYPE
) AS
    v_order_id VARCHAR2(8);
    v_next_order_num NUMBER;
    v_current_stock NUMBER;
    v_new_key NUMBER;
    v_part_key NUMBER;
    v_unitprice NUMBER(10,2);
    v_part_name VARCHAR2(100);
    v_order_prefix VARCHAR2(2);
BEGIN
    BEGIN
        SELECT MAX(OrderID) INTO v_order_id FROM Part_Order;
        IF v_order_id IS NOT NULL THEN
            v_next_order_num := TO_NUMBER(REGEXP_SUBSTR(v_order_id,
'\d+$')) + 1;
            v_order_prefix := REGEXP_SUBSTR(v_order_id, '^[A-Za-z]+');
        ELSE
            v_next_order_num := 1;
            v_order_prefix := 'PO';
        END IF;
    EXCEPTION
        WHEN OTHERS THEN
            v_next_order_num := 1;
            v_order_prefix := 'PO';
    END;

    v_order_id := v_order_prefix || LPAD(v_next_order_num, 4, '0');

    -- Generate new key for Part
    SELECT NVL(MAX(PartKey), 0) + 1 INTO v_new_key
    FROM Part;

    -- Get current active part info
    BEGIN
        SELECT PartKey, StockQuantity, UnitPrice, PartName
        INTO v_part_key, v_current_stock, v_unitprice, v_part_name
        FROM Part
        WHERE PartID = v_part_id
        AND IsActive = 1;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20000, 'Part not found or not active:
' || v_part_id);
    END;

    -- Insert order record
    INSERT INTO Part_Order VALUES (
        v_order_id,
        SYSDATE,
        v_part_key,
        v_part_id,
        v_supplier_id,
```

```
        v_order_quantity,
        v_unitprice
    );

    -- Check if price is the same
    IF v_unitprice = v_new_unit_price THEN
        -- Update quantity for existing part
        UPDATE Part
        SET StockQuantity = StockQuantity + v_order_quantity
        WHERE PartKey = v_part_key AND EndDate IS NULL;
    ELSE
        -- Expire the current record
        UPDATE Part
        SET EndDate = SYSDATE - 1,
            IsActive = 0
        WHERE PartKey = v_part_key AND EndDate IS NULL;

        -- Insert new record with new price
        INSERT INTO Part VALUES (
            v_new_key,
            v_part_id,
            v_part_name,
            v_order_quantity,
            v_new_unit_price,
            SYSDATE,
            NULL,
            1
        );
    END IF;

    DBMS_OUTPUT.PUT_LINE('Part order processed: ' || v_order_id);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20000, 'Error processing part order: ' ||
SQLERRM);
END;
/
```

Sample Output:

```
SQL> SELECT * FROM PART WHERE PARTID = 'PRT010';

   PARTKEY PARTID   PARTNAME                                           STOCKQUANTITY  UNITPRICE STARTDATE  ENDDATE      ISACTIVE
---------- -------- -------------------------------------------------- ------------- ---------- ---------- ---------- ----------
        10 PRT010   AC Compressor                                              274      58.91 16-04-2023 05-09-2025          0
        16 PRT010   AC Compressor                                              148      58.92 06-09-2025                     1

SQL> EXEC prc_Edison_2('PRT010', 'SUP003', 111, 111.11);
Part order processed: PO0310

PL/SQL procedure successfully completed.

SQL> SELECT * FROM PART WHERE PARTID = 'PRT010';

   PARTKEY PARTID   PARTNAME                                           STOCKQUANTITY  UNITPRICE STARTDATE  ENDDATE      ISACTIVE
---------- -------- -------------------------------------------------- ------------- ---------- ---------- ---------- ----------
        10 PRT010   AC Compressor                                              274      58.91 16-04-2023 05-09-2025          0
        16 PRT010   AC Compressor                                              148      58.92 06-09-2025 05-09-2025          0
        17 PRT010   AC Compressor                                              111     111.11 06-09-2025                     1
```

4.2.5 Trigger 1: Audit Log for Staff

Purpose: To track update, delete and insert on the staff table, update ensures that no negative value can be placed into the staff's salary.

SQL Statement:

```
-- TRIGGER 1
DROP SEQUENCE staff_audit_seq;
DROP TABLE Staff_Audit;

CREATE SEQUENCE staff_audit_seq
MINVALUE 1
MAXVALUE 999999
START WITH 1
INCREMENT BY 1
NOCACHE;

CREATE TABLE Staff_Audit (
    AuditID     NUMBER PRIMARY KEY,
    StaffID     VARCHAR2(8) NOT NULL,
    FullName    VARCHAR2(35),
    Position    VARCHAR2(20),
    Salary      NUMBER(10, 2),
    Email       VARCHAR2(40),
    PhoneNo     VARCHAR2(12),
    HireDate    DATE,
    Operation   VARCHAR2(10),
    ChangedBy   VARCHAR2(30),
    ChangedAt   DATE DEFAULT SYSDATE
);

CREATE OR REPLACE TRIGGER trg_Edison_Staff_Audit
BEFORE INSERT OR UPDATE OR DELETE ON Staff
FOR EACH ROW
DECLARE
    ex_negative_salary EXCEPTION;
BEGIN
    IF INSERTING OR UPDATING THEN
        IF :NEW.Salary < 0 THEN
            RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be negative.');
        END IF;
    END IF;

    IF INSERTING THEN
        INSERT INTO Staff_Audit (AuditID, StaffID, FullName, Position,
Salary, Email, PhoneNo, HireDate, Operation, ChangedBy)
        VALUES (staff_audit_seq.nextval, :NEW.StaffID, :NEW.FullName,
:NEW.Position, :NEW.Salary, :NEW.Email, :NEW.PhoneNo, :NEW.HireDate,
'INSERT', 'USER');

    ELSIF UPDATING THEN
        INSERT INTO Staff_Audit (AuditID, StaffID, FullName, Position,
Salary, Email, PhoneNo, HireDate, Operation, ChangedBy)
        VALUES (staff_audit_seq.nextval, :NEW.StaffID, :NEW.FullName,
:NEW.Position, :NEW.Salary, :NEW.Email, :NEW.PhoneNo, :NEW.HireDate,
'UPDATE', 'USER');

    ELSIF DELETING THEN
        INSERT INTO Staff_Audit (AuditID, StaffID, FullName, Position,
Salary, Email, PhoneNo, HireDate, Operation, ChangedBy)
        VALUES (staff_audit_seq.nextval, :OLD.StaffID, :OLD.FullName,
:OLD.Position, :OLD.Salary, :OLD.Email, :OLD.PhoneNo, :OLD.HireDate,
'DELETE', 'USER');
    END IF;
EXCEPTION
    WHEN ex_negative_salary THEN
```

```
        RAISE_APPLICATION_ERROR(-20002, 'Negative salary not allowed for
staff!!!');
END;
/
```

Sample Output:

-ON UPDATE, SETTING NEGATIVE SALARY:

```
UPDATE STAFF SET SALARY = -1 WHERE STAFFID = 'STF001'
       *
ERROR at line 1:
ORA-20001: Salary cannot be negative.
ORA-06512: at "EDISON.TRG_EDISON_STAFF_AUDIT", line 6
ORA-04088: error during execution of trigger 'EDISON.TRG_EDISON_STAFF_AUDIT'
```

-AUDIT LOG ON UPDATE:

```
SQL> SELECT * FROM Staff_Audit;

no rows selected

SQL> UPDATE STAFF SET FULLNAME = 'Mary Married Mari' WHERE STAFFID = 'STF001';

1 row updated.

SQL> SELECT * FROM Staff_Audit;

   AUDITID STAFFID  FULLNAME                          POSITION                SALARY
---------- -------- --------------------------------- ------------------- ----------
         1 STF001   Mary Married Mari                  Manager                 3226.6
```

4.2.6 Trigger 2: Update Ticket Based on Extension Table

Purpose: When a ticket extension has been approved, the ExtendedTrip attribute of the ticket in the Ticket table will be changed from 'N' to 'Y' and status from 'Active' to 'Past', a new ticket will be created with the requested date. The trigger will also update the Extension table to charge an extension fee RM 5 of the original ticket fare.

SQL Statement:

```
CREATE OR REPLACE TRIGGER trg_Edison_Extension
BEFORE UPDATE OR INSERT ON Extension
FOR EACH ROW
DECLARE
    v_TicketID       Ticket.TicketID%TYPE;
    v_Status         Ticket.Status%TYPE;
    v_Extended       Ticket.ExtendedTrip%TYPE;
    v_Fare           Ticket.Fare%TYPE;
    v_SeatNo         Ticket.SeatNo%TYPE;
    v_MaxNum         NUMBER;
    v_NewTicket      VARCHAR2(20);
    v_OldDestination VARCHAR2(50);
    v_OldStation     VARCHAR2(50);
    v_OldScheduleID  VARCHAR2(50);
    v_NewScheduleID  VARCHAR2(50);
    v_DepartureDate  DATE;

BEGIN
    -- Find the original ticket
    SELECT BD.TicketID
    INTO v_TicketID
    FROM BookingDetails BD
    WHERE BD.BookingDetailsID = :NEW.BookingDetailsID;

    -- Get old schedule info
    SELECT Status, T.ScheduleID, ExtendedTrip, Fare, SeatNo,
S.Destination, S.Station, S.ScheduleDate
    INTO v_Status, v_OldScheduleID, v_Extended, v_Fare, v_SeatNo,
v_OldDestination, v_OldStation, v_DepartureDate
    FROM Ticket T
    JOIN Schedule S ON T.ScheduleID = S.ScheduleID
    WHERE TicketID = v_TicketID;

    -- Rule 1: cannot extend refunded ticket
    IF UPPER(v_Status) = 'REFUNDED' THEN
        :NEW.ExtensionStatus := 'REJECTED';
        :NEW.ExtensionFee    := 0;
        RETURN;
    END IF;

    -- Rule 2: only 1 extension allowed
    IF v_Extended = 'Y' THEN
        :NEW.ExtensionStatus := 'REJECTED';
        :NEW.ExtensionFee    := 0;
        RETURN;
    END IF;

    -- Rule 3: must request at least 2 days before departure
    IF :NEW.RequestDate > (v_DepartureDate - 2) THEN
        :NEW.ExtensionStatus := 'REJECTED';
        :NEW.ExtensionFee    := 0;
        RETURN;
    END IF;

    -- query to auto pick a new schedule (same route, active, different
schedule ID)
    BEGIN
```

```
        SELECT ScheduleID
        INTO v_NewScheduleID
        FROM (
            SELECT ScheduleID
            FROM Schedule
            WHERE ScheduleID <> v_OldScheduleID
              AND UPPER(ScheduleStatus) = 'ACTIVE'
              AND UPPER(Station) = UPPER(v_OldStation)
              AND UPPER(Destination) = UPPER(v_OldDestination)
            ORDER BY ScheduleDate, DepartureTime
        )
        WHERE ROWNUM = 1;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            :NEW.ExtensionStatus := 'REJECTED';
            :NEW.ExtensionFee    := 0;
            RETURN;
    END;

    -- Generate new ticketID
    SELECT MAX(TO_NUMBER(SUBSTR(TicketID, 4)))
    INTO v_MaxNum
    FROM Ticket;

    v_NewTicket := 'TCK' || LPAD(v_MaxNum + 1, 5, '0');

    -- Save the chosen schedule back into Extension row
    :NEW.NewTicketID := v_NewTicket;
    :NEW.NewScheduleID  := v_NewScheduleID;
    :NEW.ExtensionStatus := 'APPROVED';
    :NEW.ApproveDate     := SYSDATE;
    :NEW.ExtensionFee    := v_Fare + 5;   -- fixed RM5.00 extra charge

    -- Update original ticket
    UPDATE Ticket
    SET ExtendedTrip = 'Y', Status = 'Past'
    WHERE TicketID = v_TicketID;

    -- Create a new ticket under the new schedule
    INSERT INTO Ticket (
        TicketID,
        ScheduleID,
        SeatNo,
        Fare,
        Status,
        ExtendedTrip
    ) VALUES (
        v_NewTicket,
        v_NewScheduleID,
        v_SeatNo,
        v_Fare + 5,   -- new fare = old fare + RM5.00 cause of the xtra
charge
        'Active',
        'Y'
    );

EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20000, 'Unexpected error in ticket
extension: ' || SQLERRM);
END;
/
```

Sample Output:

```
SQL> -- Show that extendedtrip = 'N'
SQL> select * from ticket where ticketid = 'TCK00001';

TICKETID   SCHEDULEID SEATN       FARE STATUS               E
---------- ---------- ----- ---------- -------------------- -
TCK00001   SCH0169    C12        20.39 Past                 N

SQL>
SQL> -- Insert intitial extension request
SQL> insert into extension (extensionid, BOOKINGDETAILSID, requestdate, reason) values ('EXT0001', 'BD000001', SYSDATE + 5,'Very busy :C');

1 row created.

SQL> select * from extension;

EXTENSIONI BOOKINGDET NEWSCHEDUL EXTENSIONSTATUS      REQUESTDA APPROVEDA EXTENSIONFEE REASON
---------- ---------- ---------- -------------------- --------- --------- ------------ --------------------------------------------------
EXT0001    BD000001                                   22-AUG-25                       Very busy :C

SQL>
SQL>
SQL> select * from ticket where ticketid = 'TCK00001';

TICKETID   SCHEDULEID SEATN       FARE STATUS               E
---------- ---------- ----- ---------- -------------------- -
TCK00001   SCH0169    C12        20.39 Past                 N

SQL>
SQL> -- Update the data to trigger the trigger
SQL> update extension
  2  set extensionstatus = 'APPROVED'
  3  where BOOKINGDETAILSID = 'BD000001';

1 row updated.

SQL>
SQL> -- Show that extendedtrip is now 'Y'
SQL> select * from ticket where ticketid = 'TCK00001';

TICKETID   SCHEDULEID SEATN       FARE STATUS               E
---------- ---------- ----- ---------- -------------------- -
TCK00001   SCH0169    C12        20.39 Past                 Y

SQL>
SQL>
SQL> select * from extension where BOOKINGDETAILSID = 'BD00001';

no rows selected

SQL> select * from bookingdetails where ticketid = 'TCK00001';

BOOKINGDET TICKETID   BOOKINGID       PRICE
---------- ---------- ---------- ----------
BD000001   TCK00001   BK0082          46.56
```

4.2.7 Report 1: A Summary Report of The Number of Trips Each Drivers Has Driven For Each Company

Purpose: To provide management with a consolidated report showing the list of drivers that has driven for each company, the total number of trips performed by each driver for that company and the most popular destination that driver has driven to. Can be used to identify common destinations for each driver.

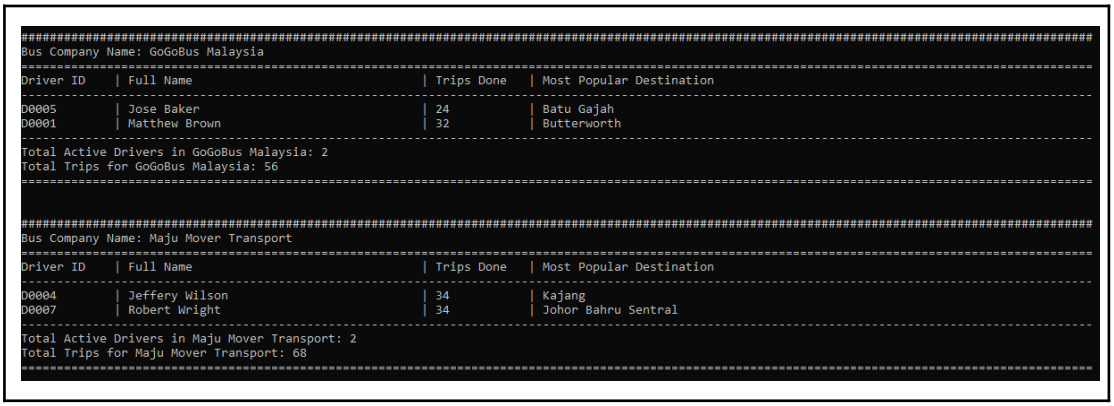
SQL Statement:

```
SET SERVEROUTPUT ON
SET LINESIZE 200
SET PAGESIZE 150
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';
CL SCR

CREATE OR REPLACE PROCEDURE prc_Edison_Report_1 IS
    -- Variables
    v_busCompanyID     Bus_Company.BusCompanyID%TYPE;
    v_companyName      Bus_Company.CompanyName%TYPE;
    v_driverID         Driver.DriverID%TYPE;
    v_fullName         Driver.FullName%TYPE;

    v_driverTrip       NUMBER := 0;
    v_companyTrip      NUMBER := 0;
    v_overallTrip      NUMBER := 0;
    v_driverCount      NUMBER := 0;
    v_companyCount     NUMBER := 0;
    v_popularDest      VARCHAR2(50);
    v_hasActiveDrivers BOOLEAN := FALSE;

    -- Cursors
    CURSOR busCompanyCursor IS
        SELECT bc.BusCompanyID, bc.CompanyName
        FROM Bus_Company bc
        ORDER BY bc.CompanyName;

    CURSOR driverCursor(p_companyID VARCHAR2) IS
        SELECT d.DriverID, d.FullName
        FROM Driver d
        WHERE d.BusCompanyID = p_companyID
        ORDER BY d.FullName;
BEGIN
    v_companyCount := 0;
    v_overallTrip  := 0;

    -- Report Header
    DBMS_OUTPUT.PUT_LINE(CHR(10));
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 19, '=') || '  Summary Report of The
Number of Trips Each Drivers Has Driven For Each Company  ' || LPAD('=',
19, '='));

    FOR companyRec IN busCompanyCursor LOOP
        v_busCompanyID := companyRec.BusCompanyID;
        v_companyName  := companyRec.CompanyName;
        v_companyTrip  := 0;
        v_driverCount  := 0;
        v_hasActiveDrivers := FALSE;

        -- Check if this company has any trips at all
        SELECT COUNT(*)
        INTO v_companyTrip
        FROM DriverAllocation da
        JOIN Driver d ON da.DriverID = d.DriverID
        WHERE d.BusCompanyID = v_busCompanyID;
```

```
        -- Skip companies with 0 trips
        IF v_companyTrip = 0 THEN
            CONTINUE;
        END IF;

        v_companyCount := v_companyCount + 1;
        v_companyTrip := 0;

        -- Report header
        DBMS_OUTPUT.PUT_LINE(CHR(10));
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
        DBMS_OUTPUT.PUT_LINE('Bus Company Name: ' || v_companyName);
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
        DBMS_OUTPUT.PUT_LINE(RPAD('Driver ID', 12) || ' | ' ||
                             RPAD('Full Name', 40) || ' | ' ||
                             RPAD('Trips Done', 12) || ' | ' ||
                             RPAD('Most Popular Destination', 40));
        DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));

        FOR driverRec IN driverCursor(v_busCompanyID) LOOP
            v_driverID   := driverRec.DriverID;
            v_fullName   := driverRec.FullName;

            -- Count trips for this driver
            SELECT COUNT(da.ScheduleID)
            INTO v_driverTrip
            FROM DriverAllocation da
            WHERE da.DriverID = v_driverID;

            IF v_driverTrip > 0 THEN
                v_hasActiveDrivers := TRUE;

                -- Find most popular destination
                BEGIN
                    SELECT Destination
                    INTO v_popularDest
                    FROM (
                        SELECT s.Destination, COUNT(*) AS TripCount
                        FROM DriverAllocation da
                        JOIN Schedule s ON da.ScheduleID = s.ScheduleID
                        WHERE da.DriverID = v_driverID
                        GROUP BY s.Destination
                        ORDER BY TripCount DESC, s.Destination
                    )
                    WHERE ROWNUM = 1;
                EXCEPTION
                    WHEN NO_DATA_FOUND THEN
                        v_popularDest := 'N/A';
                END;

                DBMS_OUTPUT.PUT_LINE(RPAD(v_driverID, 12) || ' | ' ||
                                     RPAD(v_fullName, 40) || ' | ' ||
                                     RPAD(v_driverTrip, 12) || ' | ' ||
                                     RPAD(v_popularDest, 40));

                v_driverCount := v_driverCount + 1;
                v_companyTrip := v_companyTrip + v_driverTrip;
                v_overallTrip := v_overallTrip + v_driverTrip;
            END IF;
        END LOOP;

        -- Only show company footer if they have active drivers
        IF v_hasActiveDrivers THEN
            DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
            DBMS_OUTPUT.PUT_LINE('Total Active Drivers in ' ||
v_companyName || ': ' || v_driverCount);
            DBMS_OUTPUT.PUT_LINE('Total Trips for ' || v_companyName || ':
```

```
' || v_companyTrip);
            DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
        END IF;
    END LOOP;

    -- Final report footer (only show if there are companies with trips)
    IF v_companyCount > 0 THEN
        DBMS_OUTPUT.PUT_LINE(CHR(10));
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
        DBMS_OUTPUT.PUT_LINE('Total Bus Companies with Trips: ' ||
v_companyCount);
        DBMS_OUTPUT.PUT_LINE('Overall Trips Across All Companies: ' ||
v_overallTrip);
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 52, '=') || ' END OF REPORT ' ||
LPAD('=', 53, '='));
    ELSE
        DBMS_OUTPUT.PUT_LINE(CHR(10));
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 100, '='));
        DBMS_OUTPUT.PUT_LINE('NO BUS COMPANIES WITH TRIPS FOUND');
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 100, '='));
    END IF;
END;
/

EXEC prc_Edison_Report_1;
```

Sample Output:



```
################################################################################################################################################################
Bus Company Name: GoGoBus Malaysia
================================================================================================================================================================
Driver ID    | Full Name                              | Trips Done    | Most Popular Destination
----------------------------------------------------------------------------------------------------------------------------------------------------------------
D0005        | Jose Baker                             | 24            | Batu Gajah
D0001        | Matthew Brown                          | 32            | Butterworth
----------------------------------------------------------------------------------------------------------------------------------------------------------------
Total Active Drivers in GoGoBus Malaysia: 2
Total Trips for GoGoBus Malaysia: 56
================================================================================================================================================================


################################################################################################################################################################
Bus Company Name: Maju Mover Transport
================================================================================================================================================================
Driver ID    | Full Name                              | Trips Done    | Most Popular Destination
----------------------------------------------------------------------------------------------------------------------------------------------------------------
D0004        | Jeffery Wilson                         | 34            | Kajang
D0007        | Robert Wright                          | 34            | Johor Bahru Sentral
----------------------------------------------------------------------------------------------------------------------------------------------------------------
Total Active Drivers in Maju Mover Transport: 2
Total Trips for Maju Mover Transport: 68
================================================================================================================================================================
```

4.2.8 Report 2: Summary Report of Shop Rental Income

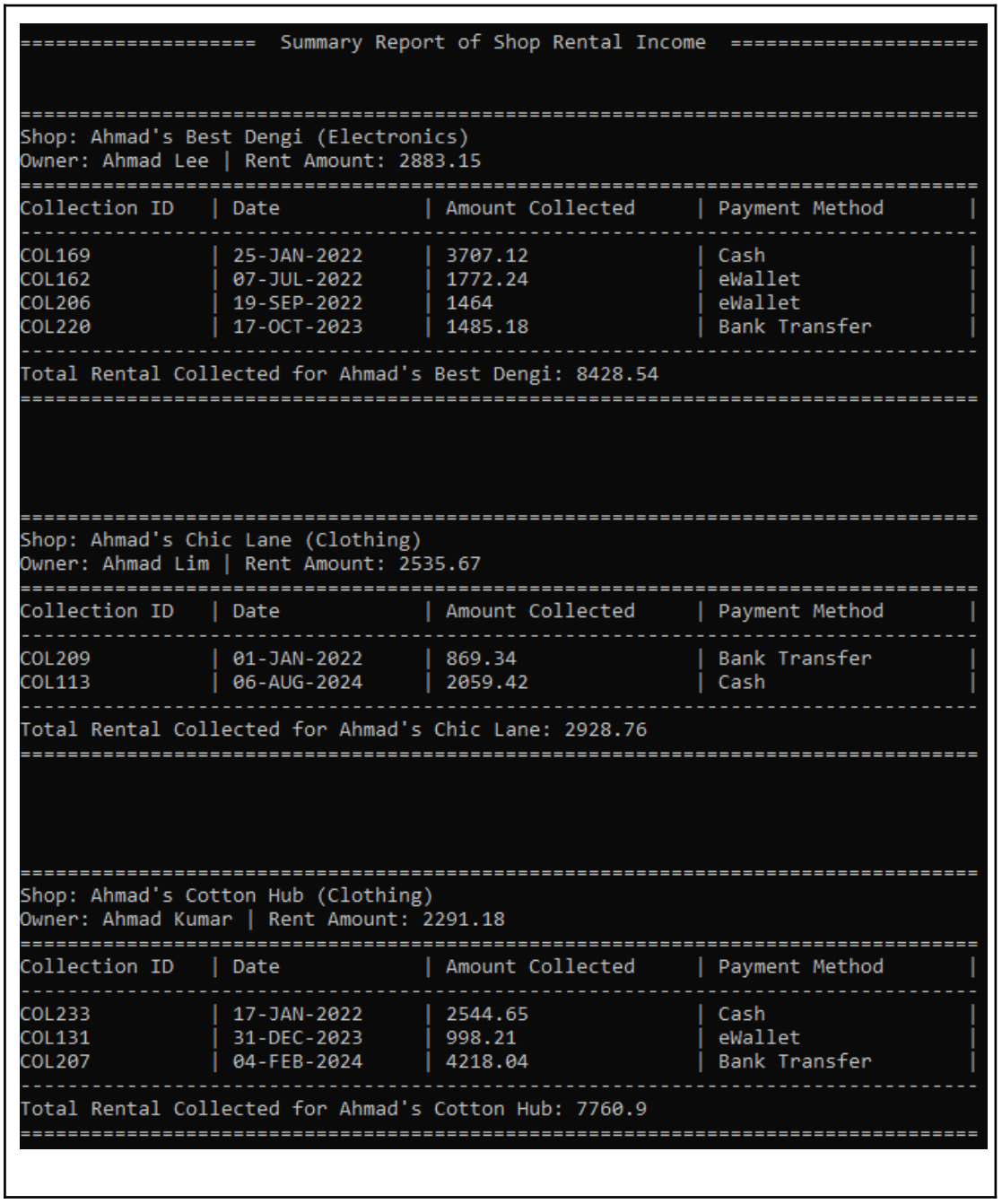
Purpose: Checks the total rental collected from all the shops, shown by the date, amount collected and payment collected. Serves to track the rental collection of all shops in the Bus Company database.

SQL Statement:

```
CL SCR
SET SERVEROUTPUT ON
SET LINESIZE 150
SET PAGESIZE 100
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

CREATE OR REPLACE PROCEDURE prc_Edison_Report_2 IS
    -- Shop variables
    v_ShopID        Shop.ShopID%TYPE;
    v_ShopName      Shop.ShopName%TYPE;
    v_ShopType      Shop.ShopType%TYPE;
    v_RentAmount    Shop.RentAmount%TYPE;
    v_OwnerName     Shop.OwnerName%TYPE;

    -- Rental variables
    v_CollectionID  RentalCollection.CollectionID%TYPE;
    v_CollectionDate RentalCollection.CollectionDate%TYPE;
    v_Amount        RentalCollection.AmountCollected%TYPE;
    v_PaymentMethod RentalCollection.PaymentMethod%TYPE;

    v_TotalRental   NUMBER := 0;
    v_OverallRental NUMBER := 0;

    -- Cursor for shops
    CURSOR shopCursor IS
        SELECT ShopID, ShopName, ShopType, RentAmount, OwnerName
        FROM Shop
        ORDER BY ShopName;

    -- Nested cursor for rentals per shop
    CURSOR rentalCursor(p_ShopID VARCHAR2) IS
        SELECT CollectionID, CollectionDate, AmountCollected,
PaymentMethod
        FROM RentalCollection
        WHERE ShopID = p_ShopID
        ORDER BY CollectionDate;
BEGIN
    v_OverallRental := 0;

    -- Loop through shops
    FOR shopRec IN shopCursor LOOP
        v_ShopID     := shopRec.ShopID;
        v_ShopName   := shopRec.ShopName;
        v_ShopType   := shopRec.ShopType;
        v_RentAmount := shopRec.RentAmount;
        v_OwnerName  := shopRec.OwnerName;
        v_TotalRental := 0;

        -- Shop header
        DBMS_OUTPUT.PUT_LINE(CHR(10));
        DBMS_OUTPUT.PUT_LINE(LPAD('#', 150, '#'));
        DBMS_OUTPUT.PUT_LINE('Shop: ' || v_ShopName || ' (' || v_ShopType
|| ')');
        DBMS_OUTPUT.PUT_LINE('Owner: ' || v_OwnerName || ' | Rent Amount:
' || v_RentAmount);
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 150, '='));
        DBMS_OUTPUT.PUT_LINE(RPAD('Collection ID', 15) || ' | ' ||
                             RPAD('Date', 15) || ' | ' ||
```

```
                              RPAD('Amount Collected', 20) || ' | ' ||
                              RPAD('Payment Method', 20));
        DBMS_OUTPUT.PUT_LINE(LPAD('-', 150, '-'));

        -- Loop rentals for this shop
        FOR rentalRec IN rentalCursor(v_ShopID) LOOP
            v_CollectionID  := rentalRec.CollectionID;
            v_CollectionDate:= rentalRec.CollectionDate;
            v_Amount        := rentalRec.AmountCollected;
            v_PaymentMethod := rentalRec.PaymentMethod;

            -- Print line
            DBMS_OUTPUT.PUT_LINE(RPAD(v_CollectionID, 15) || ' | ' ||

RPAD(TO_CHAR(v_CollectionDate,'DD-MON-YYYY'), 15) || ' | ' ||
                                 RPAD(v_Amount, 20) || ' | ' ||
                                 RPAD(v_PaymentMethod, 20));

            v_TotalRental   := v_TotalRental + v_Amount;
            v_OverallRental := v_OverallRental + v_Amount;
        END LOOP;

        -- Shop footer
        DBMS_OUTPUT.PUT_LINE(LPAD('-', 150, '-'));
        DBMS_OUTPUT.PUT_LINE('Total Rental Collected for ' || v_ShopName
|| ': ' || v_TotalRental);
        DBMS_OUTPUT.PUT_LINE(LPAD('=', 150, '='));
    END LOOP;

    -- Final footer
    DBMS_OUTPUT.PUT_LINE(CHR(10));
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 150, '='));
    DBMS_OUTPUT.PUT_LINE('Overall Rental Collected Across All Shops: ' ||
v_OverallRental);
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 60, '=') || ' END OF REPORT ' ||
LPAD('=', 60, '='));
END;
/

EXEC prc_Edison_Report_2;
```

Sample Output:



```
==================  Summary Report of Shop Rental Income  ====================


================================================================================
Shop: Ahmad's Best Dengi (Electronics)
Owner: Ahmad Lee | Rent Amount: 2883.15
================================================================================
Collection ID   | Date            | Amount Collected     | Payment Method       |
--------------------------------------------------------------------------------
COL169          | 25-JAN-2022     | 3707.12              | Cash                 |
COL162          | 07-JUL-2022     | 1772.24              | eWallet              |
COL206          | 19-SEP-2022     | 1464                 | eWallet              |
COL220          | 17-OCT-2023     | 1485.18              | Bank Transfer        |
--------------------------------------------------------------------------------
Total Rental Collected for Ahmad's Best Dengi: 8428.54
================================================================================




================================================================================
Shop: Ahmad's Chic Lane (Clothing)
Owner: Ahmad Lim | Rent Amount: 2535.67
================================================================================
Collection ID   | Date            | Amount Collected     | Payment Method       |
--------------------------------------------------------------------------------
COL209          | 01-JAN-2022     | 869.34               | Bank Transfer        |
COL113          | 06-AUG-2024     | 2059.42              | Cash                 |
--------------------------------------------------------------------------------
Total Rental Collected for Ahmad's Chic Lane: 2928.76
================================================================================




================================================================================
Shop: Ahmad's Cotton Hub (Clothing)
Owner: Ahmad Kumar | Rent Amount: 2291.18
================================================================================
Collection ID   | Date            | Amount Collected     | Payment Method       |
--------------------------------------------------------------------------------
COL233          | 17-JAN-2022     | 2544.65              | Cash                 |
COL131          | 31-DEC-2023     | 998.21               | eWallet              |
COL207          | 04-FEB-2024     | 4218.04              | Bank Transfer        |
--------------------------------------------------------------------------------
Total Rental Collected for Ahmad's Cotton Hub: 7760.9
================================================================================
```

4.3 KELLY TAN JIE LI

4.3.1 Query 1: Ranked Bus Reliability

Purpose: This query ranks buses based on their schedule reliability percentage, allowing the company to identify the best and worst performing buses in the fleet. It supports tactical decision-making by helping allocate reliable buses to key routes, plan maintenance for underperforming ones, and improve overall service quality in the bus ticketing system.

SQL Statement:

```
CREATE INDEX idx_schedule_status ON Schedule(UPPER(scheduleStatus));

SET LINESIZE 100
SET PAGESIZE 50
ALTER SESSION SET nls_date_format = 'DD-MM-YYYY';


TTITLE CENTER 'Bus Reliability Ranking as of ' _DATE SKIP 2

COLUMN BusID FORMAT A8 HEADING 'Bus ID'
COLUMN PlateNo FORMAT A12 HEADING 'Plate No'
COLUMN BusType FORMAT A20 HEADING 'Bus Type'
COLUMN TotalSchedules FORMAT 999999 HEADING 'Total Schedules'
COLUMN TotalCancelledSchedules FORMAT 999999 HEADING 'Total Cancellation'
COLUMN ReliabilityPercentage FORMAT 990.99 HEADING 'Reliability %'
COLUMN ReliabilityRank FORMAT 999 HEADING 'Rank'

CREATE OR REPLACE VIEW Ranked_Bus_Reliability AS
SELECT
    b.busID,
    b.plateNo,
    b.busType,
    s.totalSchedules,
    s.totalCancelledSchedules,
    ROUND(
      ((s.totalSchedules - s.totalCancelledSchedules) * 100.0 /
NULLIF(s.totalSchedules, 0)), 2
    ) AS reliabilityPercentage,
    DENSE_RANK() OVER (
      ORDER BY ((s.totalSchedules - s.totalCancelledSchedules) * 100.0 /
NULLIF(s.totalSchedules, 0)) DESC
    ) AS reliabilityRank
FROM Bus b
JOIN (
    SELECT busID,
           COUNT(*) AS totalSchedules,
           SUM(CASE WHEN UPPER(scheduleStatus) = 'CANCELLED' THEN 1 ELSE 0
END) AS totalCancelledSchedules
    FROM Schedule
    GROUP BY busID
) s ON b.busID = s.busID
ORDER BY reliabilityRank;

SELECT * FROM Ranked_Bus_Reliability;

DROP INDEX idx_schedule_status;

CLEAR COLUMNS
TTITLE OFF;
```

Sample Output:

Bus Reliability Ranking as of 06-09-2025

Bus ID	Plate No	Bus Type	Total Schedules	Total Cancellation	Reliability %	Rank
B0020	TUV0005	Double Decker	3	0	100.00	1
B0057	ABC0042	Standard	3	0	100.00	1
B0001	ABC1234	Standard	3	0	100.00	1
B0034	JKL0019	Double Decker	3	0	100.00	1
B0080	RST0065	Double Decker	3	0	100.00	1
B0025	IJK0010	Standard	3	0	100.00	1
B0023	CDE0008	Standard	3	0	100.00	1
B0051	IJK0036	Standard	3	0	100.00	1
B0010	OPQ4444	Double Decker	3	0	100.00	1
B0024	FGH0009	Double Decker	3	0	100.00	1
B0078	LMN0063	Double Decker	3	0	100.00	1
B0067	EFG0052	Standard	3	0	100.00	1
B0089	STU0074	Standard	3	0	100.00	1
B0099	WXY0084	Standard	4	1	75.00	2
B0091	YZA0076	Standard	4	1	75.00	2
B0090	VWX0075	Double Decker	4	1	75.00	2
B0060	JKL0045	Double Decker	3	1	66.67	3
B0100	ZAB0085	Double Decker	3	1	66.67	3
B0029	UVW0014	Standard	3	1	66.67	3
B0049	CDE0034	Standard	3	1	66.67	3
B0061	MNO0046	Standard	3	1	66.67	3
B0085	GHI0070	Standard	3	1	66.67	3

4.3.2 Query 2: Yearly Maintenance Spending Trends Across Bus Companies

Purpose: This query compares yearly maintenance costs across different bus companies, showing both cost differences and percentage changes over time. It supports strategic decision-making by helping management detect cost patterns, control inefficiencies, plan budgets, and forecast long-term financial needs for the bus ticketing system.

SQL Statement:

```
SET LINESIZE 100
SET PAGESIZE 50
ALTER SESSION SET nls_date_format = 'DD-MM-YYYY';

CREATE INDEX idx_maintenance_date ON Maintenance(serviceDate);

TTITLE CENTER 'Yearly Maintenance Spending Trends Across Bus Companies as
of ' _DATE SKIP 2
BREAK ON year SKIP 1

COLUMN companyName FORMAT A25 HEADING 'Company Name'
COLUMN year FORMAT 9999 HEADING 'Year'
COLUMN totalCost FORMAT 999999999.99 HEADING 'Total Cost'
COLUMN prevYearCost FORMAT 999999990.99 HEADING 'Previous Year Cost'
COLUMN costDifference FORMAT 999999999.99 HEADING 'Cost Difference'
COLUMN costChangePct FORMAT 990.99 HEADING 'Cost Change %'

CREATE OR REPLACE VIEW Bus_Company_Cost_Trend AS
SELECT
    bc.companyName,
    EXTRACT(YEAR FROM m.serviceDate) AS year,
    SUM(m.cost) AS totalCost,
    NVL(LAG(SUM(m.cost)) OVER (
        PARTITION BY bc.companyName
        ORDER BY EXTRACT(YEAR FROM m.serviceDate)
    ),0) AS prevYearCost,
    (SUM(m.cost) - NVL(LAG(SUM(m.cost)) OVER (
        PARTITION BY bc.companyName
        ORDER BY EXTRACT(YEAR FROM m.serviceDate)
    ),0) AS costDifference,
    ROUND(
```

```
        CASE
            WHEN NVL(LAG(SUM(m.cost)) OVER (
                 PARTITION BY bc.companyName
                 ORDER BY EXTRACT(YEAR FROM m.serviceDate)
                 ),0) = 0
            THEN 0
            ELSE ((SUM(m.cost) - NVL(LAG(SUM(m.cost)) OVER (
                 PARTITION BY bc.companyName
                 ORDER BY EXTRACT(YEAR FROM m.serviceDate)),0))
                 / NVL(LAG(SUM(m.cost)) OVER (
                 PARTITION BY bc.companyName
                 ORDER BY EXTRACT(YEAR FROM m.serviceDate)),0) * 100)
        END,2
    ) AS costChangePct
FROM Maintenance m
JOIN Bus b ON m.busID = b.busID
JOIN Bus_Company bc ON b.busCompanyID = bc.busCompanyID
GROUP BY bc.companyName, EXTRACT(YEAR FROM m.serviceDate)
ORDER BY year, totalCost DESC;

SELECT * FROM Bus_Company_Cost_Trend;

DROP INDEX idx_maintenance_date;

CLEAR COLUMNS
TTITLE OFF;
```

Sample Output:

Yearly Maintenance Spending Trends Across Bus Companies as of 07-09-2025

Company Name	Year	Total Cost	Previous Year Cost	Cost Difference	Cost Change %
Southern Star Coaches	2020	3690.52	0.00	3690.52	0.00
Sabah InterCity Transport		3224.36	0.00	3224.36	0.00
TransNation Express		2888.31	0.00	2888.31	0.00
MetroLine Travel Services		2459.21	0.00	2459.21	0.00
GoGoBus Malaysia		2403.25	0.00	2403.25	0.00
Eastline Travel		2006.32	0.00	2006.32	0.00
Northern Gateway Buses		1455.93	0.00	1455.93	0.00
Maju Mover Transport		1432.51	0.00	1432.51	0.00
Skybus Express		1352.90	0.00	1352.90	0.00
Borneo Highway Link		713.94	0.00	713.94	0.00
Northern Gateway Buses	2021	3443.52	1455.93	1987.59	136.52
Eastline Travel		3336.53	2006.32	1330.21	66.30
Maju Mover Transport		2792.67	1432.51	1360.16	94.95
MetroLine Travel Services		2316.88	2459.21	-142.33	-5.79
TransNation Express		2010.64	2888.31	-877.67	-30.39
Southern Star Coaches		1760.72	3690.52	-1929.80	-52.29
Skybus Express		1607.14	1352.90	254.24	18.79
Sabah InterCity Transport		1342.88	3224.36	-1881.48	-58.35
Borneo Highway Link		1170.92	713.94	456.98	64.01
GoGoBus Malaysia		1098.56	2403.25	-1304.69	-54.29
Sabah InterCity Transport	2022	3964.34	1342.88	2621.46	195.21
TransNation Express		2728.43	2010.64	717.79	35.70
Eastline Travel		2522.54	3336.53	-813.99	-24.40

4.3.3 Procedure 1: Add New Maintenance Record for a Bus

Purpose: The purpose of this procedure is to register a new maintenance activity for a bus by inserting its details into the maintenance records. It ensures that each maintenance entry is properly stored with relevant information such as the bus ID, maintenance date, and description of the work performed. This allows the system to track the history of bus maintenance systematically, supporting operational reliability and future reference.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_addNewMaintenance (
    v_busID         IN VARCHAR2,
    v_serviceTypeID IN VARCHAR2,
    v_cost          IN NUMBER,
    v_notes         IN VARCHAR2,
    v_serviceDate   IN DATE
) AS
    v_existsBus         NUMBER;
    v_existsService     NUMBER;
    v_new_maintenanceID Maintenance.MaintenanceID%TYPE;
    v_next_num          NUMBER;
    v_standardCost      NUMBER;
    v_final_cost        NUMBER;
    v_lastStatus        Maintenance.status%TYPE;

    ex_no_bus_found      EXCEPTION;
    ex_no_service_found  EXCEPTION;
    ex_bus_service_exists EXCEPTION;

BEGIN
    SELECT COUNT(*) INTO v_existsBus
    FROM Bus
    WHERE busID = v_busID;

    SELECT COUNT(*) INTO v_existsService
    FROM Service_Type
    WHERE serviceTypeID = v_serviceTypeID;

    IF v_existsBus = 0 THEN
        RAISE ex_no_bus_found;
    ELSIF v_existsService = 0 THEN
        RAISE ex_no_service_found;
    END IF;

    BEGIN
        SELECT status
        INTO v_lastStatus
        FROM (
           SELECT status
           FROM Maintenance
           WHERE busID = v_busID
           ORDER BY serviceDate DESC, maintenanceID DESC
        )
        WHERE ROWNUM = 1;

        IF UPPER(v_lastStatus) != 'COMPLETED' THEN
            RAISE ex_bus_service_exists;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            NULL;
    END;

    SELECT standardCost
    INTO v_standardCost
    FROM Service_Type
    WHERE serviceTypeID = v_serviceTypeID;

    v_final_cost := CASE
                       WHEN v_cost IS NULL OR v_cost < v_standardCost
                       THEN v_standardCost
                       ELSE v_cost
                    END;

    SELECT NVL(MAX(TO_NUMBER(SUBSTR(maintenanceID, 3))), 0) + 1
```

```
    INTO v_next_num
    FROM Maintenance;

    v_new_maintenanceID := 'MT' || LPAD(v_next_num, 5, '0');

    INSERT INTO Maintenance (maintenanceID, busID, serviceTypeID, cost,
notes, serviceDate, status)
    VALUES (v_new_maintenanceID, v_busID, v_serviceTypeID, v_final_cost,
v_notes, v_serviceDate, 'Scheduled');

    DBMS_OUTPUT.PUT_LINE('Maintenance ' || v_new_maintenanceID || ' added
for Bus ' || v_busID);
    DBMS_OUTPUT.PUT_LINE('Service Type: ' || v_serviceTypeID || ' | Cost:
' || v_final_cost);

EXCEPTION
    WHEN ex_no_bus_found THEN
        DBMS_OUTPUT.PUT_LINE('Bus ' || v_busID || ' not found.');
    WHEN ex_no_service_found THEN
        DBMS_OUTPUT.PUT_LINE('Service Type ' || v_serviceTypeID || ' not
found.');
    WHEN ex_bus_service_exists THEN
        DBMS_OUTPUT.PUT_LINE('Bus ' || v_busID || ' already has an ongoing
maintenance (not completed).');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/

-- Case1: Bus exists, service type exists, no existing incomplete
maintenance
EXEC prc_addNewMaintenance('B0097', 'SV001', 400, 'Annual inspection',
'30-09-2025');

-- Case2: Bus does not exist
EXEC prc_addNewMaintenance('B9999', 'SV003', 100, 'Non-existent bus',
SYSDATE);

-- Case3: Service type does not exist
EXEC prc_addNewMaintenance('B0097', 'SV999', 200, 'Non-existent service',
SYSDATE);

-- Case4: Bus already under maintenance and it's not complete yet
EXEC prc_addNewMaintenance('B0023', 'SV005', 500, 'Attempt to add while
already scheduled', SYSDATE);

-- Case5: Bus exists, service type exists, no existing incomplete
maintenance but with low cost
EXEC prc_addNewMaintenance('B0059', 'SV008', 2, 'Engine diagnostics with
low cost', '01-10-2025');
```

Sample Output:

```
SQL> EXEC prc_addNewMaintenance('B0097', 'SV001', 400, 'Annual inspection', '30-09-2025');
Maintenance MT00306 added for Bus B0097
Service Type: SV001 | Cost: 400

PL/SQL procedure successfully completed.

SQL> EXEC prc_addNewMaintenance('B9999', 'SV003', 100, 'Non-existent bus', SYSDATE);
Bus B9999 not found.

PL/SQL procedure successfully completed.

SQL> EXEC prc_addNewMaintenance('B0097', 'SV999', 200, 'Non-existent service', SYSDATE);
Service Type SV999 not found.

PL/SQL procedure successfully completed.

SQL> EXEC prc_addNewMaintenance('B0023', 'SV005', 500, 'Attempt to add while already scheduled', SYSDATE);
Bus B0023 already has an ongoing maintenance (not completed).

PL/SQL procedure successfully completed.

SQL> EXEC prc_addNewMaintenance('B0059', 'SV008', 2, 'Engine diagnostics with low cost', '01-10-2025');
Maintenance MT00307 added for Bus B0059
Service Type: SV008 | Cost: 392.93

PL/SQL procedure successfully completed.
```

4.3.4 Procedure 2: Search Bus Schedules with Passenger Details

Purpose: The purpose of this procedure is to allow searching of bus schedules based on a given date and destination, while retrieving detailed booking information. When executed, the procedure validates that schedules exist for the input date and destination. For each matching schedule, it lists booking records along with passenger details such as member ID, full name, phone number, and reserved seats. This helps operational staff quickly view passenger allocations for a schedule, improving coordination, monitoring, and customer support.

SQL Statement:

```
SET serveroutput on FORMAT WRAPPED
SET linesize 120
SET pagesize 100

CREATE OR REPLACE PROCEDURE Search_Bus_Schedules_Details (
      v_scheduleDate IN DATE,
      v_destination  IN VARCHAR2
  ) IS
      no_schedule_found EXCEPTION;
      no_destination_found EXCEPTION;

      v_scheduleID    Schedule.ScheduleID%TYPE;
      v_departureTime Schedule.DepartureTime%TYPE;
      v_destFetched   Schedule.Destination%TYPE;
      v_scheduleDateFetched Schedule.ScheduleDate%TYPE;
      v_status        Schedule.ScheduleStatus%TYPE;

      v_bookingID  Booking.BookingID%TYPE;
      v_memberID   Member.MemberID%TYPE;
      v_fullName   Member.FullName%TYPE;
      v_phoneNo    Member.PhoneNo%TYPE;

      v_seatNo Ticket.SeatNo%TYPE;

      v_count NUMBER := 0;
      v_count_schedule     NUMBER := 0;
      v_count_destination  NUMBER := 0;

      CURSOR cur_schedule IS
```

```
          SELECT s.ScheduleID, s.ScheduleDate, s.DepartureTime,
s.Destination, s.ScheduleStatus
          FROM Schedule s
          WHERE TRUNC(s.ScheduleDate) = TRUNC(v_scheduleDate)
            AND s.Destination = v_destination
          ORDER BY s.DepartureTime;

      CURSOR cur_booking IS
          SELECT DISTINCT b.BookingID, m.MemberID, m.FullName, m.PhoneNo
          FROM Ticket t
               JOIN BookingDetails bd ON t.TicketID = bd.TicketID
               JOIN Booking b ON bd.BookingID = b.BookingID
               JOIN Member m ON b.MemberID = m.MemberID
          WHERE t.ScheduleID = v_scheduleID;

      CURSOR cur_seat IS
          SELECT t.SeatNo
          FROM Ticket t
               JOIN BookingDetails bd ON t.TicketID = bd.TicketID
          WHERE t.ScheduleID = v_scheduleID
            AND bd.BookingID = v_bookingID
          ORDER BY t.SeatNo;

  BEGIN
      SELECT COUNT(*)
      INTO v_count_schedule
      FROM Schedule
      WHERE TRUNC(ScheduleDate) = TRUNC(v_scheduleDate);

      IF v_count_schedule = 0 THEN
          RAISE no_schedule_found;
      END IF;

      SELECT COUNT(*)
      INTO v_count_destination
      FROM Schedule
      WHERE TRUNC(ScheduleDate) = TRUNC(v_scheduleDate)
        AND Destination = v_destination;

      IF v_count_destination = 0 THEN
          RAISE no_destination_found;
      END IF;

      OPEN cur_schedule;
      LOOP
          FETCH cur_schedule INTO v_scheduleID, v_scheduleDateFetched,
v_departureTime, v_destFetched, v_status;
          EXIT WHEN cur_schedule%NOTFOUND;

          DBMS_OUTPUT.PUT_LINE(CHR(10));
          DBMS_OUTPUT.PUT_LINE(LPAD('=', 80, '='));
          DBMS_OUTPUT.PUT_LINE('ScheduleID     : ' || v_scheduleID);
          DBMS_OUTPUT.PUT_LINE('Schedule Date  : ' ||
TO_CHAR(v_scheduleDateFetched,'DD-MM-YYYY'));
          DBMS_OUTPUT.PUT_LINE('Departure Time : ' ||
TO_CHAR(v_departureTime,'HH24:MI'));
          DBMS_OUTPUT.PUT_LINE('Destination    : ' || v_destFetched);
          DBMS_OUTPUT.PUT_LINE('Status         : ' || v_status);
          DBMS_OUTPUT.PUT_LINE(LPAD('=', 80, '='));

          -- Heading row
          DBMS_OUTPUT.PUT_LINE(RPAD('BookingID', 11, ' ') || ' ' ||
                               RPAD('MemberID', 10, ' ') || ' ' ||
                               RPAD('Full Name', 26, ' ') || ' ' ||
                               RPAD('Phone No', 12, ' ') || ' ' ||
                               'Seat(s)');
          DBMS_OUTPUT.PUT_LINE(LPAD('-', 80, '-'));
```

```
          v_count := 0;

          OPEN cur_booking;
          LOOP
              FETCH cur_booking INTO v_bookingID, v_memberID, v_fullName,
v_phoneNo;
              EXIT WHEN cur_booking%NOTFOUND;

              OPEN cur_seat;
              FETCH cur_seat INTO v_seatNo;

              IF cur_seat%FOUND THEN
                  -- First seat line
                  DBMS_OUTPUT.PUT_LINE(RPAD(v_bookingID, 12) ||
                                       RPAD(v_memberID, 11) ||
                                       RPAD(v_fullName, 27) ||
                                       RPAD(v_phoneNo, 13) ||
                                       v_seatNo);
                  v_count := v_count + 1;

                  -- Remaining seats
                  LOOP
                      FETCH cur_seat INTO v_seatNo;
                      EXIT WHEN cur_seat%NOTFOUND;
                      DBMS_OUTPUT.PUT_LINE(RPAD(' ', 63) || v_seatNo);
                      v_count := v_count + 1;
                  END LOOP;
              END IF;

              CLOSE cur_seat;
          END LOOP;
          CLOSE cur_booking;

          DBMS_OUTPUT.PUT_LINE(LPAD('=', 80, '='));
          DBMS_OUTPUT.PUT_LINE('Total Passenger: ' || v_count);
          DBMS_OUTPUT.PUT_LINE(CHR(10));
      END LOOP;
      CLOSE cur_schedule;

  EXCEPTION
      WHEN no_schedule_found THEN
          DBMS_OUTPUT.PUT_LINE('No schedules found on the given date.');
      WHEN no_destination_found THEN
          DBMS_OUTPUT.PUT_LINE('Destination not found for the given
date.');
      WHEN OTHERS THEN
          DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
  END;
  /


-- Test 1: Valid input with schedules date and destination
EXEC Search_Bus_Schedules_Details('30-09-2024', 'Johor Bahru Sentral');

-- Test 2: No schedules on this date
EXEC Search_Bus_Schedules_Details('30-09-9999', 'Johor Bahru Sentral');

-- Test 3: Valid date but no such destination
EXEC Search_Bus_Schedules_Details('30-09-2024', 'Nonexistent Town');
```

Sample Output:

```
SQL> EXEC Search_Bus_Schedules_Details('30-09-2024', 'Johor Bahru Sentral');


================================================================================
ScheduleID     : SCH0300
Schedule Date  : 30-09-2024
Departure Time : 22:45
Destination    : Johor Bahru Sentral
Status         : Active
================================================================================
BookingID   MemberID   Full Name                 Phone No     Seat(s)
--------------------------------------------------------------------------------
BK0074      M0004      Lim Chee How              0137894561   C3
                                                              C4
                                                              C5
                                                              C6
                                                              C7
                                                              C8
================================================================================
Total Passenger: 6



PL/SQL procedure successfully completed.

SQL>
SQL> -- Test 2: No schedules on this date
SQL> EXEC Search_Bus_Schedules_Details('30-09-9999', 'Johor Bahru Sentral');
No schedules found on the given date.

PL/SQL procedure successfully completed.

SQL>
SQL> -- Test 3: Valid date but no such destination
SQL> EXEC Search_Bus_Schedules_Details('30-09-2024', 'Nonexistent Town');
Destination not found for the given date.

PL/SQL procedure successfully completed.
```

4.3.5 Trigger 1: Update Bus Status and Cancel Conflicting Schedules During Maintenance

Purpose: The purpose of this trigger is to automatically update the status of a bus to “Under Maintenance” whenever a new maintenance record is inserted. It also identifies and cancels any active schedules for the same bus that fall within the calculated maintenance period based on the service type’s duration. This ensures that buses under maintenance are not assigned to trips, prevents schedule conflicts, and maintains the accuracy of the operational timetable.

SQL Statement:

```
CREATE OR REPLACE TRIGGER trg_bus_maintenance_update
AFTER INSERT ON Maintenance
FOR EACH ROW
DECLARE
    v_duration_days   NUMBER;
    v_end_date        DATE;
    v_old_status      Bus.Status%TYPE;
    v_cancelled_count NUMBER := 0;

    CURSOR c_schedules IS
        SELECT scheduleID, scheduleDate, departureTime, arrivalTime
          FROM Schedule
         WHERE busID = :NEW.busID
           AND UPPER(scheduleStatus) = 'ACTIVE'
```

```
           AND scheduleDate BETWEEN :NEW.serviceDate AND v_end_date;

    v_scheduleID    Schedule.scheduleID%TYPE;
    v_scheduleDate  Schedule.scheduleDate%TYPE;
    v_departureTime Schedule.departureTime%TYPE;
    v_arrivalTime   Schedule.arrivalTime%TYPE;
BEGIN
    SELECT CEIL(estimatedDuration/24)
      INTO v_duration_days
      FROM Service_Type
     WHERE serviceTypeID = :NEW.serviceTypeID;

    v_end_date := :NEW.serviceDate + v_duration_days;

    SELECT Status
      INTO v_old_status
      FROM Bus
     WHERE busID = :NEW.busID;

    IF UPPER(v_old_status) <> 'UNDER MAINTENANCE' THEN
        UPDATE Bus
           SET Status = 'Under Maintenance'
         WHERE busID = :NEW.busID;

        DBMS_OUTPUT.PUT_LINE('Bus '||:NEW.busID||
                             ' status changed from "'||v_old_status||
                             '" to "Under Maintenance".');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Bus '||:NEW.busID||
                             ' status remains "Under Maintenance".');
    END IF;

    OPEN c_schedules;
    LOOP
        FETCH c_schedules INTO v_scheduleID, v_scheduleDate,
v_departureTime, v_arrivalTime;
        EXIT WHEN c_schedules%NOTFOUND;

        UPDATE Schedule
           SET scheduleStatus = 'Cancelled'
         WHERE scheduleID = v_scheduleID;

        v_cancelled_count := v_cancelled_count + 1;

        DBMS_OUTPUT.PUT_LINE('Cancelled schedule '||v_scheduleID||
                             ' on
'||TO_CHAR(v_scheduleDate,'DD-MON-YYYY')||
                             ' ('||TO_CHAR(v_departureTime,'HH24:MI')||' -
'||
                                  TO_CHAR(v_arrivalTime,'HH24:MI')||')');
    END LOOP;
    CLOSE c_schedules;

    IF v_cancelled_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No active schedules affected for Bus
'||:NEW.busID||
                             ' during maintenance period '||
                             TO_CHAR(:NEW.serviceDate,'DD-MON-YYYY')||' to
'||
                             TO_CHAR(v_end_date,'DD-MON-YYYY')||'.');
    END IF;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error in trigger: '||SQLERRM);
END;
/
```

```
--Case 1: Maintenance but no schedules affected
-- Bus B0020 Active, but schedules exist only in 2023 (past)
INSERT INTO Maintenance
(MaintenanceID, BusID, ServiceTypeID, Cost, Notes, ServiceDate, Status)
VALUES ('MT10001', 'B0020', 'SV004', 0.0, DEFAULT, TO_DATE('15-Aug-2025',
'DD-MON-YYYY'), 'Scheduled');

--Case 2: Bus already "Under Maintenance"
-- B0014 already "Under Maintenance"
INSERT INTO Maintenance
(MaintenanceID, BusID, ServiceTypeID, Cost, Notes, ServiceDate, Status)
VALUES ('MT10002', 'B0014', 'SV003', 0.0, DEFAULT, TO_DATE('05-Aug-2025',
'DD-MON-YYYY'), 'Scheduled');

--Case 3: Multiple schedules cancelled
-- B0023 has schedules across different months
-- SV005 estimatedDuration = 97 hrs (~5 days)
-- Add two active schedules for B0023 within July 2025
INSERT INTO Schedule (ScheduleID, BusID, DepartureTime, ArrivalTime,
Station, Platform, Destination, ScheduleDate, ScheduleStatus)
VALUES ('SCH2001', 'B0023', TO_DATE('02-Jul-2025 09:00', 'DD-MON-YYYY
HH24:MI'),
        TO_DATE('02-Jul-2025 12:00', 'DD-MON-YYYY HH24:MI'),
        'Kuala Lumpur Central', 'Platform 2', 'Penang Sentral',
TO_DATE('02-Jul-2025', 'DD-MON-YYYY'), 'Active');

INSERT INTO Schedule (ScheduleID, BusID, DepartureTime, ArrivalTime,
Station, Platform, Destination, ScheduleDate, ScheduleStatus)
VALUES ('SCH2002', 'B0023', TO_DATE('04-Jul-2025 14:00', 'DD-MON-YYYY
HH24:MI'),
        TO_DATE('04-Jul-2025 17:30', 'DD-MON-YYYY HH24:MI'),
        'Ipoh', 'Platform 3', 'Johor Bahru Sentral',
TO_DATE('04-Jul-2025', 'DD-MON-YYYY'), 'Active');

INSERT INTO Maintenance
(MaintenanceID, BusID, ServiceTypeID, Cost, Notes, ServiceDate, Status)
VALUES ('MT10003', 'B0023', 'SV005', 0.0, DEFAULT, TO_DATE('01-Jul-2025',
'DD-MON-YYYY'), 'Scheduled');
```

Sample Output:

```
SQL> INSERT INTO Maintenance
  2  (MaintenanceID, BusID, ServiceTypeID, Cost, Notes, ServiceDate, Status)
  3  VALUES ('MT10002', 'B0020', 'SV004', 0.0, DEFAULT, TO_DATE('15-Aug-2025', 'DD-MON-YYYY'), 'Scheduled');
Bus B0020 status changed from "Active" to "Under Maintenance".
No active schedules affected for Bus B0020 during maintenance period 15-AUG-2025 to 17-AUG-2025.

1 row created.

SQL> INSERT INTO Maintenance
  2  (MaintenanceID, BusID, ServiceTypeID, Cost, Notes, ServiceDate, Status)
  3  VALUES ('MT10004', 'B0014', 'SV003', 0.0, DEFAULT, TO_DATE('05-Aug-2025', 'DD-MON-YYYY'), 'Scheduled');
Bus B0014 status remains "Under Maintenance".
No active schedules affected for Bus B0014 during maintenance period 05-AUG-2025 to 12-AUG-2025.

1 row created.

SQL> INSERT INTO Schedule (ScheduleID, BusID, DepartureTime, ArrivalTime, Station, Platform, Destination, ScheduleDate, ScheduleStatus)
  2  VALUES ('SCH2001', 'B0023', TO_DATE('02-Jul-2025 09:00', 'DD-MON-YYYY HH24:MI'),
  3          TO_DATE('02-Jul-2025 12:00', 'DD-MON-YYYY HH24:MI'),
  4          'Kuala Lumpur Central', 'Platform 2', 'Penang Sentral', TO_DATE('02-Jul-2025', 'DD-MON-YYYY'), 'Active');

1 row created.

SQL>
SQL> INSERT INTO Schedule (ScheduleID, BusID, DepartureTime, ArrivalTime, Station, Platform, Destination, ScheduleDate, ScheduleStatus)
  2  VALUES ('SCH2002', 'B0023', TO_DATE('04-Jul-2025 14:00', 'DD-MON-YYYY HH24:MI'),
  3          TO_DATE('04-Jul-2025 17:30', 'DD-MON-YYYY HH24:MI'),
  4          'Ipoh', 'Platform 3', 'Johor Bahru Sentral', TO_DATE('04-Jul-2025', 'DD-MON-YYYY'), 'Active');

1 row created.

SQL>
SQL> INSERT INTO Maintenance
  2  (MaintenanceID, BusID, ServiceTypeID, Cost, Notes, ServiceDate, Status)
  3  VALUES ('MT20003', 'B0023', 'SV005', 0.0, DEFAULT, TO_DATE('01-Jul-2025', 'DD-MON-YYYY'), 'Scheduled');
Bus B0023 status changed from "Active" to "Under Maintenance".
Cancelled schedule SCH0123 on 05-JUL-2025 (13:15 - 15:31)
Cancelled schedule SCH2001 on 02-JUL-2025 (09:00 - 12:00)
Cancelled schedule SCH2002 on 04-JUL-2025 (14:00 - 17:30)

1 row created.
```

4.3.6 Trigger 2: Audit Maintenance Status Changes

Purpose: The purpose of this trigger is to automatically record all insert, update, and delete operations performed on the Maintenance table into an audit log. It captures key details such as the maintenance ID, bus ID, old and new status values, the type of operation, the user who made the change, and the exact time of modification. It also enforces data integrity by preventing invalid status values outside of Scheduled, Completed, or Cancelled. This ensures that all changes to maintenance records are traceable, supporting accountability, compliance, and operational transparency in the bus management system.

SQL Statement:

```
SET linesize 180
SET pagesize 50

DROP SEQUENCE maintenanceStatus_audit_seq;
DROP TABLE MaintenanceStatus_Audit;

CREATE SEQUENCE maintenanceStatus_audit_seq
MINVALUE 1
MAXVALUE 99999
START WITH 1
INCREMENT BY 1
NOCACHE;

CREATE TABLE MaintenanceStatus_Audit (
    AuditID       NUMBER PRIMARY KEY,
    MaintenanceID VARCHAR2(10) NOT NULL,
    BusID         VARCHAR2(10),
    Status_Old    VARCHAR2(20),
    Status_New    VARCHAR2(20),
    Operation     VARCHAR2(10),
    ChangedBy     VARCHAR2(15),
    ChangedAt     VARCHAR2(10)
);
```

```
CREATE OR REPLACE TRIGGER trg_MaintenanceStatus_Audit
BEFORE INSERT OR UPDATE OR DELETE ON Maintenance
FOR EACH ROW
DECLARE
    ex_invalid_status EXCEPTION;
    PRAGMA EXCEPTION_INIT(ex_invalid_status, -20011);
BEGIN
    IF INSERTING OR UPDATING THEN
        IF UPPER(:NEW.Status) NOT IN ('SCHEDULED', 'COMPLETED',
'CANCELLED') THEN
            RAISE ex_invalid_status;
        END IF;
    END IF;

    IF INSERTING THEN
        INSERT INTO MaintenanceStatus_Audit (
            AuditID, MaintenanceID, BusID,
            Status_Old, Status_New, Operation, ChangedBy, ChangedAt
        )
        VALUES (
            maintenanceStatus_audit_seq.NEXTVAL,
            :NEW.MaintenanceID, :NEW.BusID,
            NULL, :NEW.Status, 'INSERT', USER,
            TO_CHAR(SYSDATE,'HH24:MI:SS')
        );

    ELSIF UPDATING THEN
        IF :OLD.Status <> :NEW.Status THEN
            INSERT INTO MaintenanceStatus_Audit (
                AuditID, MaintenanceID, BusID,
                Status_Old, Status_New, Operation, ChangedBy, ChangedAt
            )
            VALUES (
                maintenanceStatus_audit_seq.NEXTVAL,
                :NEW.MaintenanceID, :NEW.BusID,
                :OLD.Status, :NEW.Status, 'UPDATE', USER,
                TO_CHAR(SYSDATE,'HH24:MI:SS')
            );
        END IF;

    ELSIF DELETING THEN
        INSERT INTO MaintenanceStatus_Audit (
            AuditID, MaintenanceID, BusID,
            Status_Old, Status_New, Operation, ChangedBy, ChangedAt
        )
        VALUES (
            maintenanceStatus_audit_seq.NEXTVAL,
            :OLD.MaintenanceID, :OLD.BusID,
            :OLD.Status, NULL, 'DELETE', USER,
            TO_CHAR(SYSDATE,'HH24:MI:SS')
        );
    END IF;

EXCEPTION
    WHEN ex_invalid_status THEN
        RAISE_APPLICATION_ERROR(-20011, 'Invalid status value. Must be
Scheduled, Completed, or Cancelled.');
END;
/

-- Insert
INSERT INTO Maintenance VALUES ('MT30001', 'B0007', 'SV001', 100, NULL,
SYSDATE, 'Scheduled');
SELECT * FROM MaintenanceStatus_Audit;

-- Insert with invalid status
```

```
INSERT INTO Maintenance
VALUES ('MT30002', 'B0007', 'SV001', 123, NULL, SYSDATE, 'InProgress');
SELECT * FROM MaintenanceStatus_Audit;

-- Update (status change)
UPDATE Maintenance SET Status = 'Completed' WHERE MaintenanceID =
'MT30001';
SELECT * FROM MaintenanceStatus_Audit;

-- Delete
DELETE FROM Maintenance WHERE MaintenanceID = 'MT30001';
SELECT * FROM MaintenanceStatus_Audit;
```

Sample Output:

```
SQL> INSERT INTO Maintenance VALUES ('MT30001', 'B0007', 'SV001', 100, NULL, SYSDATE, 'Scheduled');

1 row created.

SQL> SELECT * FROM MaintenanceStatus_Audit;

   AUDITID MAINTENANC BUSID      STATUS_OLD           STATUS_NEW           OPERATION  CHANGEDBY       CHANGEDAT
---------- ---------- ---------- -------------------- -------------------- ---------- --------------- ----------
         1 MT30001    B0007                           Scheduled            INSERT     PERSONAL        08:19:45
```

```
SQL> INSERT INTO Maintenance
  2  VALUES ('MT30002', 'B0007', 'SV001', 123, NULL, SYSDATE, 'InProgress');
INSERT INTO Maintenance
            *
ERROR at line 1:
ORA-20011: Invalid status value. Must be Scheduled, Completed, or Cancelled.
ORA-06512: at "PERSONAL.TRG_MAINTENANCESTATUS_AUDIT", line 52
ORA-04088: error during execution of trigger 'PERSONAL.TRG_MAINTENANCESTATUS_AUDIT'


SQL> SELECT * FROM MaintenanceStatus_Audit;

   AUDITID MAINTENANC BUSID      STATUS_OLD           STATUS_NEW           OPERATION  CHANGEDBY       CHANGEDAT
---------- ---------- ---------- -------------------- -------------------- ---------- --------------- ----------
         1 MT30001    B0007                           Scheduled            INSERT     PERSONAL        08:19:45
```

```
SQL> UPDATE Maintenance SET Status = 'Completed' WHERE MaintenanceID = 'MT30001';

1 row updated.

SQL> SELECT * FROM MaintenanceStatus_Audit;

   AUDITID MAINTENANC BUSID      STATUS_OLD           STATUS_NEW           OPERATION  CHANGEDBY       CHANGEDAT
---------- ---------- ---------- -------------------- -------------------- ---------- --------------- ----------
         1 MT30001    B0007                           Scheduled            INSERT     PERSONAL        08:19:45
         2 MT30001    B0007      Scheduled            Completed            UPDATE     PERSONAL        08:19:45
```

```
SQL> DELETE FROM Maintenance WHERE MaintenanceID = 'MT30001';

1 row deleted.

SQL> SELECT * FROM MaintenanceStatus_Audit;

   AUDITID MAINTENANC BUSID      STATUS_OLD           STATUS_NEW           OPERATION  CHANGEDBY       CHANGEDAT
---------- ---------- ---------- -------------------- -------------------- ---------- --------------- ----------
         1 MT30001    B0007                           Scheduled            INSERT     PERSONAL        08:19:45
         2 MT30001    B0007      Scheduled            Completed            UPDATE     PERSONAL        08:19:45
         3 MT30001    B0007      Completed                                 DELETE     PERSONAL        08:19:45
```

4.3.7 Report 1: Bus Company Maintenance Report

Purpose: The purpose of this procedure is to generate a detailed maintenance report for each bus company. It summarizes maintenance activities by bus and service type, showing maintenance counts and costs. For each company, totals are calculated for cost, services taken, distinct service types, and buses involved. At the end, an overall summary is provided across all companies, including identification of the company with the highest total maintenance cost.

SQL Statement:

```
SET serveroutput on FORMAT WRAPPED
SET linesize 125
SET pagesize 100
SET DEFINE OFF

CREATE OR REPLACE PROCEDURE prc_buscom_maint_rpt IS

  v_busCompanyID   Bus_Company.BusCompanyID%TYPE;
  v_companyName    Bus_Company.CompanyName%TYPE;

  v_busID          Bus.BusID%TYPE;
  v_serviceName    Service_Type.ServiceName%TYPE;
  v_maintCount     NUMBER;
  v_totalCost      NUMBER(12,2);

  v_companyGrandTotal   NUMBER(15,2);
  v_totalServicesTaken  NUMBER;
  v_distinctServices    NUMBER;
  v_busCount            NUMBER;

  v_overallTotal        NUMBER(18,2) := 0;
  v_overallBusCount     NUMBER := 0;
  v_overallServiceTaken NUMBER := 0;
  v_overallDistinct     NUMBER := 0;

  v_maxCompanyTotal     NUMBER(15,2) := 0;
  v_maxCompanyName      Bus_Company.CompanyName%TYPE;
  v_maxCompanyID        Bus_Company.BusCompanyID%TYPE;

  v_lastBusID   Bus.BusID%TYPE := NULL;
  v_firstRow    BOOLEAN := TRUE;

  -- Outer cursor: Companies
  CURSOR companyCur IS
    SELECT BusCompanyID, CompanyName
    FROM Bus_Company
    ORDER BY BusCompanyID;

  -- Inner cursor: Maintenance per bus & service
  CURSOR maintCur IS
    SELECT b.BusID, st.ServiceName,
           COUNT(m.MaintenanceID) AS maint_count,
           SUM(m.Cost) AS total_cost
    FROM Bus b
         JOIN Maintenance m ON b.BusID = m.BusID
         JOIN Service_Type st ON m.ServiceTypeID = st.ServiceTypeID
    WHERE b.BusCompanyID = v_busCompanyID
    GROUP BY b.BusID, st.ServiceName
    ORDER BY b.BusID, st.ServiceName;

BEGIN
  OPEN companyCur;
  LOOP
    FETCH companyCur INTO v_busCompanyID, v_companyName;
    EXIT WHEN companyCur%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(CHR(10) || LPAD('=', 120, '='));
    DBMS_OUTPUT.PUT_LINE('Maintenance Report for Bus Company: ' ||
v_companyName || ' (' || v_busCompanyID || ')');
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
    DBMS_OUTPUT.PUT_LINE(RPAD('Bus ID', 10, ' ') || ' ' ||
                         RPAD('Service Type', 30, ' ') || ' ' ||
                         RPAD('Maint Count', 15, ' ') || ' ' ||
                         RPAD('Total Cost', 15, ' '));
    DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
```

```
    v_companyGrandTotal := 0;
    v_totalServicesTaken := 0;
    v_busCount := 0;
    v_lastBusID := NULL;

    SELECT COUNT(DISTINCT st.ServiceName)
    INTO v_distinctServices
    FROM Bus b
         JOIN Maintenance m ON b.BusID = m.BusID
         JOIN Service_Type st ON m.ServiceTypeID = st.ServiceTypeID
    WHERE b.BusCompanyID = v_busCompanyID;

    OPEN maintCur;
    LOOP
      FETCH maintCur INTO v_busID, v_serviceName, v_maintCount,
v_totalCost;
      EXIT WHEN maintCur%NOTFOUND;

      IF v_lastBusID IS NULL OR v_lastBusID <> v_busID THEN
        IF v_lastBusID IS NOT NULL THEN
          DBMS_OUTPUT.PUT_LINE(CHR(10)); -- blank line between buses
        END IF;
        v_lastBusID := v_busID;
        v_firstRow := TRUE;
        v_busCount := v_busCount + 1;
      END IF;

      IF v_firstRow THEN
        DBMS_OUTPUT.PUT_LINE(RPAD(v_busID, 10, ' ') || ' ' ||
                             RPAD(v_serviceName, 30, ' ') || ' ' ||
                             RPAD(TO_CHAR(v_maintCount), 15, ' ') || ' '
||
                             RPAD(TO_CHAR(v_totalCost, '$999,999.99'), 15,
' '));
        v_firstRow := FALSE;
      ELSE
        DBMS_OUTPUT.PUT_LINE(RPAD(' ', 11, ' ') ||
                             RPAD(v_serviceName, 30, ' ') || ' ' ||
                             RPAD(TO_CHAR(v_maintCount), 15, ' ') || ' '
||
                             RPAD(TO_CHAR(v_totalCost, '$999,999.99'), 15,
' '));
      END IF;

      v_companyGrandTotal := v_companyGrandTotal + NVL(v_totalCost,0);
      v_totalServicesTaken := v_totalServicesTaken + v_maintCount;

    END LOOP;
    CLOSE maintCur;

    DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
    DBMS_OUTPUT.PUT_LINE('Company Total Maintenance Cost: ' ||
TO_CHAR(v_companyGrandTotal, '$999,999,999.99'));
    DBMS_OUTPUT.PUT_LINE('Total Services Taken (with duplicates): ' ||
v_totalServicesTaken);
    DBMS_OUTPUT.PUT_LINE('Distinct Service Types (unique): ' ||
v_distinctServices);
    DBMS_OUTPUT.PUT_LINE('Total Buses Involved: ' || v_busCount);

    IF v_companyGrandTotal > v_maxCompanyTotal THEN
      v_maxCompanyTotal := v_companyGrandTotal;
      v_maxCompanyName := v_companyName;
      v_maxCompanyID := v_busCompanyID;
    END IF;

    v_overallTotal := v_overallTotal + v_companyGrandTotal;
```

```
    v_overallBusCount := v_overallBusCount + v_busCount;
    v_overallServiceTaken := v_overallServiceTaken + v_totalServicesTaken;
    v_overallDistinct := v_overallDistinct + v_distinctServices;

  END LOOP;
  CLOSE companyCur;

  DBMS_OUTPUT.PUT_LINE(CHR(10) || LPAD('-', 120, '-'));
  DBMS_OUTPUT.PUT_LINE(RPAD(' ', 45, ' ') || '  OVERALL COMPANIES SUMMARY
' || LPAD(' ', 45, ' '));
  DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
  DBMS_OUTPUT.PUT_LINE('Overall Total Maintenance Cost: ' ||
TO_CHAR(v_overallTotal, '$99,999,999,999.99'));
  DBMS_OUTPUT.PUT_LINE('Overall Services Taken: ' ||
v_overallServiceTaken);
  DBMS_OUTPUT.PUT_LINE('Overall Distinct Service Types: ' ||
v_overallDistinct);
  DBMS_OUTPUT.PUT_LINE('Overall Buses Involved: ' || v_overallBusCount);
  DBMS_OUTPUT.PUT_LINE('Company with Highest Maintenance Cost: ' ||
v_maxCompanyName ||
                       ' (' || v_maxCompanyID || ') - ' ||
TO_CHAR(v_maxCompanyTotal, '$999,999,999.99'));
  DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));

END;
/

EXEC prc_buscom_maint_rpt
```

Sample Output:

```
========================================================================================================================
Maintenance Report for Bus Company: MetroLine Travel Services (BC007)
========================================================================================================================
Bus ID     Service Type                Maint Count     Total Cost
------------------------------------------------------------------------------------------------------------------------
B0007      Engine Diagnostics          2               $786.30
           Tire Rotation               1               $438.53

B0017      Brake Inspection            1               $236.22
           Tire Rotation               2               $823.37

B0027      Oil Change                  1               $264.05

B0037      Brake Inspection            1               $236.22
           Transmission Service        1               $560.50

B0047      Brake Inspection            1               $236.22
           Tire Rotation               1               $438.53
           Wheel Alignment             1               $878.65

B0057      Brake Inspection            1               $236.22
           Engine Diagnostics          1               $393.15

B0067      Battery Replacement         1               $465.18
           Engine Diagnostics          1               $504.19

B0077      Tire Rotation               1               $438.53

B0087      Brake Inspection            1               $696.97
           Car Wash                    1               $320.79
           Wheel Alignment             1               $325.75

B0097      Tire Rotation               2               $823.37
           Transmission Service        1               $449.46
========================================================================================================================
Company Total Maintenance Cost:       $9,552.20
Total Services Taken (with duplicates): 23
Distinct Service Types (unique): 8
Total Buses Involved: 10
```

```
--------------------------------------------------------------------------------------------------------------------
                                        OVERALL COMPANIES SUMMARY
--------------------------------------------------------------------------------------------------------------------
Overall Total Maintenance Cost:         $147,242.49
Overall Services Taken: 305
Overall Distinct Service Types: 86
Overall Buses Involved: 98
Company with Highest Maintenance Cost: Sabah InterCity Transport (BC009) -       $19,079.85
--------------------------------------------------------------------------------------------------------------------
```

4.3.8 Report 2: Bus Schedule Summary Report

Purpose: This report generates a summary of bus schedules for each bus company. For every company, it lists each bus along with the number of total schedules, active schedules, and cancelled schedules. It calculates the utilization percentage and cancellation percentage. The report also identifies the bus with the most active schedules and the bus with the most cancelled schedules for each company. An overall section aggregates totals across all companies, showing total schedules, total active schedules, total cancelled schedules, total buses, and highlights the buses with the highest utilization and highest cancellation percentages across all companies.

SQL Statement:

```
SET SERVEROUTPUT ON FORMAT WRAPPED
CREATE OR REPLACE PROCEDURE prc_bus_schedule_summary IS
  v_totalSchedules    NUMBER;
  v_totalActive       NUMBER;
  v_totalCancelled    NUMBER;
  v_totalBuses        NUMBER;
  v_maxActiveBus      Bus.BusID%TYPE;
  v_maxActiveTrips    NUMBER;
  v_maxCancelledBus   Bus.BusID%TYPE;
  v_maxCancelledTrips NUMBER;

  v_overallTotalSchedules   NUMBER := 0;
  v_overallTotalActive      NUMBER := 0;
  v_overallTotalCancelled   NUMBER := 0;
  v_overallTotalBuses       NUMBER := 0;
  v_bestUtilBus             Bus.BusID%TYPE;
  v_bestUtilPct             NUMBER := -1;
  v_bestCancelBus           Bus.BusID%TYPE;
  v_bestCancelPct           NUMBER := -1;

  -- Outer cursor: companies
  CURSOR companyCur IS
    SELECT BusCompanyID, CompanyName
    FROM Bus_Company
    ORDER BY BusCompanyID;

  companyRec companyCur%ROWTYPE;

  -- Inner cursor: buses per company
  CURSOR busCur IS
    SELECT b.BusID,
           COUNT(s.ScheduleID) AS total_schedules,
           SUM(CASE WHEN UPPER(s.ScheduleStatus) = 'ACTIVE' THEN 1 ELSE 0
END) AS active_schedules,
           SUM(CASE WHEN UPPER(s.ScheduleStatus) = 'CANCELLED' THEN 1 ELSE
0 END) AS cancelled_schedules
    FROM Bus b
    JOIN Schedule s ON b.BusID = s.BusID
    WHERE b.BusCompanyID = companyRec.BusCompanyID
    GROUP BY b.BusID
```

```
    ORDER BY b.BusID;

  busRec busCur%ROWTYPE;

BEGIN
  OPEN companyCur;
  LOOP
    FETCH companyCur INTO companyRec;
    EXIT WHEN companyCur%NOTFOUND;

    v_totalSchedules := 0;
    v_totalActive := 0;
    v_totalCancelled := 0;
    v_totalBuses := 0;
    v_maxActiveBus := NULL;
    v_maxActiveTrips := 0;
    v_maxCancelledBus := NULL;
    v_maxCancelledTrips := 0;

    -- Header
    DBMS_OUTPUT.PUT_LINE(CHR(10) || LPAD('=', 120, '='));
    DBMS_OUTPUT.PUT_LINE('Bus Schedule Summary Report for Company: ' ||
                         companyRec.CompanyName || ' (' ||
companyRec.BusCompanyID || ')');
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
    DBMS_OUTPUT.PUT_LINE(RPAD('Bus ID', 8) || ' ' ||
                         RPAD('Num of Schedules', 18) || ' ' ||
                         RPAD('Active Schedules', 18) || ' ' ||
                         RPAD('Cancelled Schedules', 20) || ' ' ||
                         RPAD('Utilization %', 18) || ' ' ||
                         RPAD('Cancellation %', 15));
    DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));

    OPEN busCur;
    LOOP
      FETCH busCur INTO busRec;
      EXIT WHEN busCur%NOTFOUND;

      v_totalBuses     := v_totalBuses + 1;
      v_totalSchedules := v_totalSchedules + busRec.total_schedules;
      v_totalActive    := v_totalActive + busRec.active_schedules;
      v_totalCancelled := v_totalCancelled + busRec.cancelled_schedules;

      IF busRec.active_schedules > v_maxActiveTrips THEN
        v_maxActiveTrips := busRec.active_schedules;
        v_maxActiveBus   := busRec.BusID;
      END IF;

      IF busRec.cancelled_schedules > v_maxCancelledTrips THEN
        v_maxCancelledTrips := busRec.cancelled_schedules;
        v_maxCancelledBus   := busRec.BusID;
      END IF;
    END LOOP;
    CLOSE busCur;

    OPEN busCur;
    LOOP
      FETCH busCur INTO busRec;
      EXIT WHEN busCur%NOTFOUND;

      DECLARE
        v_utilPct   NUMBER;
        v_cancelPct NUMBER;
      BEGIN
        IF v_totalActive > 0 THEN
          v_utilPct := ROUND((busRec.active_schedules / v_totalActive) *
100, 2);
```

```
        ELSE
          v_utilPct := 0;
        END IF;

        IF v_totalCancelled > 0 THEN
          v_cancelPct := ROUND((busRec.cancelled_schedules /
v_totalCancelled) * 100, 2);
        ELSE
          v_cancelPct := 0;
        END IF;

        DBMS_OUTPUT.PUT_LINE(RPAD(busRec.BusID, 8) || ' ' ||
                             RPAD(busRec.total_schedules, 18) || ' ' ||
                             RPAD(busRec.active_schedules, 18) || ' ' ||
                             RPAD(busRec.cancelled_schedules, 20) || ' '
||
                             RPAD(TO_CHAR(v_utilPct,'FM9990.00') || '%',
18) || ' ' ||
                             RPAD(TO_CHAR(v_cancelPct,'FM9990.00') || '%',
15));

        IF v_utilPct > v_bestUtilPct THEN
          v_bestUtilPct := v_utilPct;
          v_bestUtilBus := busRec.BusID;
        END IF;

        IF v_cancelPct > v_bestCancelPct THEN
          v_bestCancelPct := v_cancelPct;
          v_bestCancelBus := busRec.BusID;
        END IF;
      END;
    END LOOP;
    CLOSE busCur;

    DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
    DBMS_OUTPUT.PUT_LINE('Total Schedules: ' || v_totalSchedules);
    DBMS_OUTPUT.PUT_LINE('Total Active Schedules: ' || v_totalActive);
    DBMS_OUTPUT.PUT_LINE('Total Cancelled Schedules: ' ||
v_totalCancelled);
    DBMS_OUTPUT.PUT_LINE('Total Buses: ' || v_totalBuses);
    DBMS_OUTPUT.PUT_LINE('Most Active Bus: ' || v_maxActiveBus || ' (' ||
v_maxActiveTrips || ' active schedules)');
    DBMS_OUTPUT.PUT_LINE('Most Cancelled Bus: ' || v_maxCancelledBus || '
(' || v_maxCancelledTrips || ' cancelled schedules)');
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));

    v_overallTotalSchedules := v_overallTotalSchedules + v_totalSchedules;
    v_overallTotalActive    := v_overallTotalActive + v_totalActive;
    v_overallTotalCancelled := v_overallTotalCancelled + v_totalCancelled;
    v_overallTotalBuses     := v_overallTotalBuses + v_totalBuses;
  END LOOP;
  CLOSE companyCur;

  -- Overall summary
  DBMS_OUTPUT.PUT_LINE(CHR(10) || LPAD('-', 120, '-'));
  DBMS_OUTPUT.PUT_LINE('OVERALL BUS SCHEDULE SUMMARY');
  DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
  DBMS_OUTPUT.PUT_LINE('Total Schedules Across Companies: ' ||
v_overallTotalSchedules);
  DBMS_OUTPUT.PUT_LINE('Total Active Schedules: ' ||
v_overallTotalActive);
  DBMS_OUTPUT.PUT_LINE('Total Cancelled Schedules: ' ||
v_overallTotalCancelled);
  DBMS_OUTPUT.PUT_LINE('Total Buses Across Companies: ' ||
v_overallTotalBuses);
  DBMS_OUTPUT.PUT_LINE('Bus with Highest Utilization %: ' || v_bestUtilBus
|| ' (' || v_bestUtilPct || '%)');
```

```
  DBMS_OUTPUT.PUT_LINE('Bus with Highest Cancellation %: ' ||
v_bestCancelBus || ' (' || v_bestCancelPct || '%)');
  DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
END;
/

EXEC prc_bus_schedule_summary;
```

Sample Output:

```
========================================================================================================================
Bus Schedule Summary Report for Company: Sabah InterCity Transport (BC009)
========================================================================================================================
Bus ID   Num of Schedules   Active Schedules   Cancelled Schedules  Utilization %      Cancellation %
------------------------------------------------------------------------------------------------------------------------
B0009    3                  1                  2                    6.25%              13.33%
B0019    3                  2                  1                    12.50%             6.67%
B0029    3                  2                  1                    12.50%             6.67%
B0039    3                  1                  2                    6.25%              13.33%
B0049    3                  2                  1                    12.50%             6.67%
B0059    3                  2                  1                    12.50%             6.67%
B0069    3                  0                  3                    0.00%              20.00%
B0079    3                  0                  3                    0.00%              20.00%
B0089    3                  3                  0                    18.75%             0.00%
B0099    4                  3                  1                    18.75%             6.67%
------------------------------------------------------------------------------------------------------------------------
Total Schedules: 31
Total Active Schedules: 16
Total Cancelled Schedules: 15
Total Buses: 10
Most Active Bus: B0089 (3 active schedules)
Most Cancelled Bus: B0069 (3 cancelled schedules)
========================================================================================================================

========================================================================================================================
Bus Schedule Summary Report for Company: GoGoBus Malaysia (BC010)
========================================================================================================================
Bus ID   Num of Schedules   Active Schedules   Cancelled Schedules  Utilization %      Cancellation %
------------------------------------------------------------------------------------------------------------------------
B0010    3                  3                  0                    15.00%             0.00%
B0020    3                  3                  0                    15.00%             0.00%
B0030    3                  2                  1                    10.00%             9.09%
B0040    3                  1                  2                    5.00%              18.18%
B0050    3                  0                  3                    0.00%              27.27%
B0060    3                  2                  1                    10.00%             9.09%
B0070    3                  1                  2                    5.00%              18.18%
B0080    3                  3                  0                    15.00%             0.00%
B0090    4                  3                  1                    15.00%             9.09%
B0100    3                  2                  1                    10.00%             9.09%
------------------------------------------------------------------------------------------------------------------------
Total Schedules: 31
Total Active Schedules: 20
Total Cancelled Schedules: 11
Total Buses: 10
Most Active Bus: B0010 (3 active schedules)
Most Cancelled Bus: B0050 (3 cancelled schedules)
========================================================================================================================

------------------------------------------------------------------------------------------------------------------------
OVERALL BUS SCHEDULE SUMMARY
------------------------------------------------------------------------------------------------------------------------
Total Schedules Across Companies: 305
Total Active Schedules: 154
Total Cancelled Schedules: 150
Total Buses Across Companies: 100
Bus with Highest Utilization %: B0023 (25%)
Bus with Highest Cancellation %: B0050 (27.27%)
------------------------------------------------------------------------------------------------------------------------
```

4.4 KENNY LOH KIT YI

4.4.1 Query 1: Ranking Driver With Most Schedules Allocated for the Past 3 Months

Purpose: The purpose of this query is to show the ranking of the most active drivers for the past 3 months by the number of schedules they have been assigned. This can help the company's tactical level to see which drivers are the most active, make sure work is distributed fairly and decide who should be scheduled for upcoming trips.

SQL Statement:

```
DROP INDEX idx_schedule_date;
CREATE INDEX idx_schedule_date ON Schedule(ScheduleDate);

SET PAGESIZE 50
SET LINESIZE 100

TTITLE CENTER 'Active Drivers Ranking (Past 3 Months)' SKIP 2

COLUMN "Driver ID" FORMAT A20
COLUMN "Driver Name" FORMAT A50
COLUMN "Total Schedules" FORMAT 999
COLUMN Ranking FORMAT 999

CREATE OR REPLACE VIEW DriversActiveRankingView AS
SELECT D.DriverID AS "Driver ID",
       D.FullName AS "Driver Name",
       COUNT(S.ScheduleID) AS "Total Schedules",
       RANK() OVER (ORDER BY COUNT(S.ScheduleID) DESC) AS Ranking
FROM Driver D
LEFT JOIN DriverAllocation DA ON D.DriverID = DA.DriverID
LEFT JOIN Schedule S ON DA.ScheduleID = S.ScheduleID
AND S.ScheduleDate >= ADD_MONTHS(SYSDATE, -3)
GROUP BY D.DriverID, D.FullName;

SELECT * FROM DriversActiveRankingView;

CLEAR COLUMNS
TTITLE OFF
```

Sample Output:

Active Drivers Ranking (Past 3 Months)

Driver ID	Driver Name	Total Schedules	RANKING
D0001	Matthew Brown	3	1
D0033	Rebecca Williamson	3	1
D0061	John Williams	3	1
D0010	Nicole Willis	3	1
D0012	Jennifer Caldwell	2	5
D0049	Jason Olson	2	5
D0015	Robert Villanueva	2	5
D0002	Elizabeth Sullivan	2	5
D0091	Jesse Neal	2	5
D0023	Aaron Vega	2	5
D0062	Erica Walters	2	5
D0067	Robert Wagner	2	5
D0078	Gloria Campos	2	5
D0099	Kathleen Martinez	2	5
D0035	Kevin Garza	2	5
D0090	Andrea Bauer	2	5
D0086	Marissa Sexton	2	5
D0057	Tiffany Williams	2	5
D0046	Jeffrey Blackburn	2	5
D0027	Emily White	2	5
D0004	Jeffery Wilson	1	21
D0024	Heather Perry	1	21
D0037	Joshua Brown	1	21
D0051	Erica Brooks	1	21
D0068	Eric Cooper	1	21
D0089	Kyle Jacobs	1	21
D0104	Lapkd	1	21
D0019	Shawn Jones	1	21
D0032	Crystal Solomon	1	21
D0045	Mary Nixon	1	21
D0056	Jasmine Duarte	1	21
D0071	Joe Aguilar	1	21
D0076	Casey Schwartz	1	21
D0097	Thomas Arnold	1	21
D0030	Jerry Rodgers	1	21
D0055	Tasha Harris	1	21
D0085	Kenneth Jackson	1	21

4.4.2 Query 2: Ranking Staff With Most Rent Collected And Dominant Shop Type

Purpose: The purpose of this query is to show the ranking of the staff members who have collected the most rent from shops and also the type of shops they collected from the most. This can help the company's tactical level to see which staff has the highest efficiency and also what type of shops are they good at handling so that they can assign the staff to collect rent in a more effective way.

SQL Statement:

```
DROP INDEX idx_rental_shopid;
CREATE INDEX idx_rental_shopid ON RentalCollection(UPPER(ShopID));

SET PAGESIZE 50
SET LINESIZE 120

TTITLE CENTER 'Staff Rent Collecting Ranking' SKIP 2

COLUMN StaffID FORMAT A9
COLUMN FullName FORMAT A20
COLUMN "Dominant Shop Type" FORMAT A50
COLUMN TotalCollections FORMAT 99
COLUMN CollectionRank FORMAT 9

CREATE OR REPLACE VIEW StaffRentCollectingView AS
WITH ShopTypeRanking AS (
```

```
    SELECT
        S.StaffID,
        S.FullName,
        SH.ShopType,
        COUNT(SH.ShopType) AS CollectionCount,
        RANK() OVER (
            PARTITION BY S.StaffID
            ORDER BY COUNT(*) DESC
        ) AS ShopTypeRank
    FROM Staff S
    LEFT JOIN RentalCollection RC ON S.StaffID = RC.StaffID
    LEFT JOIN Shop SH ON RC.ShopID = SH.ShopID
    GROUP BY S.StaffID, S.FullName, SH.ShopType
),
DominantShopTypes AS (
    SELECT
        StaffID,
        FullName,
        LISTAGG(ShopType, ', ') WITHIN GROUP (ORDER BY ShopType) AS
DominantShopTypes
    FROM ShopTypeRanking
    WHERE ShopTypeRank = 1
    GROUP BY StaffID, FullName
),
TotalCollections AS (
    SELECT
        S.StaffID,
        COUNT(RC.ShopID) AS TotalCollections,
        RANK() OVER (
            ORDER BY COUNT(*) DESC
        ) AS CollectionRank
    FROM Staff S
    LEFT JOIN RentalCollection RC ON S.StaffID = RC.StaffID
    GROUP BY S.StaffID
)
SELECT
    DST.StaffID,
    DST.FullName,
    DST.DominantShopTypes AS "Dominant Shop Type",
    TC.TotalCollections,
    TC.CollectionRank AS "Staff Collection Rank"
FROM DominantShopTypes DST
JOIN TotalCollections TC ON DST.StaffID = TC.StaffID
ORDER BY TC.CollectionRank;

SELECT * FROM StaffRentCollectingView;

CLEAR COLUMNS
TTITLE OFF
```

Sample Output:

Staff Rent Collecting Ranking

STAFFID	FULLNAME	Dominant Shop Type	TOTALCOLLECTIONS	Staff Collection Rank
STF001	Mary Chong	Electronics	35	1
STF002	Daniel Teoh	Electronics	35	1
STF009	Mary Chan	Electronics	35	1
STF006	Liam Lee	Pharmacy	34	4
STF003	John Teoh	Electronics	32	5
STF004	David Chong	Cafe, Clothing, Pharmacy	31	6
STF005	Liam Tan	Electronics	30	7
STF007	David Chan	Clothing	28	8
STF008	Emma Wong	Grocery	24	9
STF010	Liam Koh	Electronics	21	10
STF011	Heng Heng		0	11
STF012	Yun Looong		0	11
STF013	Loong Loong		0	11
STF014	Rap God		0	11
STF015	Dating Tutor		0	11

Use this format for every other subsections

4.4.3 Procedure 1: Adding Driver Allocation while Ensuring no Multiple Allocation on Same Date

Purpose: The purpose of this procedure is to provide the ease in allocating drivers to a schedule while ensuring that no driver will be allocated to more than one schedule that is on the same date.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_AddDriverAllocation (v_driverID IN
Driver.DriverID%TYPE, v_scheduleID IN Schedule.ScheduleID%TYPE) IS
v_exists NUMBER;
MULTIPLE_ALLOCATION EXCEPTION;
PRAGMA EXCEPTION_INIT(MULTIPLE_ALLOCATION, -20000);

BEGIN
    SELECT COUNT(*)
    INTO v_exists
    FROM DriverAllocation DA
    JOIN Schedule S1 ON DA.ScheduleID = S1.ScheduleID
    JOIN Schedule S2 ON S2.ScheduleID = v_scheduleID
    WHERE DA.DriverID = v_driverID
    AND S1.ScheduleDate = S2.ScheduleDate;

    IF v_exists > 0 THEN
        RAISE MULTIPLE_ALLOCATION;
    ELSE
        INSERT INTO DriverAllocation (DriverID, ScheduleID) VALUES
(v_driverID, v_scheduleID);
        DBMS_OUTPUT.PUT_LINE('Driver allocation added successfully.');
    END IF;

    EXCEPTION
    WHEN MULTIPLE_ALLOCATION THEN
        DBMS_OUTPUT.PUT_LINE('Driver ' || v_driverID || ' is already
allocated on the same date as ' || v_scheduleID || '.');

END;
/
```

Sample Output:

```
SQL> exec prc_AddDriverAllocation('D0001', 'SCH0002')
Driver allocation added successfully.

PL/SQL procedure successfully completed.

SQL> exec prc_AddDriverAllocation('D0001', 'SCH0102')
Driver D0001 is already allocated on the same date as SCH0102.

PL/SQL procedure successfully completed.

SQL> select * from schedule where scheduledate = '06-07-2025'
  2  ;

SCHEDULEID          BUSID               DEPARTURET ARRIVALTIM STATION                                                                                       PLATFORM
     DESTINATION                                                                                                  SCHEDULEDA SCHEDULESTATUS
------------------- ------------------- ---------- ---------- ------------------------------------------------------------------------------------------- -----------------------
---------------- ---------------------------------------------------------------------------------------------------- ---------- ----------------------------------------
SCH0002             B0002               06-07-2025 06-07-2025 Alor Setar                                                                                    Platform 4
     Rawang                                                                                                       06-07-2025 Past
SCH0102             B0002               06-07-2025 07-07-2025 Seremban                                                                                      Platform 9
     Melaka Sentral                                                                                               06-07-2025 Refunded
```

4.4.4 Procedure 2: Adding Collected Rent with Validation and Auto Assign ID and Date

Purpose: The purpose of this procedure is to provide the ease in recording the rent collected from a shop and automatically assigns an id and collection date to the record while validating the staff, shop and amount collected.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_AddRentCollection (
    v_shopID           IN RentalCollection.ShopID%TYPE,
    v_staffID          IN RentalCollection.StaffID%TYPE,
    v_amountCollected  IN RentalCollection.AmountCollected%TYPE,
    v_paymentMethod    IN RentalCollection.PaymentMethod%TYPE,
    v_notes            IN RentalCollection.Notes%TYPE DEFAULT '-'
) IS
v_shopExists   NUMBER;
v_staffExists  NUMBER;
v_maxNum       NUMBER;
v_newID        RentalCollection.CollectionID%TYPE;

BEGIN
    SELECT COUNT(*) INTO v_shopExists
    FROM Shop
    WHERE ShopID = v_shopID;

    IF v_shopExists = 0 THEN
        RAISE_APPLICATION_ERROR(-20000, 'Invalid Shop ID.');
    END IF;

    SELECT COUNT(*) INTO v_staffExists
    FROM Staff
    WHERE StaffID = v_staffID;

    IF v_staffExists = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid Staff ID.');
    END IF;

    IF v_amountCollected <= 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Amount collected must be greater
than 0.');
    END IF;

    SELECT NVL(MAX(TO_NUMBER(SUBSTR(CollectionID, 4))), 0)
    INTO v_maxNum
    FROM RentalCollection;

    v_newID := 'COL' || LPAD(v_maxNum + 1, 3, '0');


    INSERT INTO RentalCollection (CollectionID, ShopID, StaffID,
CollectionDate, AmountCollected, PaymentMethod, Notes) VALUES (v_newID,
v_shopID, v_staffID, SYSDATE, v_amountCollected, v_paymentMethod,
v_notes);

    DBMS_OUTPUT.PUT_LINE('Rent collection added successfully in ' ||
v_newID || '.');

END;
/
```

Sample Output:

```
SQL> exec prc_AddRentCollection('SHP002', 'STF002', 2500, 'Cash')
Rent collection added successfully in COL312.

PL/SQL procedure successfully completed.

SQL> select * from rentalcollection where collectionid = 'COL312';

COLLECTIONID     SHOPID          STAFFID         COLLECTIONDATE                 AMOUNTCOLLECTED PAYMENTMETHOD            NOTES
---------------- --------------- --------------- ------------------------------ --------------- ------------------------ ------------------------
COL312           SHP002          STF002          24-08-2025                                2500 Cash                     -

SQL> exec prc_AddRentCollection('SHP002', 'STF006', 2500, 'Cash', 'Paid on time')
Rent collection added successfully in COL313.

PL/SQL procedure successfully completed.

SQL> select * from rentalcollection where collectionid = 'COL313';

COLLECTIONID     SHOPID          STAFFID         COLLECTIONDATE                 AMOUNTCOLLECTED PAYMENTMETHOD            NOTES
---------------- --------------- --------------- ------------------------------ --------------- ------------------------ ------------------------
COL313           SHP002          STF006          24-08-2025                                2500 Cash                     Paid on time
```

4.4.5 Trigger 1: Audit Log for Driver Allocation

Purpose: The purpose of this trigger is to track update, delete and insert on the driver allocation table.

SQL Statement:

```
DROP SEQUENCE driverAllocation_audit_seq;

CREATE SEQUENCE driverAllocation_audit_seq
MINVALUE 1
MAXVALUE 99999
START WITH 1
INCREMENT BY 1
NOCACHE;

DROP TABLE ActionDriverAllocation;

CREATE TABLE ActionDriverAllocation(
AuditID                 NUMBER       NOT NULL,
DriverID            VARCHAR2(10) NOT NULL,
ScheduleID          VARCHAR2(10) NOT NULL,
NewScheduleID              VARCHAR2(10),
Shift               VARCHAR2(10) NOT NULL,
NewShift            VARCHAR2(10),
UserID                     VARCHAR2(30),
TransDate           DATE,
TransTime           CHAR(8),
TransAction         VARCHAR2(6)
);

CREATE OR REPLACE TRIGGER trg_track_DriverAllocation
AFTER INSERT OR UPDATE OR DELETE ON DriverAllocation
FOR EACH ROW
BEGIN
  CASE
    WHEN INSERTING THEN
      INSERT INTO ActionDriverAllocation (
        AuditID, DriverID, ScheduleID, NewScheduleID,
        Shift, NewShift,
        UserID, TransDate, TransTime, TransAction
      ) VALUES (
        driverAllocation_audit_seq.NEXTVAL,
        :NEW.DriverID, :NEW.ScheduleID, NULL,
        :NEW.Shift, NULL,
        USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'INSERT'
      );

    WHEN UPDATING THEN
      INSERT INTO ActionDriverAllocation (
        AuditID, DriverID, ScheduleID, NewScheduleID,
        Shift, NewShift,
```

```
        UserID, TransDate, TransTime, TransAction
      ) VALUES (
        driverAllocation_audit_seq.NEXTVAL,
        :OLD.DriverID, :OLD.ScheduleID, :NEW.ScheduleID,
        :OLD.Shift, :NEW.Shift,
        USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'UPDATE'
      );

    WHEN DELETING THEN
      INSERT INTO ActionDriverAllocation (
        AuditID, DriverID, ScheduleID, NewScheduleID,
        Shift, NewShift,
        UserID, TransDate, TransTime, TransAction
      ) VALUES (
        driverAllocation_audit_seq.NEXTVAL,
        :OLD.DriverID, :OLD.ScheduleID, NULL,
        :OLD.Shift, NULL,
        USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'DELETE'
      );
  END CASE;
END;
/
```

Sample Output:

```
SQL> set scheduleid = 'SCH0015'
SP2-0158: unknown SET option "scheduleid"
SQL> update driverallocation
  2  set scheduleid = 'SCH0020'
  3  where driverid = 'D0007'
  4  and scheduleid = 'SCH0010';

1 row updated.

SQL> select * from actiondriverallocation;

   AUDITID DRIVERID            SCHEDULEID          NEWSCHEDULEID       SHIFT               NEWSHIFT            USERID     TRANSDATE   TRANSTIME       TRANSACTION
---------- -------------------- -------------------- -------------------- -------------------- -------------------- ---------- ----------- --------------- ------------
         1 D0007                SCH0107              SCH0010             Evening             Night               KENNY      07-SEP-25   14:55:27        UPDATE
         2 D0007                SCH0010              SCH0020             Night               Evening             KENNY      07-SEP-25   14:58:15        UPDATE
```

4.4.6 Trigger 2: Update Driver Allocation Shift Based on Schedule

Purpose: The purpose of this trigger is to update the shift of the driver based on the departure time set in the schedule table.

SQL Statement:

```
CREATE OR REPLACE TRIGGER trg_UpdateDriverShift
BEFORE INSERT OR UPDATE ON DriverAllocation
FOR EACH ROW
DECLARE
    v_departureTime Schedule.DepartureTime%TYPE;
    v_time          NUMBER;
BEGIN
    SELECT DepartureTime INTO v_departureTime
    FROM Schedule
    WHERE ScheduleID = :NEW.ScheduleID;

    v_time := TO_NUMBER(TO_CHAR(v_departureTime, 'HH24MI'));

    IF v_time BETWEEN 500 AND 1159 THEN
        :NEW.Shift := 'Morning';
    ELSIF v_time BETWEEN 1200 AND 1659 THEN
        :NEW.Shift := 'Afternoon';
    ELSIF v_time BETWEEN 1700 AND 2059 THEN
        :NEW.Shift := 'Evening';
    ELSE
```

```
        :NEW.Shift := 'Night';
    END IF;

END;
/
```

Sample Output:

```
SQL> INSERT INTO DriverAllocation (DriverID, ScheduleID, Shift, Email, ContactNo) VALUES ('D0007', 'SCH0267', 'Night', 'driver.d0007@transport.com', '016912
1567');

1 row created.

SQL> select d.driverid, s.scheduleid, to_char(s.departuretime, 'DD-MON-YYYY HH24:MI:SS') AS DepartureTime, da.shift
  2   from driver d
  3  join driverallocation da on d.driverid = da.driverid
  4  join schedule s on da.scheduleid = s.scheduleid
  5  where d.driverid = 'D0007'
  6   and s.scheduleid = 'SCH0267';

DRIVERID            SCHEDULEID          DEPARTURETIME                                                   SHIFT
------------------- ------------------- --------------------------------------------------------------- --------------------
D0007               SCH0267             17-AUG-2025 06:00:00                                            Morning
```

4.4.7 Report 1: Driver Schedule Report

Purpose: The purpose of this report is to show the detailed schedule information of each driver such as schedule date, departure time, arrival time, destination and shift.

SQL Statement:

```
SET PAGESIZE 1000
SET LINESIZE 1200
SET SERVEROUTPUT ON

CREATE OR REPLACE PROCEDURE prc_DriverScheduleReport IS

CURSOR driverCursor IS
SELECT Distinct D.DriverID, D.FullName, D.Email, D.ContactNo,
BC.CompanyName
FROM Driver D
JOIN Bus_Company BC
ON D.BusCompanyID = BC.BusCompanyID
JOIN DriverAllocation DA
ON D.DriverID = DA.DriverID
JOIN Schedule S
ON S.ScheduleID = DA.ScheduleID
ORDER BY D.DriverID;

CURSOR scheduleCursor(v_driverID Driver.DriverID%TYPE) IS
SELECT DA.ScheduleID, S.ScheduleDate, TO_CHAR(S.DepartureTime,
'DD-MON-YYYY HH24:MI:SS') AS DepartureTime, TO_CHAR(S.ArrivalTime,
'DD-MON-YYYY HH24:MI:SS') AS ArrivalTime, S.Destination, DA.Shift
FROM Schedule S
JOIN DriverAllocation DA
ON S.ScheduleID = DA.ScheduleID
WHERE DA.DriverID = v_driverID;

driverList driverCursor%ROWTYPE;
scheduleList scheduleCursor%ROWTYPE;

BEGIN
OPEN driverCursor;
DBMS_OUTPUT.PUT_LINE(LPAD('=', 135, '='));
DBMS_OUTPUT.PUT_LINE(LPAD('-',55, '-') || RPAD('DRIVER SCHEDULE REPORT',
80, '-'));

DBMS_OUTPUT.PUT_LINE(LPAD('=', 135, '='));
```

```
LOOP
       FETCH driverCursor into driverList;
       EXIT WHEN driverCursor%NOTFOUND;

--Header

DBMS_OUTPUT.PUT_LINE('Driver ID: ' || driverList.DriverID);
DBMS_OUTPUT.PUT_LINE('Driver Name: ' || driverList.FullName);
DBMS_OUTPUT.PUT_LINE('Email: ' || driverList.Email);
DBMS_OUTPUT.PUT_LINE('Contact No: ' || driverList.ContactNo);
DBMS_OUTPUT.PUT_LINE('Company Name: ' || driverList.CompanyName);

--Heading
DBMS_OUTPUT.PUT_LINE(LPAD('-', 135, '-'));
DBMS_OUTPUT.PUT_LINE (RPAD('Schedule ID',15,' ') || ' ' ||
                      RPAD('Schedule Date', 25, ' ') || ' ' ||
                      RPAD('Departure Time', 25, ' ') || ' ' ||
                      RPAD('Arrival Time', 25, ' ') || ' ' ||
                    RPAD('Destination',25,' ') || ' ' ||
                      RPAD('Shift',15, ' '));
DBMS_OUTPUT.PUT_LINE(LPAD('-', 135, '-'));

--Body
OPEN scheduleCursor(driverList.DriverID);
LOOP
       FETCH scheduleCursor INTO scheduleList;
       EXIT WHEN scheduleCursor%NOTFOUND;

       DBMS_OUTPUT.PUT_LINE(RPAD(scheduleList.ScheduleID, 15,' ') || ' ' ||
                         RPAD(scheduleList.ScheduleDate, 25, ' ') || ' ' ||
                         RPAD(scheduleList.DepartureTime, 25,' ') || ' ' ||
                         RPAD(scheduleList.ArrivalTime, 25,' ') || ' '||
                       RPAD(scheduleList.Destination, 25,' ') || ' ' ||
                         RPAD(scheduleList.Shift, 15,' '));
END LOOP;

DBMS_OUTPUT.PUT_LINE(LPAD('-', 135, '-'));
DBMS_OUTPUT.PUT_LINE('Total Number of Schedules: ' ||
scheduleCursor%ROWCOUNT);
DBMS_OUTPUT.PUT_LINE(LPAD('=', 135, '='));
DBMS_OUTPUT.PUT_LINE(CHR(10));

CLOSE scheduleCursor;

END LOOP;

--Footer
DBMS_OUTPUT.PUT_LINE('Total number of Drivers: ' ||
driverCursor%ROWCOUNT);
CLOSE driverCursor;
DBMS_OUTPUT.PUT_LINE(CHR(10));
DBMS_OUTPUT.PUT_LINE(LPAD('=',65, '=') || RPAD('END OF REPORT', 70, '='));

END;
/

EXEC prc_DriverScheduleReport
```

Sample Output:

```
================================================================================================================================================
----------------------------------------------------DRIVER SCHEDULE REPORT----------------------------------------------------------------------
================================================================================================================================================
Driver ID: D0001
Driver Name: Matthew Brown
Email: gregory42@example.org
Contact No: 582576239397
Company Name: GoGoBus Malaysia
------------------------------------------------------------------------------------------------------------------------------------------------
Schedule ID     Schedule Date            Departure Time           Arrival Time             Destination              Shift
------------------------------------------------------------------------------------------------------------------------------------------------
SCH0053         29-JUL-23                29-JUL-2023 07:30:00     29-JUL-2023 11:51:00     Sungai Petani            Morning
SCH0061         15-MAR-25                15-MAR-2025 13:00:00     15-MAR-2025 16:07:00     Melaka Sentral           Afternoon
SCH0016         27-FEB-23                27-FEB-2023 11:45:00     27-FEB-2023 16:35:00     Pasir Gudang             Morning
SCH0023         11-JAN-23                11-JAN-2023 13:45:00     11-JAN-2023 14:28:00     Putrajaya                Afternoon
SCH0028         24-JAN-24                24-JAN-2024 13:15:00     24-JAN-2024 16:46:00     Sungai Petani            Afternoon
SCH0029         03-MAR-24                03-MAR-2024 07:15:00     03-MAR-2024 10:30:00     Batu Gajah               Morning
SCH0030         25-SEP-24                25-SEP-2024 07:30:00     25-SEP-2024 08:59:00     Taiping                  Morning
SCH0031         15-MAR-24                15-MAR-2024 09:45:00     15-MAR-2024 12:06:00     Batu Gajah               Morning
SCH0032         24-JUL-25                24-JUL-2025 09:00:00     24-JUL-2025 11:41:00     Johor Bahru Sentral      Morning
SCH0038         11-APR-23                11-APR-2023 07:30:00     11-APR-2023 10:16:00     Johor Bahru Sentral      Morning
SCH0087         27-FEB-24                27-FEB-2024 10:45:00     27-FEB-2024 12:37:00     Putrajaya                Morning
SCH0092         08-OCT-23                08-OCT-2023 11:45:00     08-OCT-2023 15:57:00     Putrajaya                Morning
SCH0100         05-JUN-25                05-JUN-2025 14:15:00     05-JUN-2025 16:12:00     Kajang                   Afternoon
SCH0102         09-AUG-24                09-AUG-2024 20:00:00     09-AUG-2024 20:42:00     Melaka Sentral           Evening
SCH0104         20-NOV-23                20-NOV-2023 11:15:00     20-NOV-2023 14:06:00     Batu Gajah               Morning
SCH0112         16-AUG-23                16-AUG-2023 21:00:00     16-AUG-2023 23:57:00     Melaka Sentral           Night
SCH0114         01-JUL-23                01-JUL-2023 14:30:00     01-JUL-2023 17:05:00     Taiping                  Afternoon
SCH0129         04-DEC-24                04-DEC-2024 07:45:00     04-DEC-2024 10:04:00     Putrajaya                Morning
SCH0152         11-JAN-23                11-JAN-2023 11:45:00     11-JAN-2023 16:34:00     Kajang                   Morning
SCH0175         22-JUN-23                22-JUN-2023 19:30:00     22-JUN-2023 22:44:00     Johor Bahru Sentral      Evening
SCH0185         25-JUL-25                25-JUL-2025 06:45:00     25-JUL-2025 11:11:00     Shah Alam                Morning
SCH0191         15-JUN-23                15-JUN-2023 05:15:00     15-JUN-2023 09:39:00     Taiping                  Morning
SCH0195         30-AUG-25                30-AUG-2025 05:15:00     30-AUG-2025 09:43:00     Pasir Gudang             Morning
SCH0206         21-OCT-23                21-OCT-2023 19:15:00     21-OCT-2023 20:31:00     Muar                     Evening
SCH0218         19-DEC-23                19-DEC-2023 06:00:00     19-DEC-2023 06:44:00     Pasir Gudang             Morning
SCH0221         03-JAN-23                03-JAN-2023 21:00:00     03-JAN-2023 23:36:00     Johor Bahru Sentral      Night
SCH0243         10-JAN-25                10-JAN-2025 05:45:00     10-JAN-2025 09:50:00     Kuala Lumpur Central     Morning
SCH0250         19-JUL-25                19-JUL-2025 15:15:00     19-JUL-2025 16:14:00     Seremban                 Afternoon
SCH0262         26-APR-24                26-APR-2024 14:15:00     26-APR-2024 16:19:00     Butterworth              Afternoon
SCH0273         05-DEC-23                05-DEC-2023 18:15:00     05-DEC-2023 19:35:00     Shah Alam                Evening
SCH0276         19-AUG-24                19-AUG-2024 05:30:00     19-AUG-2024 10:10:00     Butterworth              Morning
SCH0291         08-NOV-24                08-NOV-2024 10:15:00     08-NOV-2024 15:00:00     Batu Gajah               Morning
------------------------------------------------------------------------------------------------------------------------------------------------
Total Number of Schedules: 32
================================================================================================================================================
```

4.4.8 Report 2: Staff Rent Collection Report

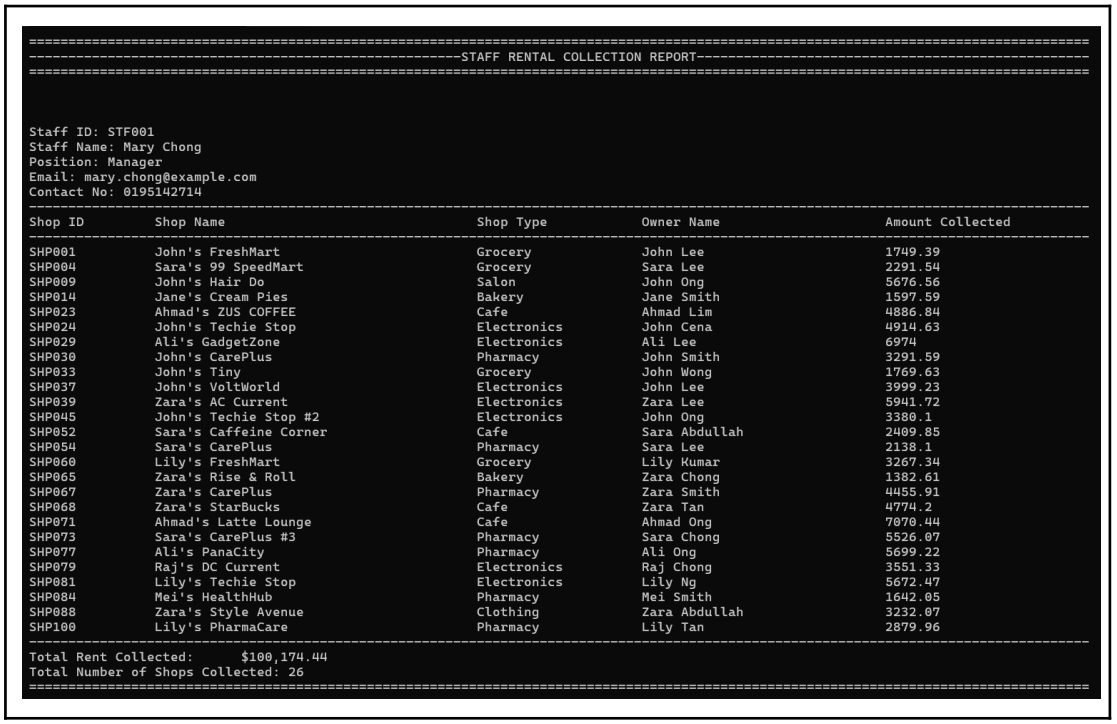
Purpose: The purpose of this report is to show the detailed rent collection information of each staff member and also the total number of shops and total rent amount collected.

SQL Statement:

```
SET PAGESIZE 1000
SET LINESIZE 1200
SET SERVEROUTPUT ON

CREATE OR REPLACE PROCEDURE prc_StaffRentCollectionReport IS
v_grandTotal    RentalCollection.AmountCollected%TYPE;
v_totalValue    RentalCollection.AmountCollected%TYPE;

CURSOR staffCursor IS
SELECT Distinct StaffID, FullName, Position, Email, PhoneNo
FROM Staff
ORDER BY StaffID;

CURSOR shopCursor(v_staffID Staff.StaffID%TYPE) IS
SELECT SH.ShopID, SH.ShopName, SH.ShopType, SH.OwnerName,
SUM(RC.AmountCollected) AS Total_Rent_Collected
FROM Shop SH
JOIN RentalCollection RC
ON SH.ShopID = RC.ShopID
WHERE RC.StaffID = v_staffID
GROUP BY SH.ShopID, SH.ShopName, SH.ShopType, SH.OwnerName
ORDER BY SH.ShopID;

staffList staffCursor%ROWTYPE;
shopList shopCursor%ROWTYPE;

BEGIN
OPEN staffCursor;
v_totalValue := 0;
DBMS_OUTPUT.PUT_LINE(LPAD('=', 135, '='));
```

```
DBMS_OUTPUT.PUT_LINE(LPAD('-',55, '-') || RPAD('STAFF RENTAL COLLECTION
REPORT', 80, '-'));
DBMS_OUTPUT.PUT_LINE(LPAD('=', 135, '='));
LOOP
       FETCH staffCursor into staffList;
       EXIT WHEN staffCursor%NOTFOUND;

--Header
DBMS_OUTPUT.PUT_LINE(CHR(10));
DBMS_OUTPUT.PUT_LINE('Staff ID: ' || staffList.StaffID);
DBMS_OUTPUT.PUT_LINE('Staff Name: ' || staffList.FullName);
DBMS_OUTPUT.PUT_LINE('Position: ' || staffList.Position);
DBMS_OUTPUT.PUT_LINE('Email: ' || staffList.Email);
DBMS_OUTPUT.PUT_LINE('Contact No: ' || staffList.PhoneNo);

--Heading
DBMS_OUTPUT.PUT_LINE(LPAD('-', 135, '-'));
DBMS_OUTPUT.PUT_LINE (RPAD('Shop ID',15,' ') || ' ' ||
                      RPAD('Shop Name', 40, ' ') || ' ' ||
                      RPAD('Shop Type', 20, ' ') || ' ' ||
                   RPAD('Owner Name',30,' ') || ' ' ||
                      RPAD('Amount Collected', 30, ' '));
DBMS_OUTPUT.PUT_LINE(LPAD('-', 135, '-'));

--Body
OPEN shopCursor(staffList.StaffID);
v_grandTotal := 0;
LOOP
       FETCH shopCursor INTO shopList;
       EXIT WHEN shopCursor%NOTFOUND;

       DBMS_OUTPUT.PUT_LINE(RPAD(shopList.ShopID, 15,' ') || ' ' ||
                        RPAD(shopList.ShopName, 40, ' ') || ' ' ||
                        RPAD(shopList.ShopType, 20,' ') || ' ' ||
                        RPAD(shopList.OwnerName, 30,' ') || ' '||
                     RPAD(shopList.Total_Rent_Collected, 30,' '));
       v_grandTotal := v_grandTotal + shopList.Total_Rent_Collected;
END LOOP;

       v_totalValue := v_totalValue + v_grandTotal;

DBMS_OUTPUT.PUT_LINE(LPAD('-', 135, '-'));
DBMS_OUTPUT.PUT_LINE('Total Rent Collected: ' || TO_CHAR(v_grandTotal,
'$999,999,999.99'));
DBMS_OUTPUT.PUT_LINE('Total Number of Shops Collected: ' ||
shopCursor%ROWCOUNT);
DBMS_OUTPUT.PUT_LINE(LPAD('=', 135, '='));

CLOSE shopCursor;

END LOOP;

--Footer
DBMS_OUTPUT.PUT_LINE(CHR(10));
DBMS_OUTPUT.PUT_LINE('Total amount of rent collected: ' ||
TO_CHAR(v_totalValue, '$99,999,999,999.99'));
DBMS_OUTPUT.PUT_LINE('Total number of staff: ' || staffCursor%ROWCOUNT);
CLOSE staffCursor;
DBMS_OUTPUT.PUT_LINE(LPAD('=',65, '=') || RPAD('END OF REPORT', 70, '='));

END;
/

EXEC prc_StaffRentCollectionReport
```

Sample Output:



```
================================================================================================================================================
----------------------------------------------------------STAFF RENTAL COLLECTION REPORT----------------------------------------------------------
================================================================================================================================================



Staff ID: STF001
Staff Name: Mary Chong
Position: Manager
Email: mary.chong@example.com
Contact No: 0195142714
------------------------------------------------------------------------------------------------------------------------------------------------
Shop ID         Shop Name                                Shop Type            Owner Name                  Amount Collected
------------------------------------------------------------------------------------------------------------------------------------------------
SHP001          John's FreshMart                         Grocery              John Lee                    1749.39
SHP004          Sara's 99 SpeedMart                      Grocery              Sara Lee                    2291.54
SHP009          John's Hair Do                           Salon                John Ong                    5676.56
SHP014          Jane's Cream Pies                        Bakery               Jane Smith                  1597.59
SHP023          Ahmad's ZUS COFFEE                       Cafe                 Ahmad Lim                   4886.84
SHP024          John's Techie Stop                       Electronics          John Cena                   4914.63
SHP029          Ali's GadgetZone                         Electronics          Ali Lee                     6974
SHP030          John's CarePlus                          Pharmacy             John Smith                  3291.59
SHP033          John's Tiny                              Grocery              John Wong                   1769.63
SHP037          John's VoltWorld                         Electronics          John Lee                    3999.23
SHP039          Zara's AC Current                        Electronics          Zara Lee                    5941.72
SHP045          John's Techie Stop #2                    Electronics          John Ong                    3380.1
SHP052          Sara's Caffeine Corner                   Cafe                 Sara Abdullah               2409.85
SHP054          Sara's CarePlus                          Pharmacy             Sara Lee                    2138.1
SHP060          Lily's FreshMart                         Grocery              Lily Kumar                  3267.34
SHP065          Zara's Rise & Roll                       Bakery               Zara Chong                  1382.61
SHP067          Zara's CarePlus                          Pharmacy             Zara Smith                  4455.91
SHP068          Zara's StarBucks                         Cafe                 Zara Tan                    4774.2
SHP071          Ahmad's Latte Lounge                     Cafe                 Ahmad Ong                   7070.44
SHP073          Sara's CarePlus #3                       Pharmacy             Sara Chong                  5526.07
SHP077          Ali's PanaCity                           Pharmacy             Ali Ong                     5699.22
SHP079          Raj's DC Current                         Electronics          Raj Chong                   3551.33
SHP081          Lily's Techie Stop                       Electronics          Lily Ng                     5672.47
SHP084          Mei's HealthHub                          Pharmacy             Mei Smith                   1642.05
SHP088          Zara's Style Avenue                      Clothing             Zara Abdullah               3232.07
SHP100          Lily's PharmaCare                        Pharmacy             Lily Tan                    2879.96
------------------------------------------------------------------------------------------------------------------------------------------------
Total Rent Collected:      $100,174.44
Total Number of Shops Collected: 26
================================================================================================================================================
```

4.5 KHIEW JIT CHEN

4.5.1 Query 1: Route Occupancy Rate

Management Level: Operational

Purpose: This query is designed to provide operational managers with a performance report that identifies the top 10 most popular bus routes based on their occupancy rate. By aggregating data across all schedules, it calculates the total number of tickets sold against the total seating capacity for each unique route (i.e., each origin-destination pair). The final report ranks these routes from highest to lowest occupancy, allowing management to quickly see which routes are most efficiently utilizing their available resources. This insight is crucial for making informed, day-to-day decisions regarding resource allocation, such as assigning more buses to high-demand routes, adjusting schedules to better meet passenger traffic, or identifying underperforming routes that may require further investigation or marketing support.

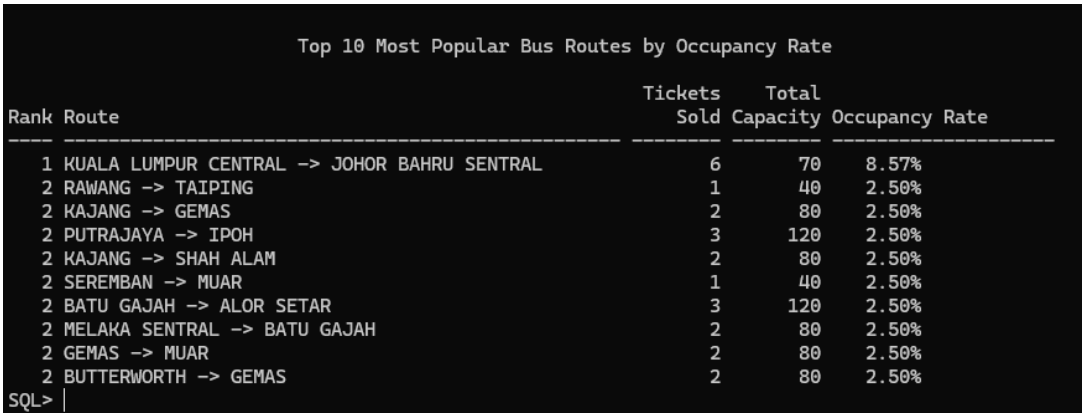
SQL Statement:

```
SET LINESIZE 100
SET PAGESIZE 35
SET FEEDBACK OFF
CL SCR

-- Set a title for the report
TTITLE center 'Top 10 Most Popular Bus Routes by Occupancy Rate' SKIP 2

-- Define the format for each column
COLUMN "Rank"           FORMAT 999 HEADING 'Rank'
COLUMN "Route"          FORMAT A50 HEADING 'Route'
COLUMN "TicketsSold"    FORMAT 999,999 HEADING 'Tickets|Sold'
COLUMN "TotalCapacity"  FORMAT 999,999 HEADING 'Total|Capacity'
COLUMN "Occupancy Rate" FORMAT A20 HEADING 'Occupancy Rate'

--
================================================================================
===
-- Performance Optimization: Create Secondary Indexes
-- As per the assignment, primary key indexes are created automatically by
Oracle.
-- The following function-based indexes are created to speed up
case-insensitive
-- grouping on character fields like Station and Destination.
--
================================================================================
===

CREATE INDEX idx_schedule_station_upper ON Schedule(UPPER(Station));
CREATE INDEX idx_schedule_destination_upper ON
Schedule(UPPER(Destination));

--
================================================================================
===
-- Create or Replace the View
-- This view is updated to use the UPPER() function in its GROUP BY
clause,
-- allowing the query to leverage the new function-based indexes. It now
-- includes total tickets sold and total capacity for each route.
--
================================================================================
===
```

```
CREATE OR REPLACE VIEW TOP_10_ROUTES_VIEW AS
SELECT Route, "Occupancy_Rate", "Rank", "TotalCapacity", "TicketsSold"
FROM (
    SELECT
      r.Station || ' -> ' || r.Destination AS Route,
      TO_CHAR(r.Occupancy * 100, '990.00') || '%' AS "Occupancy_Rate",
      r.TotalCapacity AS "TotalCapacity",
      r.TotalTicketsSold AS "TicketsSold",
      RANK() OVER (ORDER BY r.Occupancy DESC) AS "Rank",
      ROW_NUMBER() OVER (ORDER BY r.Occupancy DESC) as rn
    FROM (
        SELECT
          UPPER(p.Station) AS Station,
          UPPER(p.Destination) AS Destination,
          SUM(p.TicketsSold) AS TotalTicketsSold,
          SUM(p.Capacity) AS TotalCapacity,
          -- Calculate occupancy rate based on total tickets and capacity
for the route
          CASE WHEN SUM(p.Capacity) > 0 THEN SUM(p.TicketsSold) * 1.0 /
SUM(p.Capacity) ELSE 0 END AS Occupancy
        FROM (
            SELECT
              s.Station,
              s.Destination,
              b.Capacity,
              COUNT(t.TicketID) AS TicketsSold
            FROM
              Schedule s
            JOIN
              Bus b ON s.BusID = b.BusID
            LEFT JOIN
              Ticket t ON s.ScheduleID = t.ScheduleID
            GROUP BY
              s.ScheduleID, s.Station, s.Destination, b.Capacity
        ) p
        GROUP BY
          UPPER(p.Station), UPPER(p.Destination) -- Uses the new
function-based indexes
    ) r
)
WHERE
  rn <= 10;

--
================================================================================
===
-- Display the Formatted Report from the View
--
================================================================================
===

SELECT
  "Rank",
  Route,
  "TicketsSold",
  "TotalCapacity",
  "Occupancy_Rate" AS "Occupancy Rate"
FROM TOP_10_ROUTES_VIEW
ORDER BY "Rank";

--
================================================================================
===
-- Cleanup Commands
-- Drop the secondary indexes created for this report.
--
================================================================================
```

```
===

DROP INDEX idx_schedule_station_upper;
DROP INDEX idx_schedule_destination_upper;

TTITLE OFF
CLEAR COLUMNS
```

Sample Output:



```
                    Top 10 Most Popular Bus Routes by Occupancy Rate

                                                    Tickets    Total
Rank Route                                             Sold Capacity Occupancy Rate
---- -------------------------------------------------- -------- -------- --------------------
   1 KUALA LUMPUR CENTRAL -> JOHOR BAHRU SENTRAL            6       70    8.57%
   2 RAWANG -> TAIPING                                      1       40    2.50%
   2 KAJANG -> GEMAS                                        2       80    2.50%
   2 PUTRAJAYA -> IPOH                                      3      120    2.50%
   2 KAJANG -> SHAH ALAM                                    2       80    2.50%
   2 SEREMBAN -> MUAR                                       1       40    2.50%
   2 BATU GAJAH -> ALOR SETAR                               3      120    2.50%
   2 MELAKA SENTRAL -> BATU GAJAH                           2       80    2.50%
   2 GEMAS -> MUAR                                          2       80    2.50%
   2 BUTTERWORTH -> GEMAS                                   2       80    2.50%
SQL>
```

4.5.2 Query 2: Monthly Part Orders and Cost Trend Analysis

Management Level: Tactical

Purpose: This query tracks the number of part orders and the total cost of these orders on a monthly basis. This is essential for the procurement and finance teams to monitor spending, forecast future budget needs, and identify any unusual spikes in purchasing activity. A rising trend in costs could trigger a review of supplier pricing or an investigation into increased part failure rates.

SQL Statement:

```
SET LINESIZE 120
SET PAGESIZE 100
CL SCR

-- Set a dynamic title for the report including the current date
TTITLE LEFT 'Monthly Order Summary Report as of ' _DATE  SKIP 2

-- Define the format for each column from your query
COLUMN OrderMonth      FORMAT A20            HEADING 'Order Month'
COLUMN NumberOfOrders  FORMAT 999,999        HEADING 'Number of Orders'
COLUMN TotalMonthlyCost FORMAT $99,999,999.99 HEADING 'Total Monthly Cost'

-- Set up a grand total calculation at the end of the report
BREAK ON REPORT
COMPUTE SUM LABEL 'Grand Total: ' OF NumberOfOrders TotalMonthlyCost ON
REPORT


CREATE INDEX idx_part_order_date ON Part_Order(OrderDate);


CREATE OR REPLACE VIEW MonthlyOrderSummaryView AS
SELECT
    TO_CHAR(po.OrderDate, 'YYYY-MM') AS OrderMonth,
    COUNT(po.OrderID) AS NumberOfOrders,
    SUM(p.UnitPrice * po.OrderQuantity) AS TotalMonthlyCost
FROM
    Part_Order po
JOIN
    Part p ON po.PartID = p.PartID
GROUP BY
    TO_CHAR(po.OrderDate, 'YYYY-MM') -- This grouping now uses the new
index
ORDER BY
    OrderMonth;



SELECT * FROM MonthlyOrderSummaryView;
DROP INDEX idx_part_order_date;

CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF
```

Sample Output:

Monthly Order Summary Report as of 06-SEP-25

Order Month	Number of Orders	Total Monthly Cost
2024-01	26	$102,674.96
2024-02	18	$43,568.21
2024-03	20	$44,813.97
2024-04	13	$26,691.26
2024-05	9	$33,386.28
2024-06	10	$13,000.64
2024-07	21	$48,345.29
2024-08	20	$67,012.47
2024-09	13	$31,204.42
2024-10	20	$52,344.20
2024-11	13	$46,442.88
2024-12	6	$12,515.02
2025-01	9	$25,464.19
2025-02	7	$13,346.67
2025-03	22	$43,831.45
2025-04	14	$36,948.76
2025-05	9	$31,844.12
2025-06	13	$39,635.93
2025-07	21	$47,004.36
2025-08	14	$32,503.72
2025-09	21	$56,055.98
2025-10	16	$49,404.52
2025-11	18	$36,445.75
2025-12	14	$23,913.76
Grand Total:	367	$958,398.81

SQL>

4.5.3 Procedure 1: Full refund for bus schedule cancellation by any bus company

Purpose: This procedure automates the entire process of cancelling a bus schedule and ensuring all affected passengers are refunded.

Its primary purpose is to provide an atomic and reliable way for an administrator to cancel a specific bus trip. When provided with a ScheduleID, the procedure first marks that schedule as 'Cancelled'. It then automatically finds every ticket sold for that trip and processes a full, approved refund for each one by creating a record in the Refund table and updating the ticket's status to 'Refunded'. The procedure includes critical safeguards, such as validating the schedule's existence and preventing duplicate refunds, while its error-handling ensures that the entire operation succeeds or fails as a single unit, thus maintaining data integrity.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_cancel_schedule (
    p_schedule_id IN Schedule.ScheduleID%TYPE
)
IS
    CURSOR cur_bookings IS
        SELECT bd.BookingDetailsID, bd.Price, b.PaymentMethod, t.TicketID
        FROM BookingDetails bd
        JOIN Booking b ON bd.BookingID = b.BookingID
        JOIN Ticket t ON bd.TicketID = t.TicketID
        WHERE t.ScheduleID = p_schedule_id;

v_schedule_count NUMBER;

v_refund_id Refund.RefundID % TYPE;

v_booking_rec cur_bookings % ROWTYPE;

v_refund_count NUMBER;

SCHEDULE_NOT_FOUND EXCEPTION;

PRAGMA EXCEPTION_INIT (SCHEDULE_NOT_FOUND, -20002);

BEGIN

    SELECT COUNT(*)
    INTO v_schedule_count
    FROM Schedule
    WHERE ScheduleID = p_schedule_id;

    IF v_schedule_count = 0 THEN
        RAISE SCHEDULE_NOT_FOUND;
    END IF;


    UPDATE Schedule
    SET ScheduleStatus = 'Cancelled'
    WHERE ScheduleID = p_schedule_id;


    FOR v_booking_rec IN cur_bookings LOOP

        SELECT COUNT(*)
        INTO v_refund_count
        FROM Refund
        WHERE BookingDetailsID = v_booking_rec.BookingDetailsID;
```

```
        IF v_refund_count = 0 THEN

            SELECT 'R' || LPAD(NVL(MAX(TO_NUMBER(SUBSTR(RefundID, 2))), 0)
+ 1, 4, '0')
            INTO v_refund_id
            FROM Refund;

            INSERT INTO Refund (
                RefundID,
                BookingDetailsID,
                RefundDate,
                RefundAmount,
                RefundStatus,
                PaymentMethod,
                RefundReason
            ) VALUES (
                v_refund_id,
                v_booking_rec.BookingDetailsID,
                SYSDATE,
                v_booking_rec.Price,
                'Approved',
                v_booking_rec.PaymentMethod,
                'Schedule ' || p_schedule_id || ' has been cancelled by
the bus company.'
            );


            UPDATE Ticket
            SET Status = 'Refunded'
            WHERE TicketID = v_booking_rec.TicketID;

            DBMS_OUTPUT.PUT_LINE('Booking ' ||
v_booking_rec.BookingDetailsID || ' has been refunded.');

        ELSE
            DBMS_OUTPUT.PUT_LINE('Booking ' ||
v_booking_rec.BookingDetailsID || ' has already been refunded.
Skipping.');
        END IF;

    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Schedule ' || p_schedule_id || ' has been
cancelled and all applicable bookings have been refunded.');

EXCEPTION
    WHEN SCHEDULE_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Schedule ID ' || p_schedule_id || '
not found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
        ROLLBACK;
END;
/
```

Sample Output:

4.5.4 Procedure 2: Staff Shifting Allocation

Purpose: This procedure is designed to assign a qualified staff member to a specific maintenance task and log their involvement.

The purpose is to create a controlled and validated way of recording which staff member worked on a particular maintenance job, what their role was, and for how many hours. It enforces critical business rules by performing several checks before making the assignment: it first confirms that both the maintenance task and the staff member actually exist in the database. Most importantly, it verifies that the assigned staff member's official position is 'Mechanic', preventing non-technical staff from being allocated to maintenance work. If all conditions are met, it inserts a record into the StaffAllocation table; otherwise, it provides a specific error message, ensuring data accuracy and operational integrity.

SQL Statement:

```
CREATE OR REPLACE PROCEDURE prc_manage_maintenance_staff (
    p_maintenance_id IN Maintenance.MaintenanceID%TYPE,
    p_staff_id       IN Staff.StaffID%TYPE,
    p_role           IN VARCHAR2,
    p_worked_hours   IN NUMBER
)
IS
    v_maintenance_count  NUMBER;
    v_staff_position     Staff.Position%TYPE;

    MAINTENANCE_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(MAINTENANCE_NOT_FOUND, -20003);

    STAFF_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(STAFF_NOT_FOUND, -20004);


    STAFF_NOT_A_MECHANIC EXCEPTION;
    PRAGMA EXCEPTION_INIT(STAFF_NOT_A_MECHANIC, -20005);

BEGIN

    SELECT COUNT(*)
    INTO v_maintenance_count
    FROM Maintenance
    WHERE MaintenanceID = p_maintenance_id;

    IF v_maintenance_count = 0 THEN
        RAISE MAINTENANCE_NOT_FOUND;
    END IF;


    BEGIN
        SELECT Position
        INTO v_staff_position
        FROM Staff
        WHERE StaffID = p_staff_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE STAFF_NOT_FOUND;
    END;


    IF UPPER(v_staff_position) != 'MECHANIC' THEN
        RAISE STAFF_NOT_A_MECHANIC;
    END IF;


    INSERT INTO StaffAllocation (
```

```
        StaffID,
        MaintenanceID,
        Role,
        WorkedHour
    ) VALUES (
        p_staff_id,
        p_maintenance_id,
        p_role,
        p_worked_hours
    );

    DBMS_OUTPUT.PUT_LINE('Staff ' || p_staff_id || ' has been successfully
allocated to maintenance task ' || p_maintenance_id || '.');

EXCEPTION
    WHEN MAINTENANCE_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Maintenance ID ' || p_maintenance_id
|| ' not found.');
    WHEN STAFF_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Staff ID ' || p_staff_id || ' not
found.');
    WHEN STAFF_NOT_A_MECHANIC THEN
        DBMS_OUTPUT.PUT_LINE('Error: Staff ' || p_staff_id || ' is a ' ||
v_staff_position || ', not a Mechanic. Allocation failed.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
        ROLLBACK;
END;
/
```

Sample Output:

```
Staff STF030 has been successfully allocated to maintenance task MT00320.

MAINTENANCEID       STAFFID             ROLE                  WORKEDHOUR
------------------- ------------------- --------------------- ----------
MT00320             STF030              Lead Mechanic         8.5
```

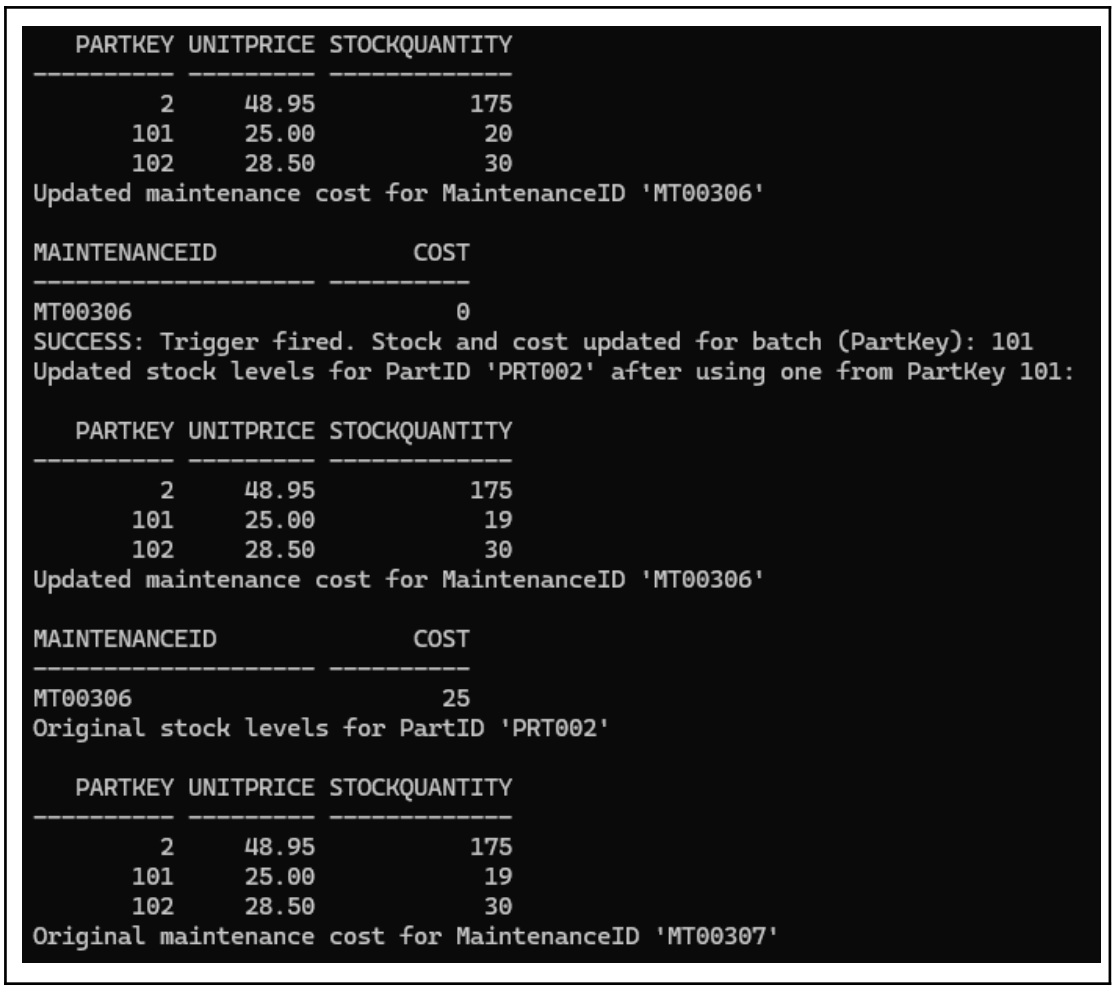
4.5.5 Trigger 1: Audit log on Part table changes

Purpose: This script creates an audit trail for the Part table by implementing the trg_Part_Audit trigger. Its purpose is to automatically log every change made to part data—including new part additions (INSERT), modifications to price or stock levels (UPDATE), and part removals (DELETE). The log captures the old and new values, the user who made the change, and the exact timestamp. This provides managers with a historical record to track inventory changes, monitor price fluctuations from suppliers, and enhance accountability by showing who is responsible for data modifications. This is vital for investigating discrepancies and ensuring compliance.

SQL Statement:

```
CREATE OR REPLACE TRIGGER trg_update_scd_stock_and_cost
AFTER INSERT ON Using_Parts
FOR EACH ROW
DECLARE
    v_current_stock NUMBER;
    v_part_unit_price NUMBER(10, 2);
BEGIN
    -- Step 1: Lock the specific part row (the specific batch) to get its
    -- current stock and unit price. This now uses the composite key.
    SELECT StockQuantity, UnitPrice
    INTO v_current_stock, v_part_unit_price
    FROM Part
    WHERE PartKey = :NEW.PartKey AND PartID = :NEW.PartID -- Using the
composite key
    FOR UPDATE;

    -- Step 2: Check if there is enough stock in this specific batch.
    IF v_current_stock < :NEW.QuantityUsed THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: Insufficient stock for
PartID ' || :NEW.PartID || ' (Batch/PartKey: ' || :NEW.PartKey || ').
Available: ' || v_current_stock || ', Needed: ' || :NEW.QuantityUsed);
    ELSE
        -- Step 3: Update the stock for that specific batch only.
        UPDATE Part
        SET StockQuantity = StockQuantity - :NEW.QuantityUsed
        WHERE PartKey = :NEW.PartKey AND PartID = :NEW.PartID; --
Targeting the specific row

        -- Step 4: Add the cost of the used part(s) from this batch to the
Maintenance record.
        -- Note: The column in Maintenance is named "Cost", not
"MaintenanceCost".
        UPDATE Maintenance
        SET Cost = Cost + (v_part_unit_price * :NEW.QuantityUsed)
        WHERE MaintenanceID = :NEW.MaintenanceID;

        DBMS_OUTPUT.PUT_LINE('SUCCESS: Trigger fired. Stock and cost
updated for batch (PartKey): ' || :NEW.PartKey);
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- Handles cases where the composite key (PartKey, PartID) does
not exist.
        RAISE_APPLICATION_ERROR(-20002, 'Error: The specified part batch
(PartKey ' || :NEW.PartKey || ', PartID ' || :NEW.PartID || ') does not
exist.');
END;
/
```

Sample Output:



```
   PARTKEY UNITPRICE STOCKQUANTITY
---------- --------- -------------
         2     48.95           175
       101     25.00            20
       102     28.50            30
Updated maintenance cost for MaintenanceID 'MT00306'

MAINTENANCEID              COST
-------------------- ----------
MT00306                         0
SUCCESS: Trigger fired. Stock and cost updated for batch (PartKey): 101
Updated stock levels for PartID 'PRT002' after using one from PartKey 101:

   PARTKEY UNITPRICE STOCKQUANTITY
---------- --------- -------------
         2     48.95           175
       101     25.00            19
       102     28.50            30
Updated maintenance cost for MaintenanceID 'MT00306'

MAINTENANCEID              COST
-------------------- ----------
MT00306                        25
Original stock levels for PartID 'PRT002'

   PARTKEY UNITPRICE STOCKQUANTITY
---------- --------- -------------
         2     48.95           175
       101     25.00            19
       102     28.50            30
Original maintenance cost for MaintenanceID 'MT00307'
```

4.5.6 Trigger 2: Real-time Part Stock and Maintenance Cost Update Trigger

Purpose: This trigger, named trg_update_scd_stock_and_cost, automates the core operational process of inventory and cost management. When a part is recorded as being used in a maintenance job (by inserting a row into the Using_Parts table), the trigger instantly performs two critical actions:

1. It decrements the stock quantity of the specific part batch that was used, ensuring the inventory count is always accurate and up-to-date.
2. It calculates the cost of the used part(s) and adds it to the total cost of the corresponding maintenance record.

This ensures data integrity in real-time, preventing stock discrepancies and guaranteeing that maintenance costs are recorded accurately at the moment of use. It also includes error handling to block transactions if there is insufficient stock, which is a crucial operational control.

SQL Statement:

```
CREATE OR REPLACE TRIGGER trg_Part_Audit
AFTER INSERT OR UPDATE OR DELETE ON Part
FOR EACH ROW
DECLARE
    v_action VARCHAR2(10);
BEGIN
    -- Determine the action being performed
    IF INSERTING THEN
        v_action := 'INSERT';
    ELSIF UPDATING THEN
        v_action := 'UPDATE';
    ELSIF DELETING THEN
        v_action := 'DELETE';
    END IF;

    -- Insert the change record into the audit log table
    INSERT INTO Part_Audit_Log (
        LogID,
        TableName,
        PartID,
        Action,
        Old_PartName,
        New_PartName,
        Old_Price,
        New_Price,
        Old_StockQuantity,
        New_StockQuantity,
        ChangedBy,
        ChangeTimestamp
    )
    VALUES (
        audit_log_seq.NEXTVAL,          -- LogID
        'Part',                         -- TableName
        NVL(:OLD.PartID, :NEW.PartID),  -- PartID
        v_action,                       -- Action
        :OLD.PartName,                  -- Old_PartName
        :NEW.PartName,                  -- New_PartName
        :OLD.UnitPrice,                 -- Old_Price
        :NEW.UnitPrice,                 -- New_Price
        :OLD.StockQuantity,             -- Old_StockQuantity
        :NEW.StockQuantity,             -- New_StockQuantity
        USER,                           -- ChangedBy
        SYSTIMESTAMP                    -- ChangeTimestamp
    );
```

```
END;
/
```

Sample Output:

LOGID	PARTID	ACTION	CHANGEDBY	CHANGETIMESTAMP
1	PRT011	INSERT	LIONEL	06-SEP-25 09.28.39.558000 PM
2	PRT011	UPDATE	LIONEL	06-SEP-25 09.28.39.560000 PM
3	PRT011	DELETE	LIONEL	06-SEP-25 09.28.39.562000 PM

4.5.7 Report 1: MONTHLY PARTS USAGE AND COST ANALYSIS REPORT

Purpose: The purpose of this report is to provide a detailed monthly breakdown of parts consumption and associated costs derived from maintenance activities. It is designed to help management and inventory controllers track spending, identify high-usage parts, and make informed decisions regarding budget allocation and stock management.

Specifically, the report:

- Summarizes by Month: Groups all parts usage data into distinct months for clear period-over-period comparison.
- Details Part Consumption: For each month, it lists every part used, the total quantity consumed, and the total cost for that part.
- Provides Consumption Analysis: It calculates the consumption percentage for each part relative to the total units used in that month, making it easy to identify the most frequently used items.
- Aggregates Totals: The report provides subtotals for units and costs at the end of each month and concludes with a grand total summary for the entire period, offering both a granular and a high-level view of parts expenditure.

SQL Statement:

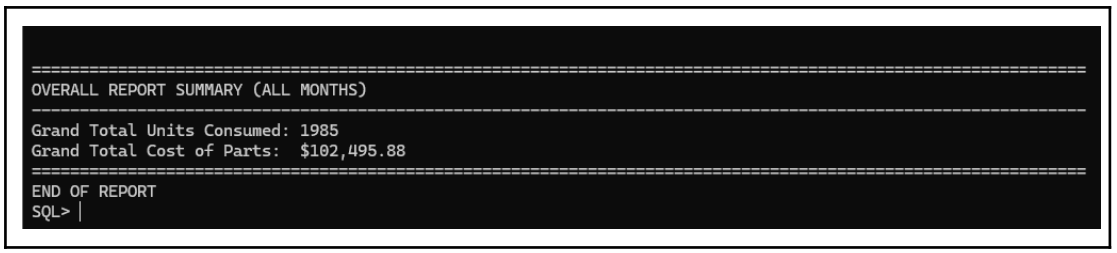
```
SET SERVEROUTPUT ON;
SET LINESIZE 120
SET PAGESIZE 100
CL SCR

CREATE OR REPLACE PROCEDURE Monthly_Part_Cost_And_Usage
IS
    -- Outer cursor: Gets a distinct list of months where parts were used.
    CURSOR c_months IS
        SELECT DISTINCT TO_CHAR(m.ServiceDate, 'YYYY-MM') AS UsageMonth
        FROM Maintenance m
        JOIN Using_Parts up ON m.MaintenanceID = up.MaintenanceID
        ORDER BY UsageMonth DESC;

    -- Inner (nested) cursor: For a given month, summarizes the usage and
cost of each part.
    CURSOR c_part_details (cp_month VARCHAR2) IS
        SELECT
            p.PartName,
            SUM(up.QuantityUsed) AS TotalUsed,
            SUM(up.QuantityUsed * p.UnitPrice) AS TotalCostForPart
        FROM Using_Parts up
        JOIN Part p ON up.PartID = p.PartID
        JOIN Maintenance m ON up.MaintenanceID = m.MaintenanceID
        WHERE TO_CHAR(m.ServiceDate, 'YYYY-MM') = cp_month
        GROUP BY p.PartName
        ORDER BY TotalUsed DESC;

    -- Variables for monthly and grand totals
    v_total_units_in_month    NUMBER;
    v_total_cost_in_month     NUMBER;
    v_grand_total_units       NUMBER := 0;
    v_grand_total_cost        NUMBER := 0;
    v_consumption_percentage  NUMBER;

BEGIN
    -- Print the main report header
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 110, '='));
    DBMS_OUTPUT.PUT_LINE('MONTHLY PARTS USAGE AND COST ANALYSIS REPORT');
    DBMS_OUTPUT.PUT_LINE('Generated on: ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY
```

```
HH24:MI:SS'));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 110, '='));

    -- Loop through each month
    FOR rec_month IN c_months LOOP
        -- Calculate the total units and total cost for the current month
        SELECT SUM(up.QuantityUsed), SUM(up.QuantityUsed * p.UnitPrice)
        INTO v_total_units_in_month, v_total_cost_in_month
        FROM Using_Parts up
        JOIN Maintenance m ON up.MaintenanceID = m.MaintenanceID
        JOIN Part p ON up.PartID = p.PartID
        WHERE TO_CHAR(m.ServiceDate, 'YYYY-MM') = rec_month.UsageMonth;

        -- Print the header for the month
        DBMS_OUTPUT.PUT_LINE(CHR(10));
        DBMS_OUTPUT.PUT_LINE('--- MONTH: ' ||
TO_CHAR(TO_DATE(rec_month.UsageMonth, 'YYYY-MM'), 'Month YYYY') || '
---');
        DBMS_OUTPUT.PUT_LINE(RPAD('Part Name', 40) || RPAD('Units Used',
20) || RPAD('Consumption %', 25) || 'Total Cost');
        DBMS_OUTPUT.PUT_LINE(RPAD('-', 38, '-') || ' ' || RPAD('-', 18,
'-') || ' ' || RPAD('-', 23, '-') || ' ' || RPAD('-', 20, '-'));

        -- For the current month, loop through the part details
        FOR rec_part IN c_part_details(rec_month.UsageMonth) LOOP
            -- Calculate the percentage of total parts used this month
            IF v_total_units_in_month > 0 THEN
                v_consumption_percentage := (rec_part.TotalUsed /
v_total_units_in_month) * 100;
            ELSE
                v_consumption_percentage := 0;
            END IF;

            -- Print the detail line for the part
            DBMS_OUTPUT.PUT_LINE(
                RPAD(rec_part.PartName, 40) ||
                RPAD(rec_part.TotalUsed, 20) ||
                RPAD(TO_CHAR(v_consumption_percentage, '990.00') || '%',
25) ||
                TO_CHAR(rec_part.TotalCostForPart, 'FM$999,999,999.00')
            );
        END LOOP;

        -- Print the summary footer for the month
        DBMS_OUTPUT.PUT_LINE(RPAD('-', 108, '-'));
        DBMS_OUTPUT.PUT_LINE(
            RPAD('MONTHLY TOTALS:', 60) ||
            RPAD('Total Units: ' || v_total_units_in_month, 25) ||
            'Total Cost: ' || TO_CHAR(v_total_cost_in_month,
'FM$999,999,999.00')
        );
        DBMS_OUTPUT.PUT_LINE(CHR(10));

        -- Add monthly totals to grand totals
        v_grand_total_units := v_grand_total_units +
v_total_units_in_month;
        v_grand_total_cost := v_grand_total_cost + v_total_cost_in_month;
    END LOOP;

    -- Print the final Grand Total summary for the entire report
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 110, '='));
    DBMS_OUTPUT.PUT_LINE('OVERALL REPORT SUMMARY (ALL MONTHS)');
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 110, '-'));
    DBMS_OUTPUT.PUT_LINE('Grand Total Units Consumed: ' ||
v_grand_total_units);
    DBMS_OUTPUT.PUT_LINE('Grand Total Cost of Parts:  ' ||
TO_CHAR(v_grand_total_cost, 'FM$999,999,999.00'));
```

```
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 110, '='));
    DBMS_OUTPUT.PUT_LINE('END OF REPORT');

END;
/

-- ======================================================================
-- HOW TO RUN THIS ANALYSIS REPORT
-- ======================================================================
--
-- Simply execute the new procedure.
--
EXEC Monthly_Part_Cost_And_Usage;
--
-- ======================================================================
```

Sample Output:

```
================================================================================================================
MONTHLY PARTS USAGE AND COST ANALYSIS REPORT
Generated on: 06-SEP-2025 21:34:03
================================================================================================================



--- MONTH: December  2025 ---
Part Name                          Units Used         Consumption %          Total Cost
---------------------------------- ------------------ ---------------------- --------------------
AC Compressor                      14                   38.89%               $824.74
Oil Filter                         9                    25.00%               $533.34
Fuel Pump                          9                    25.00%               $823.05
Spark Plug                         4                    11.11%               $111.04
----------------------------------------------------------------------------------------------------------------
MONTHLY TOTALS:                                         Total Units: 36      Total Cost: $2,292.17




--- MONTH: November  2025 ---
Part Name                          Units Used         Consumption %          Total Cost
---------------------------------- ------------------ ---------------------- --------------------
Air Filter                         12                   36.36%               $409.80
Alternator                         10                   30.30%               $534.70
Timing Belt                        5                    15.15%               $407.55
Fuel Pump                          3                     9.09%               $274.35
Battery                            3                     9.09%               $60.36
----------------------------------------------------------------------------------------------------------------
MONTHLY TOTALS:                                         Total Units: 33      Total Cost: $1,686.76




--- MONTH: October   2025 ---
Part Name                          Units Used         Consumption %          Total Cost
---------------------------------- ------------------ ---------------------- --------------------
Air Filter                         57                   75.00%               $1,946.55
Battery                            13                   17.11%               $261.56
Oil Filter                         6                     7.89%               $355.56
----------------------------------------------------------------------------------------------------------------
MONTHLY TOTALS:                                         Total Units: 76      Total Cost: $2,563.67




--- MONTH: September 2025 ---
Part Name                          Units Used         Consumption %          Total Cost
---------------------------------- ------------------ ---------------------- --------------------
Brake Pad                          14                   82.35%               $1,290.10
AC Compressor                      2                    11.76%               $117.82
Timing Belt                        1                     5.88%               $81.51
----------------------------------------------------------------------------------------------------------------
MONTHLY TOTALS:                                         Total Units: 17      Total Cost: $1,489.43
```


OVERALL REPORT SUMMARY (ALL MONTHS)
Grand Total Units Consumed: 1985
Grand Total Cost of Parts: $102,495.88
END OF REPORT
SQL>

4.5.8 Report 2: SUPPLIER CONTRIBUTION AND VALUE ANALYSIS REPORT

Purpose: The purpose of this report is to provide a comprehensive analysis of each supplier's contribution to the organization's procurement activities. It evaluates suppliers based on the total financial value of the parts they provide, allowing for a clear understanding of which partners are most critical to the supply chain.

Specifically, the report is designed to:

- Quantify Supplier Value: It calculates the total monetary value of all parts ordered from each individual supplier.
- Measure Strategic Importance: By comparing each supplier's total value against the grand total of all orders, it determines their percentage contribution, making it easy to identify key and high-value partners.
- Detail Sourced Parts: For each supplier, it provides a breakdown of every unique part they have supplied, including the total quantity ordered and the associated subtotal value. This helps to understand what specific items drive a supplier's value.
- Provide Actionable Insights: With summary metrics like the total value, contribution percentage, and average value per part type, the report equips management for strategic sourcing, contract negotiations, and identifying potential supply chain risks or dependencies.

SQL Statement:

```
SET SERVEROUTPUT ON;
SET LINESIZE 120
SET PAGESIZE 100
CL SCR

CREATE OR REPLACE PROCEDURE Supplier_Contribution_Analysis
IS
    -- Outer cursor: Gets all suppliers.
    CURSOR c_suppliers IS
        SELECT SupplierID, SupplierName
        FROM Supplier
        ORDER BY SupplierName;

    -- Inner cursor: Gets the details for each unique part ordered from a
supplier.
    CURSOR c_part_details (cp_supplier_id VARCHAR2) IS
        SELECT
            p.PartName,
            SUM(po.OrderQuantity) AS TotalQuantityOrdered,
            SUM(p.UnitPrice * po.OrderQuantity) AS TotalValueForPart
        FROM Part_Order po
        JOIN Part p ON po.PartID = p.PartID
        WHERE po.SupplierID = cp_supplier_id
        GROUP BY p.PartName
        ORDER BY TotalValueForPart DESC; -- Order by value to see the most
impactful parts first

    -- Variables for calculations
    v_supplier_total_value    NUMBER;
    v_supplier_unique_parts   NUMBER;
    v_grand_total_value       NUMBER;
    v_contribution_percentage NUMBER;
    v_average_part_cost       NUMBER;
    v_has_orders              BOOLEAN;

BEGIN
    -- Pre-calculate the grand total value of all orders across all
suppliers
```

```
    SELECT SUM(p.UnitPrice * po.OrderQuantity)
    INTO v_grand_total_value
    FROM Part_Order po
    JOIN Part p ON po.PartID = p.PartID;

    -- Print the main report header
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 120, '='));
    DBMS_OUTPUT.PUT_LINE('SUPPLIER CONTRIBUTION AND VALUE ANALYSIS
REPORT');
    DBMS_OUTPUT.PUT_LINE('Generated on: ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY
HH24:MI:SS'));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 120, '='));

    -- Loop through each supplier
    FOR rec_supplier IN c_suppliers LOOP
        -- Reset totals and flag for each new supplier
        v_supplier_total_value := 0;
        v_supplier_unique_parts := 0;
        v_has_orders := FALSE;

        -- Process all parts for the current supplier to gather totals
        FOR rec_part IN c_part_details(rec_supplier.SupplierID) LOOP
            -- If this is the FIRST part, print the supplier's header
            IF NOT v_has_orders THEN
                DBMS_OUTPUT.PUT_LINE(CHR(10));
                DBMS_OUTPUT.PUT_LINE('Supplier: ' ||
rec_supplier.SupplierName);
                DBMS_OUTPUT.PUT_LINE(RPAD('-', 115, '-'));
                DBMS_OUTPUT.PUT_LINE('  ' || RPAD('Part Name', 50) ||
RPAD('Total Quantity', 25) || 'Subtotal Value');
                DBMS_OUTPUT.PUT_LINE('  ' || RPAD('-', 48, '-') || ' ' ||
RPAD('-', 23, '-') || ' ' || RPAD('-', 20, '-'));
                v_has_orders := TRUE;
            END IF;

            -- Print the detail line for the specific part
            DBMS_OUTPUT.PUT_LINE(
                '  ' || RPAD(rec_part.PartName, 50) ||
                RPAD(rec_part.TotalQuantityOrdered, 25) ||
                TO_CHAR(rec_part.TotalValueForPart, 'FM$999,999,999.00')
            );

            -- Add to the running totals for the supplier
            v_supplier_total_value := v_supplier_total_value +
rec_part.TotalValueForPart;
            v_supplier_unique_parts := v_supplier_unique_parts + 1;
        END LOOP;

        -- After processing parts, if the supplier had orders, print their
enhanced summary footer
        IF v_has_orders THEN
            -- Perform the new calculations
            v_contribution_percentage := (v_supplier_total_value /
v_grand_total_value) * 100;
            v_average_part_cost := v_supplier_total_value /
v_supplier_unique_parts;

            DBMS_OUTPUT.PUT_LINE(RPAD('-', 115, '-'));
            DBMS_OUTPUT.PUT_LINE(
                RPAD('Summary for ' || rec_supplier.SupplierName || ':',
75) ||
                'Total Value: ' || TO_CHAR(v_supplier_total_value,
'FM$999,999,999.00')
            );
            DBMS_OUTPUT.PUT_LINE(
                RPAD(' ', 75) || 'Contribution to Grand Total: ' ||
TO_CHAR(v_contribution_percentage, '990.00') || '%'
```

```
            );
            DBMS_OUTPUT.PUT_LINE(
                RPAD(' ', 75) || 'Average Value per Part Type: ' ||
TO_CHAR(v_average_part_cost, 'FM$999,999.00')
            );
        END IF;
    END LOOP;

    -- Print the final Grand Total summary for the entire report
    DBMS_OUTPUT.PUT_LINE(CHR(10) || RPAD('=', 110, '='));
    DBMS_OUTPUT.PUT_LINE('OVERALL REPORT SUMMARY');
    DBMS_OUTPUT.PUT_LINE('Grand Total Value of All Orders: ' ||
TO_CHAR(v_grand_total_value, 'FM$999,999,999.00'));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 110, '='));
    DBMS_OUTPUT.PUT_LINE('END OF REPORT');

END;
/

-- ========================================================================
-- HOW TO RUN THIS ANALYSIS REPORT
-- ========================================================================
--
EXEC Supplier_Contribution_Analysis;
--
-- ========================================================================
```

Sample Output:

```
================================================================================================================================
SUPPLIER CONTRIBUTION AND VALUE ANALYSIS REPORT
Generated on: 06-SEP-2025 21:36:31
================================================================================================================================



Supplier: Eco Solutions
--------------------------------------------------------------------------------------------------------------------------------
Part Name                                          Total Quantity          Subtotal Value
---------------------------------------------- ---------------------- --------------------
Brake Pad                                          341                     $31,423.15
Air Filter                                         489                     $16,699.35
Fuel Pump                                          146                     $13,351.70
AC Compressor                                      155                     $9,131.05
Radiator                                           237                     $6,387.15
Oil Filter                                         63                      $3,733.38
Spark Plug                                         100                     $2,776.00
Alternator                                         34                      $1,817.98
Battery                                            81                      $1,629.72
--------------------------------------------------------------------------------------------------------------------------------
Summary for Eco Solutions:                                                 Total Value: $86,949.48
Contribution to Grand Total:    9.07%
Average Value per Part Type: $9,661.05



Supplier: Eco Systems
--------------------------------------------------------------------------------------------------------------------------------
Part Name                                          Total Quantity          Subtotal Value
---------------------------------------------- ---------------------- --------------------
Alternator                                         177                     $9,464.19
Air Filter                                         243                     $8,298.45
Brake Pad                                          88                      $8,109.20
AC Compressor                                      124                     $7,304.84
Oil Filter                                         85                      $5,037.10
Battery                                            194                     $3,903.28
Radiator                                           120                     $3,234.00
Timing Belt                                        28                      $2,282.28
Fuel Pump                                          5                       $457.25
Spark Plug                                         13                      $360.88
--------------------------------------------------------------------------------------------------------------------------------
Summary for Eco Systems:                                                   Total Value: $48,451.47
Contribution to Grand Total:    5.06%
Average Value per Part Type: $4,845.15



Supplier: Max In Motion
--------------------------------------------------------------------------------------------------------------------------------
Part Name                                          Total Quantity          Subtotal Value
---------------------------------------------- ---------------------- --------------------
Timing Belt                                        225                     $18,339.75
Brake Pad                                          156                     $14,375.40
Spark Plug                                         484                     $13,435.84
Air Filter                                         207                     $7,069.05
Fuel Pump                                          65                      $5,944.25
Alternator                                         106                     $5,667.82
Battery                                            200                     $4,024.00
AC Compressor                                      43                      $2,533.13
Radiator                                           91                      $2,452.45
--------------------------------------------------------------------------------------------------------------------------------
Summary for Max In Motion:                                                 Total Value: $73,841.69
Contribution to Grand Total:    7.70%
Average Value per Part Type: $8,204.63
```

```
================================================================================================================================
OVERALL REPORT SUMMARY
Grand Total Value of All Orders: $958,398.81
================================================================================================================================
END OF REPORT
```

Chapter 5: Extra Effort Highlight

5.1 CHIN YIN ERN

5.1.1 Views

Query 1: Rank Member with Net Spending

```
CREATE OR REPLACE VIEW Member_Spending_Record AS
```

Query 2: Membership Refund Status by Year

```
CREATE OR REPLACE VIEW Membership_Refund_Yearly AS
```

5.1.2 Indexes

Query 1: Rank Member with Net Spending

```
DROP INDEX member_name_idx;
CREATE INDEX member_name_idx ON Member(UPPER(FullName));
```

Query 2: Membership Refund Status by Year

```
DROP INDEX membership_status_idx;
CREATE INDEX membership_status_idx ON Member(UPPER(MembershipStatus));
```

5.1.3 User Defined Functions

-

5.1.4 User Defined Exceptions

Procedure 1: Add New Maintenance Record for a Bus

- Raise exception when member id does not exist, ticket not available and booking id already exist.

```
    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20030, 'Member ID ' || v_MemberID || '
does not exist.');
    END IF;

    EXCEPTION
       WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20020, 'Ticket not available for
booking. It may be Past, Active, or already booked.');
    END;
```

```
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(-20021, 'Booking ID already exists.');
END;
/
```

Procedure 2: Member Refund Ticket

- Raise exceptions when no member and ticket are not found in the member and ticket table.

```
    MEMBER_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(MEMBER_NOT_FOUND, -20000);

    TICKET_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(TICKET_NOT_FOUND, -20001);

    IF v_count = 0 THEN
        RAISE MEMBER_NOT_FOUND;
    END IF;

    IF v_count = 0 THEN
        RAISE TICKET_NOT_FOUND;
    END IF;

EXCEPTION
    WHEN MEMBER_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Member ID ' || v_member_id || ' not
found.');
    WHEN TICKET_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Ticket ID ' || v_ticket_id || ' not found
for Member ID ' || v_member_id);
END;
/
```

Trigger 1: Prevent members refund tickets after scheduled departure time

- Raise exception when ticket has been purchased or refund ticket with past departure time .

```
    IF v_ticket_status = 'Available' THEN
        RAISE_APPLICATION_ERROR(-20002, 'This ticket not been purchase.');
    END IF;

    IF v_departure_time < SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20003, 'Cannot refund a ticket with past
departure time.');
    END IF;
```

5.1.5 Sequence

Trigger 2: Audit Log for Ticket Fare

```
Drop sequence ticket_audit_seq;

CREATE SEQUENCE ticket_audit_seq
MINVALUE 1
MAXVALUE 999999
START WITH 1
INCREMENT BY 1
NOCACHE;
```

5.1.6 Output Formatting

Query 1: Rank Member with Net Spending

```
SET LINESIZE 120
SET PAGESIZE 50
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

TTITLE CENTER 'Ranking of Member Net Spending on ' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2

COLUMN MemberName FORMAT A30 HEADING 'Member Name'
COLUMN TotalBookings FORMAT 9999 HEADING 'Total Bookings'
COLUMN TotalBookingAmount FORMAT $999,999,999.99 HEADING 'Total Booking
Amount'
COLUMN TotalRefundAmount FORMAT $999,999,999.99 HEADING 'Total Refunds'
COLUMN NetSpend FORMAT $999,999,999.99 HEADING 'Net Spend'
```

Query 2: Membership Refund Status by Year

```
SET LINESIZE 80
SET PAGESIZE 50
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

TTITLE CENTER 'Membership Refund Status by Year on ' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2

COLUMN BookingYear       FORMAT 9999 HEADING 'Year'
COLUMN MembershipStatus  FORMAT A20 HEADING 'Membership Status'
COLUMN TotalTickets      FORMAT 999 HEADING 'Tickets Sold'
COLUMN TotalRefunded     FORMAT 999 HEADING 'Tickets Refunded'
COLUMN TotalRefundAmount FORMAT $999,999,999.99 HEADING 'Refund Amount'
```

5.2 EDISON CHAN YOONG QI

5.2.1 Views For Query 1 and Query 2

Query 1: Revenue Generated by Bus Companies on Various Destinations view

```
CREATE OR REPLACE VIEW EDISON_PIVOT_1 AS
```

Query 2: Total Revenue Generated by each Bus Company view

```
CREATE OR REPLACE VIEW CTE_EDISON_QUERY_1 AS
```

5.2.2 Indexes

Query 1 incorporates indexes based on destination to allow for faster search based on name than id

```
DROP INDEX destination_idx;
CREATE INDEX destination_idx ON Schedule(UPPER(Destination));
```

Query 2 incorporates indexes based on company name to allow for faster search based on name than id

```
DROP INDEX companyName_idx;
CREATE INDEX companyName_idx ON Bus_Company(UPPER(CompanyName));
```

5.2.3 User Defined Functions

None

5.2.4 User Defined Exceptions

Report 1 uses an exception to handle no data found to handle NULL values when printing report

```
EXCEPTION
                    WHEN NO_DATA_FOUND THEN
                        v_popularDest := 'N/A';
                END;
```

Trigger 1 (staff audit log) uses custom exception to handle negative salary on insert or update

```
DECLARE
    ex_negative_salary EXCEPTION;
BEGIN
…
…
…

EXCEPTION
    WHEN ex_negative_salary THEN
        RAISE_APPLICATION_ERROR(-20002, 'Negative salary not allowed for
staff!!!');
```

Trigger 2 (Ticket Extension) uses exception to auto update table values when no new schedule is available for extension

```
EXCEPTION
        WHEN NO_DATA_FOUND THEN
            :NEW.ExtensionStatus := 'REJECTED';
            :NEW.ExtensionFee    := 0;
            RETURN;
    END;
```

Procedure 1 has a custom exception to handle duplicate members on insert

```
CREATE OR REPLACE PROCEDURE prc_Edison_1 (
    v_FullName IN VARCHAR2,
    v_PhoneNo IN VARCHAR2,
    v_Email IN VARCHAR2,
    v_MembershipStatus IN VARCHAR2
)
IS
    ex_dup_member EXCEPTION;

BEGIN
…
…
…
EXCEPTION
    WHEN ex_dup_member THEN
        DBMS_OUTPUT.PUT_LINE('Error: Member with name "' || v_FullName ||
```

```
'" already exists.');
        DBMS_OUTPUT.PUT_LINE('No data has been added.');
END prc_Edison_1;
/
```

5.2.5 Sequence

Staff Audit Log Table uses a sequence

```
CREATE SEQUENCE staff_audit_seq
MINVALUE 1
MAXVALUE 999999
START WITH 1
INCREMENT BY 1
NOCACHE;
```

5.2.6 Output Formatting

Query 1 , Revenue Generated by Bus Companies on Various Destinations uses column formatting and headers

```
SET LINESIZE 250
SET PAGESIZE 100
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';
COLUMN CompanyName FORMAT A20 HEADING 'Company Name'
COLUMN Total_Revenue FORMAT $999999.99 HEADING 'Total Revenue'
COLUMN Destination FORMAT 99,999.99
TTITLE CENTER 'Revenue Generated by Bus Companies on Various Destinations,
Generated on: ' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2
COMPUTE SUM LABEL 'Grand Total Revenue: ' OF Total_Revenue ON REPORT
```

Query 2, Total Revenue Generated by each Bus Company uses column formatting and headers

```
SET LINESIZE 100
SET PAGESIZE 100
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

COLUMN "Company Name" FORMAT A30 HEADING 'Company Name'
COLUMN "Total Revenue" FORMAT $999999.99 HEADING 'Total Revenue'
COLUMN "Revenue Difference" FORMAT $999999.99 HEADING 'Revenue Difference'
TTITLE CENTER 'Total Revenue Generated by each Bus Company  Generated on:
' _DATE -
LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2

BREAK ON REPORT
COMPUTE SUM LABEL 'Total Revenue: ' OF "Total Revenue" ON REPORT
COMPUTE AVG LABEL 'Average Difference: ' OF "Revenue Difference" ON REPORT
```

5.3 KELLY TAN JIE LI

5.3.1 Views

Query 1: Ranked Bus Reliability

```
CREATE OR REPLACE VIEW Ranked_Bus_Reliability AS
```

Query 2: Yearly Maintenance Spending Trends Across Bus Companies

```
CREATE OR REPLACE VIEW Bus_Company_Cost_Trend AS
```

5.3.2 Indexes

Query 1: Ranked Bus Reliability

```
CREATE INDEX idx_schedule_status ON Schedule(UPPER(scheduleStatus));
```

Query 2: Yearly Maintenance Spending Trends Across Bus Companies

```
CREATE INDEX idx_maintenance_date ON Maintenance(serviceDate);
```

5.3.3 User Defined Functions

None

5.3.4 User Defined Exceptions

Procedure 1: Add New Maintenance Record for a Bus

- Raise exception when busID not found, serviceType not found, bus already has on ongoing maintenance that hasn't completed yet

```
CREATE OR REPLACE PROCEDURE prc_addNewMaintenance (
    .
    .
    .
) AS
    .
    .
    .

    ex_no_bus_found      EXCEPTION;
    ex_no_service_found  EXCEPTION;
    ex_bus_service_exists EXCEPTION;

BEGIN
.
.
.
```

```
EXCEPTION
    WHEN ex_no_bus_found THEN
        DBMS_OUTPUT.PUT_LINE('Bus ' || v_busID || ' not found.');
    WHEN ex_no_service_found THEN
        DBMS_OUTPUT.PUT_LINE('Service Type ' || v_serviceTypeID || ' not
found.');
    WHEN ex_bus_service_exists THEN
        DBMS_OUTPUT.PUT_LINE('Bus ' || v_busID || ' already has an ongoing
maintenance (not completed).');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
```

Procedure 2: Search Bus Schedules with Passenger Details

- Raise exception when no schedules found on the input scheduleDate, destination not found for the given date

```
 CREATE OR REPLACE PROCEDURE Search_Bus_Schedules_Details (
    .
    .
    .
  ) IS
      no_schedule_found EXCEPTION;
      no_destination_found EXCEPTION;

BEGIN
    .
    .
    .

EXCEPTION
      WHEN no_schedule_found THEN
          DBMS_OUTPUT.PUT_LINE('No schedules found on the given date.');
      WHEN no_destination_found THEN
          DBMS_OUTPUT.PUT_LINE('Destination not found for the given
date.');
      WHEN OTHERS THEN
          DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
  END;
```

Trigger 2: Audit Maintenance Status Changes

- Raise exception when invalid bus status is being inserted to the maintenance

```
DECLARE
    ex_invalid_status EXCEPTION;
    PRAGMA EXCEPTION_INIT(ex_invalid_status, -20011);
BEGIN
.
.
.

EXCEPTION
    WHEN ex_invalid_status THEN
        RAISE_APPLICATION_ERROR(-20011, 'Invalid status value. Must be
```

```
Scheduled, Completed, or Cancelled.');
END;
/
```

5.3.5 Sequence

Trigger 2: Audit Maintenance Status Changes

```
CREATE SEQUENCE maintenanceStatus_audit_seq
MINVALUE 1
MAXVALUE 99999
START WITH 1
INCREMENT BY 1
NOCACHE;
```

5.3.6 Output Formatting

Query 1: Ranked Bus Reliability

```
SET LINESIZE 100
SET PAGESIZE 50
ALTER SESSION SET nls_date_format = 'DD-MM-YYYY';


TTITLE CENTER 'Bus Reliability Ranking as of ' _DATE SKIP 2

COLUMN BusID FORMAT A8 HEADING 'Bus ID'
COLUMN PlateNo FORMAT A12 HEADING 'Plate No'
COLUMN BusType FORMAT A20 HEADING 'Bus Type'
COLUMN TotalSchedules FORMAT 999999 HEADING 'Total Schedules'
COLUMN TotalCancelledSchedules FORMAT 999999 HEADING 'Total Cancellation'
COLUMN ReliabilityPercentage FORMAT 990.99 HEADING 'Reliability %'
COLUMN ReliabilityRank FORMAT 999 HEADING 'Rank'
```

Query 2: Yearly Maintenance Spending Trends Across Bus Companies

```
SET LINESIZE 100
SET PAGESIZE 50
ALTER SESSION SET nls_date_format = 'DD-MM-YYYY';

CREATE INDEX idx_maintenance_date ON Maintenance(serviceDate);

TTITLE CENTER 'Yearly Maintenance Spending Trends Across Bus Companies as
of ' _DATE SKIP 2
BREAK ON year SKIP 1

COLUMN companyName FORMAT A25 HEADING 'Company Name'
COLUMN year FORMAT 9999 HEADING 'Year'
COLUMN totalCost FORMAT 999999999.99 HEADING 'Total Cost'
COLUMN prevYearCost FORMAT 999999990.99 HEADING 'Previous Year Cost'
COLUMN costDifference FORMAT 999999999.99 HEADING 'Cost Difference'
COLUMN costChangePct FORMAT 990.99 HEADING 'Cost Change %'
```

5.4 KENNY LOH KIT YI

5.4.1 Views

Query 1: Ranking Driver With Most Schedules Allocated for the Past 3 Months view

```
CREATE OR REPLACE VIEW DriversActiveRankingView AS
```

Query 2: Ranking Staff With Most Rent Collected And Dominant Shop Type view

```
CREATE OR REPLACE VIEW StaffRentCollectingView AS
```

5.4.2 Indexes

Query 1: Ranking Driver With Most Schedules Allocated for the Past 3 Months

```
DROP INDEX idx_schedule_date;
CREATE INDEX idx_schedule_date ON Schedule(ScheduleDate);
```

Query 2: Ranking Staff With Most Rent Collected And Dominant Shop Type

```
DROP INDEX idx_rental_shopid;
CREATE INDEX idx_rental_shopid ON RentalCollection(ShopID);
```

5.4.3 User Defined Functions

None

5.4.4 User Defined Exceptions

Procedure 1: Adding Driver Allocation while Ensuring no Multiple Allocation on Same Date

```
CREATE OR REPLACE PROCEDURE prc_AddDriverAllocation (
.
.
.
) IS
.
.
MULTIPLE_ALLOCATION EXCEPTION;
PRAGMA EXCEPTION_INIT(MULTIPLE_ALLOCATION, -20000);

BEGIN
.
.
.

    EXCEPTION
    WHEN MULTIPLE_ALLOCATION THEN
        DBMS_OUTPUT.PUT_LINE('Driver ' || v_driverID || ' is already
allocated on the same date as ' || v_scheduleID || '.');

END;
/
```

Procedure 2: Adding Collected Rent with Validation and Auto Assign ID and Date

```
CREATE OR REPLACE PROCEDURE prc_AddRentCollection (
.
.
.
) IS
.
.
.

BEGIN
.
.
.

    IF v_shopExists = 0 THEN
        RAISE_APPLICATION_ERROR(-20000, 'Invalid Shop ID.');
    END IF;

.
.
.

    IF v_staffExists = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid Staff ID.');
    END IF;

    IF v_amountCollected <= 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Amount collected must be greater
than 0.');
    END IF;

.
.
.

END;
/
```

5.4.5 Sequence

Trigger 1: Audit Log for Driver Allocation

```
DROP SEQUENCE driverAllocation_audit_seq;

CREATE SEQUENCE driverAllocation_audit_seq
MINVALUE 1
MAXVALUE 99999
START WITH 1
INCREMENT BY 1
NOCACHE;
```

5.4.6 Output Formatting

Query 1: Ranking Driver With Most Schedules Allocated for the Past 3 Months

```
SET PAGESIZE 50
SET LINESIZE 100

TTITLE CENTER 'Active Drivers Ranking (Past 3 Months)' SKIP 2

COLUMN "Driver ID" FORMAT A20
COLUMN "Driver Name" FORMAT A50
COLUMN "Total Schedules" FORMAT 999
COLUMN Ranking FORMAT 999
```

Query 2: Ranking Staff With Most Rent Collected And Dominant Shop Type

```
SET PAGESIZE 50
SET LINESIZE 120

TTITLE CENTER 'Staff Rent Collecting Ranking' SKIP 2

COLUMN StaffID FORMAT A9
COLUMN FullName FORMAT A20
COLUMN "Dominant Shop Type" FORMAT A50
COLUMN TotalCollections FORMAT 99
COLUMN CollectionRank FORMAT 9
```

5.5 KHIEW JIT CHEN

5.5.1 Views

Using in both query

```
CREATE OR REPLACE VIEW TOP_10_ROUTES_VIEW AS
```

```
CREATE OR REPLACE VIEW MonthlyOrderSummaryView AS
```

5.5.2 Indexes

Using in both query to speed up the both data retrieval.

```
CREATE INDEX idx_schedule_station_upper ON Schedule(UPPER(Station));
CREATE INDEX idx_schedule_destination_upper ON
Schedule(UPPER(Destination));
```

```
CREATE INDEX idx_part_order_date ON Part_Order(UPPER(OrderDate));
```

5.5.3 User Defined Functions

Comprehensive Maintenance Cost Update Procedure (using user defined function)

Purpose: The Update_All_Maintenance_Costs stored procedure is designed to perform a comprehensive recalculation of costs for all maintenance records. It iterates through each maintenance job and aggregates two key cost components:

1. The total cost of all parts used during the service.
2. The standard service fee associated with the type of maintenance performed.

By summing these values, the procedure provides a complete and accurate total cost for each maintenance activity. This allows managers to analyze the true cost of maintaining the bus fleet, evaluate the profitability of different services, and make informed budgetary decisions based on reliable, aggregated financial data.

SQL Statement:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION Get_Parts_Total_Cost (
    p_maintenance_id IN VARCHAR2,
    p_service_date IN DATE
) RETURN NUMBER
IS
    v_total_cost NUMBER;
BEGIN
    -- Calculate the sum of (unit price * quantity) for all parts
associated
    -- with a specific maintenance ID, ensuring the service date is within
    -- the part's valid price period.
    -- NVL ensures we return 0 if no parts were used (SUM would be NULL).
    SELECT NVL(SUM(p.UnitPrice * up.QuantityUsed), 0)
    INTO v_total_cost
    FROM Using_Parts up
    JOIN Part p ON up.PartID = p.PartID
```

```
    WHERE up.MaintenanceID = p_maintenance_id
      AND p_service_date >= p.StartDate
      AND (p.EndDate IS NULL OR p_service_date <= p.EndDate);

    RETURN v_total_cost;

EXCEPTION
    -- In case of any unexpected error, return 0 as a safe default.
    WHEN OTHERS THEN
        RETURN 0;
END Get_Parts_Total_Cost;
/


CREATE OR REPLACE FUNCTION Get_Standard_Service_Fee (
    p_maintenance_id IN VARCHAR2
) RETURN NUMBER
IS
    v_service_fee NUMBER;
BEGIN
    -- Joins Maintenance and Service_Type tables to find the standard cost
    -- for the service performed under the given maintenance ID.
    SELECT st.StandardCost
    INTO v_service_fee
    FROM Maintenance m
    JOIN Service_Type st ON m.ServiceTypeID = st.ServiceTypeID
    WHERE m.MaintenanceID = p_maintenance_id;

    RETURN v_service_fee;

EXCEPTION
    -- If the MaintenanceID is not found or another error occurs, return
0.
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
    WHEN OTHERS THEN
        RETURN 0;
END Get_Standard_Service_Fee;
/


CREATE OR REPLACE PROCEDURE Update_All_Maintenance_Costs
IS
    -- Cursor to select every maintenance record that needs processing.
    CURSOR c_maintenance IS
        SELECT MaintenanceID, ServiceDate FROM Maintenance;

    v_final_maintenance_cost NUMBER;
    v_row_count NUMBER := 0;

BEGIN
    -- Loop through each record from the cursor.
    FOR rec IN c_maintenance LOOP

        -- Step 1: Calculate the final cost by calling our new functions.
        -- The logic is now hidden away, making this procedure clean and
readable.
        v_final_maintenance_cost :=
Get_Parts_Total_Cost(rec.MaintenanceID, rec.ServiceDate)
                                   +
Get_Standard_Service_Fee(rec.MaintenanceID);

        -- Step 2: Update the specific row in the Maintenance table with
the calculated cost.
        UPDATE Maintenance
        SET Cost = v_final_maintenance_cost
        WHERE MaintenanceID = rec.MaintenanceID;
```

```
        -- Increment counter for the final summary report.
        v_row_count := v_row_count + 1;

    END LOOP;

    -- Commit the transaction once after all rows have been processed.
    COMMIT;


DBMS_OUTPUT.PUT_LINE('================================================='
);
    DBMS_OUTPUT.PUT_LINE('Procedure Complete.');
    DBMS_OUTPUT.PUT_LINE('Total maintenance records updated: ' ||
v_row_count);

DBMS_OUTPUT.PUT_LINE('================================================='
);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
        DBMS_OUTPUT.PUT_LINE('Transaction has been rolled back.');
        ROLLBACK;
END;
/


-- Simply run this block to update all maintenance costs using the
refactored logic.
BEGIN
    Update_All_Maintenance_Costs;
END;
/


SELECT
    SUM(p.UnitPrice * up.QuantityUsed) AS calculated_parts_cost
FROM
    Using_Parts up
JOIN
    Part p ON up.PartID = p.PartID
JOIN
    Maintenance m ON up.MaintenanceID = m.MaintenanceID
WHERE
    up.MaintenanceID = 'MT00010'
    AND m.ServiceDate >= p.StartDate
    AND (p.EndDate IS NULL OR m.ServiceDate <= p.EndDate);


SELECT
    st.StandardCost AS standard_service_fee
FROM
    Maintenance m
JOIN
    Service_Type st ON m.ServiceTypeID = st.ServiceTypeID
WHERE
    m.MaintenanceID = 'MT00010';


SELECT
    Cost AS procedure_updated_cost
FROM
    Maintenance
WHERE
    MaintenanceID = 'MT00010';
```

Sample Output:

Booking Total Amount Calculation Procedure (using user defined function)

Purpose: The Update_Booking_Total_Cost procedure is a data integrity tool designed to ensure financial accuracy. Its function is to calculate the total cost of a booking by summing up the prices of all individual items in the BookingDetails table and then updating the TotalAmount field in the master Booking table. This is essential for correcting any data inconsistencies and ensuring that the final amount for invoicing and reporting is always accurate. The script includes a block to run this procedure for all bookings, making it a useful tool for batch data cleansing and maintenance.

SQL Statement:

```
SET SERVEROUTPUT ON;
SET LINESIZE 1000;
SET PAGESIZE 1000;

CREATE OR REPLACE FUNCTION Get_Booking_Total_Cost (
    p_booking_id IN VARCHAR2
) RETURN NUMBER
IS
    v_total_cost NUMBER;
BEGIN
    -- Calculate the sum of prices for the given booking ID.
    -- The NVL function ensures that if no rows are found (SUM returns
NULL),
    -- we return 0 instead.
    SELECT NVL(SUM(bd.Price), 0)
    INTO v_total_cost
    FROM BookingDetails bd
    WHERE bd.BookingID = UPPER(p_booking_id);

    RETURN v_total_cost;

EXCEPTION
    -- In case of an unexpected database error during the SELECT,
    -- return 0 as a safe default value.
    WHEN OTHERS THEN
        RETURN 0;
END Get_Booking_Total_Cost;
/
```

```
CREATE OR REPLACE PROCEDURE Update_Booking_Total_Cost (
    p_booking_id IN VARCHAR2
)
IS
    v_booking_id VARCHAR2(8) := UPPER(p_booking_id);
    v_calculated_total NUMBER;

    -- Custom exception for when a booking ID doesn't exist.
    e_booking_not_found EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_booking_not_found, -20080);

BEGIN
    -- Step 1: Call the user-defined function to get the total cost.
    -- This encapsulates and simplifies the calculation logic.
    v_calculated_total := Get_Booking_Total_Cost(v_booking_id);

    -- Step 2: Update the main Booking table with the value from the
function.
    UPDATE Booking
    SET TotalAmount = v_calculated_total
    WHERE BookingID = v_booking_id;

    -- Step 3: Check if the update affected any rows.
    -- If not, it means the BookingID didn't exist in the Booking table.
    IF SQL%ROWCOUNT = 0 THEN
        RAISE e_booking_not_found;
    END IF;

    -- If successful, commit the changes.
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Success: TotalAmount for BookingID ' ||
v_booking_id || ' has been updated to ' || TO_CHAR(v_calculated_total,
'FM999,999.00'));

EXCEPTION
    -- Handle our custom "not found" error.
    WHEN e_booking_not_found THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Update failed. BookingID ''' ||
v_booking_id || ''' does not exist in the Booking table.');
        ROLLBACK;
    -- Handle all other potential errors.
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
        ROLLBACK;
END Update_Booking_Total_Cost;
/

BEGIN
    FOR rec IN (SELECT BookingID FROM Booking) LOOP
        Update_Booking_Total_Cost(rec.BookingID);
    END LOOP;
END;
/

-- This will succeed (assuming 'B0001' exists)
-- EXEC Update_Booking_Total_Cost('B0001');

-- This will fail and trigger the custom exception
-- EXEC Update_Booking_Total_Cost('B9999'); -- An ID that does not exist
```

Sample Output:

```
BOOKINGDETAILSID    TICKETID            BOOKINGID               PRICE
------------------- ------------------- ------------------- ----------
BD000157            TCK00157            BK0001                      20
BD000168            TCK00168            BK0001                      20
BD000221            TCK00221            BK0001                      20
BD000244            TCK00244            BK0001                      20
BD000298            TCK00298            BK0001                      20

BOOKINGID       MEMBERID        BOOKINGDATE  TOTALAMOUNT PAYMENTSTATUS                          PAYMENTMETHOD
--------------- --------------- ----------- ----------- ---------------------------------- -------------
BK0001          M0001           15-JAN-24           100 Complete                           Credit Card
```

5.5.4 User Defined Exceptions

Use in procedure 2 : to check if the staff or maintenance record not found and check if the staff position is not 'mechanic'

Use in trigger 1 : to check if not enough stock in part

```
CREATE OR REPLACE PROCEDURE prc_manage_maintenance_staff (
    p_maintenance_id IN Maintenance.MaintenanceID%TYPE,
    p_staff_id       IN Staff.StaffID%TYPE,
    p_role           IN VARCHAR2,
    p_worked_hours   IN NUMBER
)
IS
    v_maintenance_count  NUMBER;
    v_staff_position     Staff.Position%TYPE;

    MAINTENANCE_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(MAINTENANCE_NOT_FOUND, -20003);

    STAFF_NOT_FOUND EXCEPTION;
    PRAGMA EXCEPTION_INIT(STAFF_NOT_FOUND, -20004);


    STAFF_NOT_A_MECHANIC EXCEPTION;
    PRAGMA EXCEPTION_INIT(STAFF_NOT_A_MECHANIC, -20005);

BEGIN

    SELECT COUNT(*)
    INTO v_maintenance_count
    FROM Maintenance
    WHERE MaintenanceID = p_maintenance_id;

    IF v_maintenance_count = 0 THEN
        RAISE MAINTENANCE_NOT_FOUND;
    END IF;


    BEGIN
        SELECT Position
        INTO v_staff_position
        FROM Staff
        WHERE StaffID = p_staff_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE STAFF_NOT_FOUND;
    END;


    IF UPPER(v_staff_position) != 'MECHANIC' THEN
        RAISE STAFF_NOT_A_MECHANIC;
    END IF;
```

```
    INSERT INTO StaffAllocation (
        StaffID,
        MaintenanceID,
        Role,
        WorkedHour
    ) VALUES (
        p_staff_id,
        p_maintenance_id,
        p_role,
        p_worked_hours
    );

    DBMS_OUTPUT.PUT_LINE('Staff ' || p_staff_id || ' has been successfully
allocated to maintenance task ' || p_maintenance_id || '.');

EXCEPTION
    WHEN MAINTENANCE_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Maintenance ID ' || p_maintenance_id
|| ' not found.');
    WHEN STAFF_NOT_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Staff ID ' || p_staff_id || ' not
found.');
    WHEN STAFF_NOT_A_MECHANIC THEN
        DBMS_OUTPUT.PUT_LINE('Error: Staff ' || p_staff_id || ' is a ' ||
v_staff_position || ', not a Mechanic. Allocation failed.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
        ROLLBACK;
END;
/
```

```
SET SERVEROUTPUT ON;
SET LINESIZE 200;
SET PAGESIZE 100;

-- Add this line to fix the display
COLUMN UNITPRICE FORMAT 99999.99
PROMPT Creating/Replacing trigger for Type 2 SCD Part table...
DROP TRIGGER trg_update_scd_stock_and_cost;
CREATE OR REPLACE TRIGGER trg_update_scd_stock_and_cost
AFTER INSERT ON Using_Parts
FOR EACH ROW
DECLARE
    v_current_stock NUMBER;
    v_part_unit_price NUMBER(10, 2);
BEGIN
    -- Step 1: Lock the specific part row (the specific batch) to get its
    -- current stock and unit price. This now uses the composite key.
    SELECT StockQuantity, UnitPrice
    INTO v_current_stock, v_part_unit_price
    FROM Part
    WHERE PartKey = :NEW.PartKey AND PartID = :NEW.PartID -- Using the
composite key
    FOR UPDATE;

    -- Step 2: Check if there is enough stock in this specific batch.
    IF v_current_stock < :NEW.QuantityUsed THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: Insufficient stock for
PartID ' || :NEW.PartID || ' (Batch/PartKey: ' || :NEW.PartKey || ').
Available: ' || v_current_stock || ', Needed: ' || :NEW.QuantityUsed);
    ELSE
        -- Step 3: Update the stock for that specific batch only.
        UPDATE Part
        SET StockQuantity = StockQuantity - :NEW.QuantityUsed
        WHERE PartKey = :NEW.PartKey AND PartID = :NEW.PartID; --
```

```
Targeting the specific row

        -- Step 4: Add the cost of the used part(s) from this batch to the
Maintenance record.
        -- Note: The column in Maintenance is named "Cost", not
"MaintenanceCost".
        UPDATE Maintenance
        SET Cost = Cost + (v_part_unit_price * :NEW.QuantityUsed)
        WHERE MaintenanceID = :NEW.MaintenanceID;

        DBMS_OUTPUT.PUT_LINE('SUCCESS: Trigger fired. Stock and cost
updated for batch (PartKey): ' || :NEW.PartKey);
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- Handles cases where the composite key (PartKey, PartID) does
not exist.
        RAISE_APPLICATION_ERROR(-20002, 'Error: The specified part batch
(PartKey ' || :NEW.PartKey || ', PartID ' || :NEW.PartID || ') does not
exist.');
END;
/
```

5.5.5 Sequence creation

Sequence use in part audit table

```
CREATE SEQUENCE audit_log_seq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
```

5.5.6 Output Formatting

Formatting in both query (title and column formatting)

```
SET LINESIZE 100
SET PAGESIZE 35
SET FEEDBACK OFF
CL SCR

-- Set a title for the report
TTITLE center 'Top 10 Most Popular Bus Routes by Occupancy Rate' SKIP 2

-- Define the format for each column
COLUMN "Rank"           FORMAT 999 HEADING 'Rank'
COLUMN "Route"          FORMAT A50 HEADING 'Route'
COLUMN "TicketsSold"    FORMAT 999,999 HEADING 'Tickets|Sold'
COLUMN "TotalCapacity"  FORMAT 999,999 HEADING 'Total|Capacity'
COLUMN "Occupancy Rate" FORMAT A20 HEADING 'Occupancy Rate'
```

```
SET LINESIZE 120
SET PAGESIZE 100
CL SCR

-- Set a dynamic title for the report including the current date
TTITLE LEFT 'Monthly Order Summary Report as of ' _DATE  SKIP 2

-- Define the format for each column from your query
COLUMN OrderMonth       FORMAT A20    HEADING 'Order Month'
COLUMN NumberOfOrders   FORMAT 999,999 HEADING 'Number of Orders'
COLUMN TotalMonthlyCost FORMAT $99,999,999.99 HEADING 'Total Monthly Cost'
```